# Continuum Onboarding Guide

A Comprehensive Guide to Understanding, Running,
and Extending the Continuum Knowledge Graph System

Ali Shehral

&

Claudius Opus IV · VI

Khoury College of Computer Sciences
Northeastern University

April 2026

# Contents

# Part I

# Foundations

# Chapter 1

# Knowledge Graphs

*This chapter introduces the graph data model that underpins Continuum. You will learn why graphs are a natural fit for storing architectural decisions, how labeled property graphs work, how to read and write Cypher queries, and how Continuum maps its domain—decisions, entities, and the relationships between them—onto Neo4j.*

**⇒ Skip Ahead**

If you have worked with Neo4j or any other labeled property graph database, you can skip to Chapter 2. You may still want to skim Section 1.4 to familiarize yourself with Continuum's specific node labels and relationship types.

## 1.1 Relational vs. Graph Databases

Most developers learn to think in tables. A relational database stores records in rows, groups rows into tables, and connects tables through foreign keys. When you need to answer a question that spans several tables, you write JOINs. For many workloads this is the right tool, and Continuum uses PostgreSQL for exactly those workloads—user accounts, sessions, API keys.

But consider a different kind of question: *"Which architectural decisions involved PostgreSQL?"* In a relational schema you would need at least three tables—`decisions`, `entities`, and a join table `decision_entities`—and two JOIN operations to answer the query. As the number of hops grows ("which decisions *superseded* decisions that involved PostgreSQL?"), the SQL becomes deeply nested and expensive.

A graph database stores the same information differently. Entities and decisions become **nodes**; the fact that a decision *involves* an entity becomes a directed **edge** between them. Traversing from a decision to its related entities is a single pointer hop, regardless of how many relationships exist in the database.

Figure 1.1 illustrates the difference. The graph on the right answers multi-hop questions naturally: start at a node, follow edges, collect results.

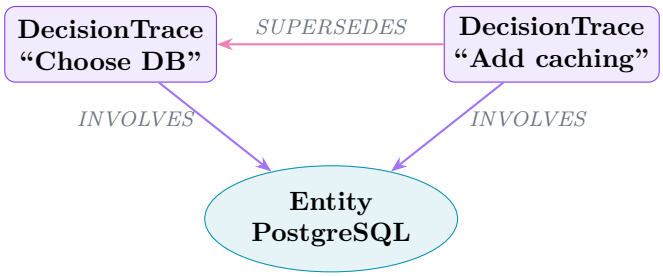


Figure 1.1: Two `DecisionTrace` nodes connected to a shared `Entity` node via `INVOLVES` edges. One decision supersedes the other. In a relational schema, answering "which decisions share an entity?" requires a self-join through the join table.

**⋆ Key Insight**

Graphs excel when the *relationships* between records are as important as the records themselves. In Continuum, the fact that one decision supersedes another, or that two entities are alternatives, carries as much meaning as the decisions and entities individually.

## 1.2 Labeled Property Graphs

Neo4j implements the **labeled property graph** model. Every element in the graph—node or relationship—can carry:

- **Labels** (nodes only). A node can have one or more labels that describe its role: `DecisionTrace`, `Entity`.
- **A type** (relationships only). Each relationship has exactly one type: `INVOLVES`, `SUPERSEDES`, `IS_A`, etc.
- **Properties**. Key–value pairs attached to nodes or relationships. A `DecisionTrace` node might have properties `trigger`, `context`, `decision`, and `confidence`.
- **Direction**. Every relationship has a start node and an end node. `(d:DecisionTrace)-[:INVOLVES]->(e:Entity)` reads as "decision $d$ involves entity $e$."

This model is flexible: you do not need to declare a schema upfront. If a new property appears on a node, Neo4j stores it without an `ALTER TABLE`. However, Continuum enforces its own conventions through application code so that the graph remains consistent (see Section 1.4).

## 1.3 Cypher Basics

Cypher is Neo4j's query language. Its syntax uses ASCII art to describe graph patterns: parentheses for nodes, square brackets for relationships, and arrows for direction. This section covers the four query patterns you will use most often when working with Continuum.

### 1.3.1 Finding decisions about an entity

The most common query retrieves all decisions that involve a particular entity:

```
MATCH (d:DecisionTrace)-[:INVOLVES]->(e:Entity)
WHERE e.name = 'PostgreSQL'
RETURN d
```

Read the pattern left to right: "find a `DecisionTrace` node `d` that has an `INVOLVES` relationship pointing to an `Entity` node `e`, where the entity's name is PostgreSQL."

### 1.3.2 Finding decision evolution

Decisions evolve over time. When a new decision replaces an older one, Continuum creates a `SUPERSEDES` edge. To find these chains:

```
MATCH (d1:DecisionTrace)-[:SUPERSEDES]->(d2:DecisionTrace)
RETURN d1, d2
```

This returns pairs where `d1` supersedes `d2`. To follow longer chains, you can use variable-length paths: `(d1)-[:SUPERSEDES*1..3]->(d2)`.

### 1.3.3 Creating an entity

New entities are created with `CREATE`:

```
CREATE (e:Entity {name: 'Redis', type: 'technology'})
```

In practice, Continuum's entity resolution pipeline (Chapter 2) handles entity creation to avoid duplicates. You rarely write raw `CREATE` statements for entities.

### 1.3.4 User-scoped queries

Every `DecisionTrace` carries a `user_id` property. To count how many decisions a specific user has recorded:

```
MATCH (d:DecisionTrace)
WHERE d.user_id = $user_id
RETURN count(d)
```

The `$user_id` syntax denotes a **parameter**—a value supplied at runtime rather than hardcoded in the query string. Parameters prevent Cypher injection and enable query plan caching.

**△ Warning**

Every Neo4j query in Continuum **must** include `user_id` filtering. If you omit it, one user's queries can return another user's decisions. The backend enforces this in its query-building layer, but if you write raw Cypher for debugging or evaluation, always include the `WHERE d.user_id = $user_id` clause.

## 1.4 Continuum's Graph Model

Continuum uses two node labels and fourteen relationship types. This section documents the full model.

### 1.4.1 DecisionTrace nodes

A `DecisionTrace` captures a single architectural decision extracted from a developer–AI conversation. Its properties are:

Table 1.1: Properties of a `DecisionTrace` node.

| Property | Type | Description |
|---|---|---|
| `trigger` | string | What prompted the decision (e.g., "need a caching layer") |
| `context` | string | Background information and constraints |
| `options` | string | Alternatives that were considered |
| `decision` | string | The option that was chosen |
| `rationale` | string | Why this option was selected over others |
| `confidence` | float | Model's self-assessed confidence (0–1) |
| `user_id` | string | Owner of this decision (data isolation key) |

### 1.4.2 Entity nodes

An `Entity` represents a technology, concept, pattern, or tool referenced by one or more decisions:

Table 1.2: Properties of an `Entity` node.

| Property | Type | Description |
|---|---|---|
| `name` | string | Canonical name (e.g., "PostgreSQL") |
| `type` | string | Category (e.g., "technology", "pattern", "tool") |
| `aliases` | string[] | Known alternative names (e.g., "postgres", "PG") |
| `embedding` | float[] | Vector embedding for semantic search (2048 dimensions) |

### 1.4.3 Relationship types

Table 1.3 lists all fourteen relationship types in Continuum's graph model, grouped by the node labels they connect.

Table 1.3: Relationship types in Continuum's graph model.

| Type | Connects | Meaning |
|---|---|---|
| `INVOLVES` | Decision → Entity | Decision references this entity |
| `IS_A` | Entity → Entity | Taxonomic (Redis *is a* datastore) |
| `PART_OF` | Entity → Entity | Composition (pg_hba.conf *part of* PostgreSQL) |
| `DEPENDS_ON` | Entity → Entity | Runtime dependency |
| `RELATED_TO` | Entity → Entity | General association |
| `ALTERNATIVE_TO` | Entity → Entity | Substitutable (Redis *alternative to* Memcached) |
| `ENABLES` | Entity → Entity | One entity makes another possible |
| `PREVENTS` | Entity → Entity | One entity blocks or conflicts with another |
| `REQUIRES` | Entity → Entity | Hard prerequisite |
| `REFINES` | Entity → Entity | Specialization or narrowing |
| `SIMILAR_TO` | Decision → Decision | Decisions address analogous concerns |
| `INFLUENCED_BY` | Decision → Decision | One decision shaped another |
| `SUPERSEDES` | Decision → Decision | Newer decision replaces older one |
| `CONTRADICTS` | Decision → Decision | Decisions are in tension |

**⋆ Key Insight**

Every Neo4j query in Continuum **must** include `user_id` filtering. Missing this is a data isolation bug—one user's decisions would leak into another user's results. The backend enforces this constraint in its query-building layer, but you must enforce it yourself when writing raw Cypher for debugging or evaluation scripts.

### ▷ Try It

Open the Neo4j browser at `http://localhost:7474` and run the following query to see all relationship types currently in your graph:

```
CALL db.relationshipTypes() YIELD relationshipType
RETURN relationshipType
```

Compare the output with Table 1.3. If you have run Continuum's extraction pipeline, you should see a subset of these types depending on what decisions were captured.

# Chapter 2

# Entity Resolution

*This chapter explains why the same real-world concept can appear under many different names, why resolving those names to a single canonical entity is essential for a useful knowledge graph, and how Continuum approaches the problem with a multi-stage cascading pipeline. You will also learn how to measure resolution quality using B-cubed metrics.*

**⇒ Skip Ahead**

If you have built deduplication or record linkage pipelines before, you can skip to Chapter 3. You may still want to read Section 2.4 for the specific evaluation methodology Continuum uses.

## 2.1 The Problem

Developers do not use consistent terminology. In a single coding session, the same database might be called "postgres," "PostgreSQL," "PG," or "Postgres 16." If Continuum's extraction pipeline creates a new `Entity` node for each surface form, the graph ends up with four disconnected nodes that all represent the same technology. Queries that should return a unified view of PostgreSQL-related decisions instead return partial, fragmented results.

Entity resolution is the process of determining when two or more mentions refer to the same real-world entity and merging them into a single canonical node. Without it, the knowledge graph is fractured; with it, the graph becomes a reliable map of a developer's architectural landscape.

## 2.2 Why It's Hard

Entity resolution in developer tooling is harder than it might appear at first glance. Several factors compound the difficulty:

- **No enforced schema.** Unlike a product catalog where every item has a SKU, developer conversations have no controlled vocabulary. New technologies appear constantly.
- **Inconsistent naming.** The same entity is referred to by full names, abbreviations, version-qualified names, and slang: "kubernetes," "K8s," "k8s," "Kube."
- **Abbreviations and acronyms.** "PG" could mean PostgreSQL, Procter & Gamble, or a movie rating. "RDS" could be Amazon RDS or Remote Desktop Services.
- **Context dependence.** "Spring" is a Java framework in a backend discussion but a season in a scheduling discussion. "Go" is a programming language or a verb.
- **Compound terms.** "React Native" is not "React" plus "Native." "Next.js App Router" is a specific feature, not a generic router.

**△ Warning**

Naive approaches that rely solely on string similarity will merge entities that should remain separate. "React" and "React Native" have high string overlap but are fundamentally different technologies. Continuum's pipeline uses semantic context, not just string distance, to handle these cases.

## 2.3 Approaches to Entity Resolution

There is no single best algorithm for entity resolution—every approach trades off speed, accuracy, and cost differently. Table 2.1 summarizes the four main strategies.

Table 2.1: Comparison of entity resolution approaches.

| Approach | Speed | Accuracy | Tradeoff |
|---|---|---|---|
| Exact matching | Fast | Brittle | Only matches identical strings; misses abbreviations, typos, and case variants |
| Fuzzy matching | Fast | Moderate | Handles typos and minor variations; produces false positives on short strings ("Go" ≈ "Go2") |
| Embedding similarity | Slow | High | Captures semantic meaning; requires an embedding model and vector index |
| Hybrid cascading | Fast (amortized) | High | Tries cheap stages first, falls through to expensive stages only when needed (Continuum's approach) |

Continuum uses the hybrid cascading strategy. Its seven-stage pipeline is ordered from cheapest to most expensive:

1. **Cache lookup** — Check Redis for a recently resolved mention (5-minute TTL).
2. **Exact match** — Case-insensitive string comparison against existing entity names.
3. **Canonical lookup** — Check a curated mapping of 534 known aliases (e.g., `postgres` → `PostgreSQL`).
4. **Alias search** — Search the `aliases` array on existing `Entity` nodes.
5. **Fuzzy match** — Fulltext-accelerated fuzzy matching using RapidFuzz with an 85% similarity threshold.
6. **Embedding similarity** — Cosine similarity against entity embeddings (threshold $> 0.9$).
7. **Create new** — If no match is found, create a new `Entity` node.

The pipeline short-circuits: once a stage finds a match, later stages are skipped entirely.

**⋆ Key Insight**

Continuum's pipeline resolves approximately 83% of mentions in the first three stages (cache, exact, canonical) without calling the LLM or embedding service. This means the vast majority of resolutions are sub-millisecond lookups, and the expensive AI-powered stages are reserved for genuinely ambiguous cases.

## 2.4 B-Cubed Metrics

How do you measure whether an entity resolution system is working well? Traditional clustering metrics like MUC and CEAF have known blind spots—MUC ignores singleton entities (mentions that map to a unique entity with no duplicates), and CEAF can produce misleading scores when cluster sizes are uneven. Continuum uses **B-cubed** metrics, which evaluate precision and recall at the level of individual mentions and naturally handle singletons.

### 2.4.1 Definitions

Given a set of mentions, let each mention $m$ belong to a *ground-truth cluster* (the correct grouping) and a *system cluster* (the grouping produced by the resolver).

- **B-cubed precision** for a mention $m$: of all mentions in $m$'s system cluster, what fraction shares $m$'s ground-truth cluster?
- **B-cubed recall** for a mention $m$: of all mentions in $m$'s ground-truth cluster, what fraction shares $m$'s system cluster?

Overall precision and recall are the averages across all mentions.

### 2.4.2 Worked example

Consider five mentions and their ground-truth clusters:

Table 2.2: Ground truth and system output for five mentions.

| Mention | Ground Truth | System Output |
|---|---|---|
| postgres | Cluster A | Cluster 1 |
| PostgreSQL | Cluster A | Cluster 1 |
| PG | Cluster A | Cluster 2 (singleton) |
| MongoDB | Cluster B | Cluster 3 |
| Mongo | Cluster B | Cluster 3 |

**Ground truth:** {postgres, PostgreSQL, PG} and {MongoDB, Mongo}.
**System output:** {postgres, PostgreSQL}, {PG}, and {MongoDB, Mongo}.

The system correctly groups MongoDB and Mongo but splits PG into its own cluster, missing that it belongs with postgres and PostgreSQL.

**Per-mention precision.** For each mention, count how many items in its system cluster share the same ground-truth cluster, then divide by the system cluster size.

- **postgres**: system cluster = {postgres, PostgreSQL}, both in ground-truth A. Precision = $2/2 = 1.0$.
- **PostgreSQL**: same cluster, same reasoning. Precision = 1.0.
- **PG**: system cluster = {PG}, singleton. Precision = $1/1 = 1.0$.
- **MongoDB**: system cluster = {MongoDB, Mongo}, both in ground-truth B. Precision = $2/2 = 1.0$.
- **Mongo**: same. Precision = 1.0.

**Average precision** = $(1.0 + 1.0 + 1.0 + 1.0 + 1.0)/5 = 1.0$.

**Per-mention recall.** For each mention, count how many items in its ground-truth cluster share the same system cluster, then divide by the ground-truth cluster size.

- **postgres**: ground-truth A = {postgres, PostgreSQL, PG}. In system cluster 1: postgres, PostgreSQL (2 of 3). Recall = 2/3.
- **PostgreSQL**: same ground-truth A, same system cluster. Recall = 2/3.
- **PG**: ground-truth A has 3 members; PG's system cluster contains only PG from A. Recall = 1/3.
- **MongoDB**: ground-truth B = {MongoDB, Mongo}. Both in system cluster 3. Recall = 2/2 = 1.0.
- **Mongo**: same. Recall = 1.0.

**Average recall** $= (2/3 + 2/3 + 1/3 + 1.0 + 1.0)/5 = (0.667 + 0.667 + 0.333 + 1.0 + 1.0)/5 = 0.733$.

The system achieves perfect precision (it never incorrectly groups unrelated mentions together) but imperfect recall (it failed to recognize that "PG" is an alias for PostgreSQL). This pattern—high precision, lower recall on abbreviations—is typical of conservative resolution pipelines.

**⋆ Key Insight**

B-cubed metrics evaluate at the mention level, which means every mention contributes equally to the score. A system that perfectly resolves 100 easy mentions but fragments 5 hard ones will show a measurable recall drop. This makes B-cubed particularly sensitive to the kinds of abbreviation and alias errors that matter most in practice.

**▷ Try It**

Manually compute B-cubed scores for a system that produces a single cluster containing all five mentions: {postgres, PostgreSQL, PG, MongoDB, Mongo}. What happens to precision? What happens to recall? This over-merging scenario is the dual failure mode to the under-splitting shown above.

# Chapter 3

# Retrieval-Augmented Generation

*This chapter explains why large language models hallucinate, how retrieval-augmented generation (RAG) grounds their responses in evidence, why hybrid retrieval outperforms either lexical or semantic search alone, and how Continuum extends basic RAG into GraphRAG by retrieving subgraphs instead of flat documents.*

**⇒ Skip Ahead**

If you have built RAG pipelines before, you can skip to Part I. You may still want to read Section 3.4 to understand how Continuum extends standard RAG with graph-structured retrieval and k-hop expansion.

## 3.1 The Hallucination Problem

Large language models generate text by predicting the most probable next token given the preceding context. This makes them fluent, but it also means they can produce text that is *plausible but wrong*. A model asked "What database did we choose for the caching layer?" might confidently answer "Redis" even if the team actually chose Memcached—or never made a caching decision at all.

This failure mode is called **hallucination**: the model fabricates facts that are not supported by any evidence. In a system that tracks architectural decisions, hallucinations are particularly dangerous because they can lead developers to act on decisions that were never actually made.

RAG addresses this by adding a retrieval step before generation. Instead of relying solely on the model's parametric memory (the knowledge baked into its weights during training), the system first searches a knowledge base for relevant evidence, injects that evidence into the prompt, and asks the model to generate an answer grounded in the retrieved content.

## 3.2 Basic RAG

A minimal RAG pipeline has four stages:

1. **Embed the query.** Convert the user's natural-language question into a vector using an embedding model.
2. **Search the index.** Find the top-$K$ most similar documents (or document chunks) by comparing the query vector against a vector index.
3. **Build the prompt.** Inject the retrieved documents into the LLM's context window alongside the original question.
4. **Generate a grounded answer.** The LLM produces a response that cites or draws on the retrieved evidence rather than relying on its parametric memory alone.

This approach works well when the knowledge base consists of flat documents (articles, wiki pages, code comments). But architectural decisions are not flat documents—they are connected through entities, supersession chains, and influence relationships. Basic RAG retrieves documents in isolation, missing the relational context that makes decisions meaningful.

## 3.3 Hybrid Retrieval

Before discussing graph-aware retrieval, it is worth understanding why Continuum combines two search strategies instead of relying on one.

### 3.3.1 Lexical search

Lexical (keyword or fulltext) search matches documents that contain the exact terms in the query. A search for "PostgreSQL" will find every document containing that string. Lexical search is fast, deterministic, and excels at matching specific identifiers, error codes, and proper names.

Its weakness is that it cannot bridge vocabulary gaps. A query for "database choice" will not match a document that says "PostgreSQL selection" unless both terms happen to co-occur.

### 3.3.2 Semantic search

Semantic (vector) search embeds both the query and documents into a shared vector space and retrieves by cosine similarity. A query for "database choice" will find documents about "PostgreSQL selection" because the embeddings capture conceptual proximity.

Its weakness is the inverse of lexical search: it can miss exact terminology. A search for "pg_hba.conf" might return documents about PostgreSQL authentication in general rather than the specific configuration file.

### 3.3.3 Reciprocal Rank Fusion

Continuum runs both searches in parallel and combines the results using **Reciprocal Rank Fusion** (RRF). For a document $d$ that appears at position $r_{\text{lex}}$ in the lexical results and $r_{\text{sem}}$ in the semantic results, its fused score is:

$$\text{Score}(d) = \frac{1}{k + r_{\text{lex}}} + \frac{1}{k + r_{\text{sem}}}$$

where $k$ is a smoothing constant (typically 60). Documents that rank highly in both lists receive the highest fused scores; documents that rank highly in only one list still appear but with lower priority.

**△ Warning**

If a document does not appear in one of the two result lists, its rank for that list is treated as $\infty$, making its contribution from that list effectively zero. RRF degrades gracefully—it never penalizes a document for being absent from one list.

**⋆ Key Insight**

Continuum's ablation study shows that removing fulltext search drops recall to 13.3%. Hybrid retrieval is essential, not optional. Semantic search alone misses exact entity names and technical identifiers that lexical search catches trivially.

## 3.4 GraphRAG

Standard RAG retrieves flat documents. Continuum's **GraphRAG** pipeline retrieves *subgraphs*—interconnected sets of decisions and entities that provide relational context no flat document can capture.

### 3.4.1 Why subgraphs matter

Consider a question: "Why did we choose PostgreSQL over MongoDB?" A flat retrieval system might find the decision that mentions PostgreSQL. But it would miss the entity node for MongoDB (which is connected via `ALTERNATIVE_TO`), the earlier decision that was superseded (connected via `SUPERSEDES`), and the downstream decisions that depend on this choice (connected via `INFLUENCED_BY`).

By retrieving the subgraph around the initial matches, GraphRAG assembles a richer context that includes:

- The seed decision(s) that directly match the query.
- Related entities and their types (`IS_A`, `PART_OF`, `DEPENDS_ON`).
- Decision chains (`SUPERSEDES`, `CONTRADICTS`, `INFLUENCED_BY`).
- Alternative technologies (`ALTERNATIVE_TO`).

### 3.4.2 The pipeline

Figure 15.1 illustrates Continuum's GraphRAG pipeline end to end.

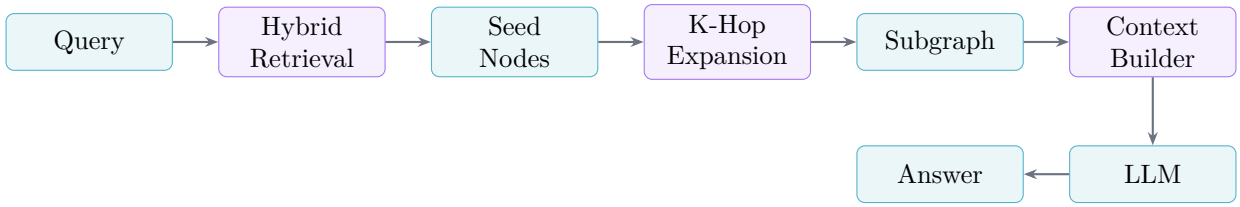


Figure 3.1: Continuum's GraphRAG pipeline. A natural-language query is matched against the graph using hybrid retrieval (lexical + semantic). The resulting seed nodes are expanded by $k$ hops to pull in relational context. The subgraph is serialized into a text context and passed to the LLM for grounded answer generation.

The pipeline proceeds in five steps:

1. **Hybrid retrieval.** The query is matched against the graph using both fulltext (lexical) and vector (semantic) search, fused with RRF as described in Section 3.3.
2. **Seed nodes.** The top-$K$ results become *seed nodes*: the starting points for graph expansion.
3. **K-hop expansion.** From each seed node, the pipeline traverses outward by $k$ hops (typically $k = 1$ or $k = 2$), collecting all connected nodes and relationships. This pulls in entities referenced by the decision, decisions that supersede or contradict it, and related technologies.
4. **Context building.** The retrieved subgraph is serialized into a structured text representation that the LLM can process: decision summaries, entity names, and relationship descriptions.
5. **Grounded generation.** The LLM receives the query and the serialized subgraph context, then generates an answer that references the retrieved evidence.

### 3.4.3 Seed nodes vs. expanded context

A common question is why not simply retrieve more seed nodes instead of expanding by hops. The answer is that more seed nodes give you more *independent* matches, while hop expansion gives you *connected* context around each match. A decision about PostgreSQL is more useful when you also see the entities it involves and the decisions it supersedes—information that would not appear as independent search results because it is not textually similar to the query.

**▷ Try It**

Run a GraphRAG query against your Continuum instance and examine the returned sources. Notice how the answer references not just the directly matched decision but also entities and related decisions pulled in by k-hop expansion:

```
curl -X POST http://localhost:8000/api/ask \
  -H "Authorization: Bearer $TOKEN" \
  -H "Content-Type: application/json" \
  -d '{"question": "Why did we choose PostgreSQL?"}'
```

**⋆ Key Insight**

GraphRAG's advantage over flat RAG is structural: it retrieves not just documents that match the query but the *neighborhood* of those documents in the graph. This relational context—which entities are involved, which decisions superseded others, what alternatives were considered—is precisely the information needed to answer architectural questions accurately.

# Part II

# What Is Continuum

# Chapter 4

# The Problem

*Every day, millions of developers sit down with an AI assistant and make dozens of consequential decisions—choosing databases, designing APIs, selecting architectures. The code that results from these conversations survives. The reasoning that produced it does not.*

## 4.1 AI-Assisted Development

Software development in 2025 looks fundamentally different from even two years ago. Over 90% of professional developers report using AI coding assistants at least weekly, and tools like GitHub Copilot, Cursor, and Claude Code have become as ubiquitous as version control itself. A typical coding session now involves dozens of conversational exchanges: the developer describes intent, the assistant proposes options, and together they iterate toward a solution.

Each of these exchanges carries information that traditional software engineering artifacts were never designed to capture. A single afternoon session might involve:

- Evaluating three caching strategies before choosing Redis with a TTL-based invalidation scheme.
- Deciding to use PostgreSQL over MongoDB after discussing ACID requirements.
- Rejecting a microservices split in favor of a modular monolith after weighing operational complexity.
- Choosing FastAPI over Flask because of native async support and automatic OpenAPI generation.

These are not trivial preferences—they are *architectural decisions* with months-long consequences. And they happen entirely inside a conversation window.

**⋆ Key Insight**

AI-assisted development has made decision-making more explicit than ever. Developers literally *write out* their reasoning when prompting an assistant. The problem is not that rationale is missing—it is that we throw it away.

## 4.2 The Rationale Gap

When the conversation window closes, the rationale evaporates. What remains is code: functions, configuration files, dependency manifests. The code tells you *what* was built, but not *why*.

Three months later, a new team member opens a pull request and asks:

> "Why did we choose PostgreSQL over MongoDB for the user service? The access patterns look document-oriented."

Nobody remembers. The original developer has moved to another project. The conversation where the trade-offs were weighed is buried in a chat log that nobody indexes, searches, or even

knows exists. The new developer makes a reasonable but uninformed decision to migrate—and re-introduces the very problem the original choice was designed to prevent.

This is the **rationale gap**: the systematic loss of decision context that occurs when the medium of decision-making (conversation) is disconnected from the medium of record (code and documentation).

△ **Warning**

The rationale gap is not merely inconvenient. It leads to repeated architectural mistakes, contradictory decisions across teams, and wasted engineering effort re-deriving conclusions that were already reached.

## 4.3 Existing Approaches

The software engineering community has tried several approaches to close the rationale gap. None have succeeded at scale.

Table 4.1: Existing approaches to capturing decision rationale.

| Approach | Auto? | Rationale? | Search? | Agent? | Problem |
|---|---|---|---|---|---|
| ADRs | No | Yes | Partially | No | <5% adoption; manual effort |
| Commit msgs | No | Rarely | Yes | No | Captures *what*, not *why* |
| Documentation | No | Sometimes | Yes | No | Perpetually stale |
| Slack / Chat | No | Yes | Barely | No | Ephemeral, unsearchable |

**Architecture Decision Records (ADRs)** are the gold standard in theory. Each ADR documents a single decision with its context, options considered, and rationale. In practice, fewer than 5% of engineering organizations maintain ADRs consistently. They require manual effort at precisely the moment when developers are least motivated to write documentation—right after making the decision and wanting to move on to implementation.

**Commit messages** are automated in the sense that every commit requires one, but they overwhelmingly describe *what* changed rather than *why*.

**Documentation** captures rationale when written well, but documentation is notoriously difficult to keep synchronized with the codebase. The half-life of accurate documentation in a fast-moving project is measured in weeks.

**Slack and chat logs** often contain the richest rationale, but they are buried in ephemeral message streams, unsearchable by future developers, and completely inaccessible to AI agents.

⋆ **Key Insight**

Every existing approach either requires manual effort (which kills adoption) or captures the wrong information (what changed, not why). The challenge is finding a source that is both *automatic* and *rationale-rich*.

## 4.4 The Insight

The insight behind Continuum is deceptively simple:

*The AI conversation **is** the decision record.*

When a developer discusses options with an AI assistant, the conversation naturally contains every element of a well-written ADR:

1. **Trigger** — what prompted the decision.
2. **Context** — background constraints.
3. **Options considered** — alternatives discussed.
4. **Decision** — what was chosen.
5. **Rationale** — why it was chosen.

This information does not need to be *created*—it already exists. It just needs to be *extracted* and *structured*.

That is the core thesis of Continuum: passively capture AI coding conversations, extract decision traces using a large language model, resolve the entities mentioned into a canonical knowledge graph, and make the result searchable by both humans and AI agents. No manual effort. No behavior change. The developer keeps working exactly as they already do.

## 4.5 Worked Example

To make this concrete, consider the following exchange between a developer and an AI coding assistant (Listing 4.1).

Listing 4.1: Developer–AI conversation about database selection.

```
Developer: I need to select a database for a new
  transaction processing service on our fintech platform.
  Requirements: strong consistency, ACID transactions,
  and complex queries on financial records.

Assistant: For a transaction-heavy financial service,
  the main contenders would be PostgreSQL, MongoDB, and
  CockroachDB. Let me compare them:
  - PostgreSQL: Mature ACID support, rich SQL, excellent
    for complex joins on financial data.
  - MongoDB: Flexible schema, but multi-document
    transactions add overhead and complexity.
  - CockroachDB: Distributed ACID, but adds operational
    complexity you may not need yet.
  Given your emphasis on ACID compliance and complex
  queries, PostgreSQL is the strongest fit.

Developer: Good point about MongoDB transactions.
  We also need PostGIS later for location-based fraud
  detection. Going with PostgreSQL.
```

From this 20-line exchange, Continuum extracts a structured **decision trace** (Table 9.1).

Each entity mentioned—PostgreSQL, MongoDB, CockroachDB, PostGIS—is resolved against Continuum's knowledge graph using the seven-stage entity resolution pipeline (Chapter 12). The decision node is linked to these entities via `INVOLVES` relationships, making it discoverable through graph traversal.

Table 4.2: Decision trace extracted by Continuum from Listing 4.1.

| **Field** | **Extracted Value** |
|---|---|
| Trigger | Need to select database for new service |
| Context | Building a transaction-heavy financial API |
| Options | PostgreSQL, MongoDB, CockroachDB |
| Decision | Use PostgreSQL |
| Rationale | ACID compliance critical for financial transactions; PostGIS needed for future fraud detection |
| Confidence | 0.92 |

**⋆ Key Insight**

A single short conversation produced a complete decision record—trigger, context, alternatives, choice, and rationale—without the developer writing any documentation at all. This is the power of treating conversations as first-class engineering artifacts.

# Chapter 5

# System Overview

*Continuum is a full-stack system that spans three tiers, five subsystems, and three databases. This chapter maps the terrain so that the deep dives in Part III have a frame of reference.*

## 5.1 Architecture

At the highest level, Continuum follows a three-tier architecture: a React-based frontend, a Python API server, and a persistence layer composed of three specialized databases plus a caching tier. Figure 5.1 shows the major components and the protocols that connect them.

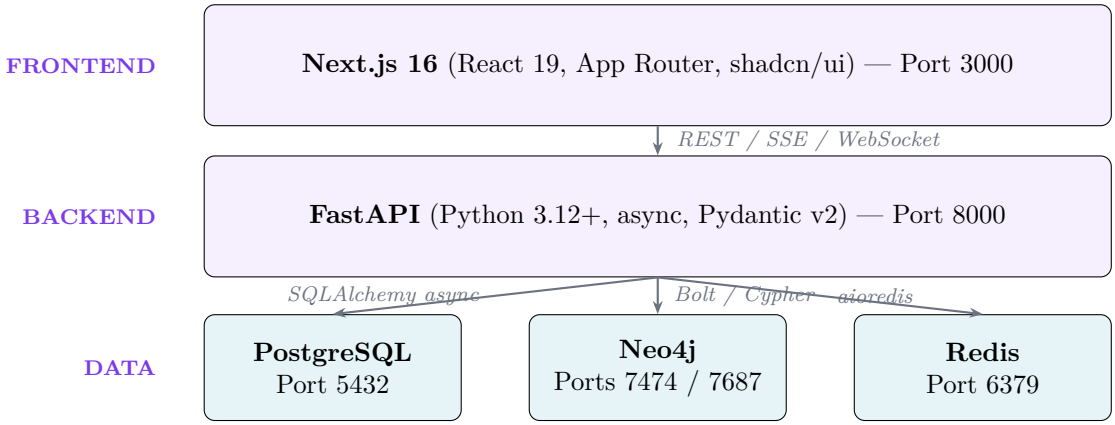


Figure 5.1: Three-tier architecture of Continuum.

The frontend communicates with the backend over three protocols: standard REST for CRUD operations, Server-Sent Events (SSE) for streaming AI responses in the Ask interface, and WebSocket for real-time conversation capture. The backend talks to three databases, each chosen for a specific strength discussed in Section 5.5.

## 5.2 Five Subsystems

Continuum is organized into five subsystems, each responsible for one stage of the pipeline from raw conversation to searchable knowledge. Table 5.1 summarizes them.

**Capture** watches for new Claude Code conversation logs on disk and accepts real-time input via WebSocket. It does not interpret the content—it simply ensures that raw conversations reach the extraction stage.

**Extraction** sends each conversation to a 49-billion-parameter language model (Llama 3.3 Nemotron Super via NVIDIA NIM) with a structured prompt that requests decision traces in JSON format. The model identifies triggers, context, options, decisions, and rationale.

**Resolution** takes the raw entity mentions from extraction (e.g., “postgres”, “Postgres”, “PostgreSQL”) and resolves them to canonical graph nodes through a seven-stage pipeline that progresses

Table 5.1: The five subsystems of Continuum.

| Subsystem | Purpose | Key Files | Technologies |
|---|---|---|---|
| Capture | Passive log extraction, AI interviews | `capture.py` | WebSocket, watcher |
| Extraction | LLM decision trace extraction | `extractor.py` | NVIDIA NIM |
| Resolution | 7-stage entity deduplication | `entity_resolver.py` | RapidFuzz, Redis |
| Retrieval | Hybrid search + GraphRAG | `graph_rag.py` | Neo4j, RRF |
| Integration | AI agent access via MCP | `agent.py` | MCP (5 tools) |

from cheap checks (cache, exact match, canonical lookup) to expensive ones (fuzzy matching, embedding similarity).

**Retrieval** provides two search modes: keyword-based hybrid search (combining fulltext and vector similarity with Reciprocal Rank Fusion) and conversational GraphRAG that expands seed results into subgraphs for context-rich answers.

**Integration** exposes the knowledge graph to external AI agents via the Model Context Protocol (MCP), providing five tools that let agents search decisions, query entities, and explore relationships.

## 5.3 Data Flow

Figure 9.1 traces a single conversation from capture to queryable knowledge.

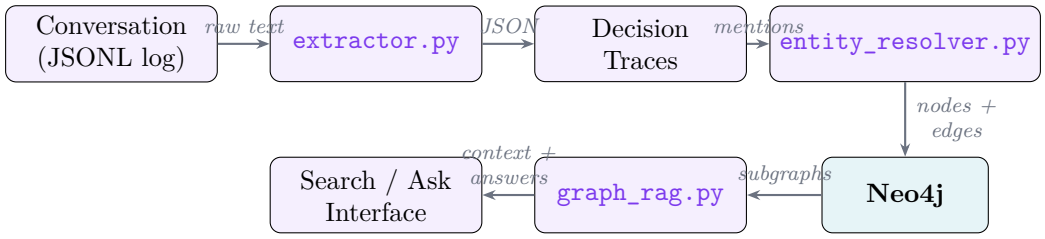


Figure 5.2: Data flow from raw conversation to queryable knowledge.

1. A **conversation** (stored as a JSONL log file) enters the system via file watcher or WebSocket.
2. The **extractor** sends the conversation text to the LLM, which returns structured decision traces as JSON.
3. Each trace contains raw entity **mentions** (e.g., "postgres", "FastAPI", "Redis").
4. The **entity resolver** maps each mention to a canonical node in Neo4j, creating new nodes only when no match is found.
5. The decision node and its `INVOLVES` edges are written to **Neo4j**.
6. The **GraphRAG** pipeline answers queries by finding seed nodes via hybrid search, expanding them into subgraphs, and synthesizing answers.

## 5.4 Multi-Tenant Architecture

Continuum enforces strict per-user isolation. Every decision, entity, and relationship in the knowledge graph is scoped by a `user_id` property. When a user queries the system, all Neo4j Cypher queries include a `WHERE n.user_id = $user_id` clause, ensuring that users see only their own decisions and entities.

This scoping is enforced at the API layer—the backend extracts the `user_id` from the JWT token and injects it into every database query. There is no shared namespace, no cross-user leakage, and no way for one user to access another user's knowledge graph.

**△ Warning**

Multi-tenancy is enforced at the application layer, not the database layer. A misconfigured query that omits the `user_id` filter would expose data across users. Production deployment would require additional database-level row security policies.

## 5.5 Tech Stack

Each technology in Continuum was chosen for a specific reason. Table 5.2 maps the major components to their justifications.

Table 5.2: Technology choices and their rationale.

| Technology | Role | Why This Choice |
|---|---|---|
| PostgreSQL 18 | Relational data + auth | Alembic migrations, async SQLAlchemy, mature ecosystem |
| Neo4j 2025.01 | Knowledge graph | Native graph traversal, Cypher queries, built-in full-text + vector indexes |
| Redis 7.4 | Cache + rate limiting | Three cache tiers (entity, LLM, query), token-bucket rate limiting |
| NVIDIA NIM | LLM + embeddings | 49B-param Nemotron for extraction, NV-EmbedQA (2048-dim) for similarity |
| Next.js 16 | Frontend | React 19, App Router, Server Components, shadcn/ui component library |
| FastAPI | Backend API | Native async, automatic OpenAPI docs, Pydantic validation |

**⋆ Key Insight**

A single decision touches all three databases plus Redis cache. A new PostgreSQL decision is stored relationally for auth context, as a Neo4j node with entity edges for graph traversal, and cached in Redis for fast retrieval. Understanding where data lives is essential for debugging.

**⇒ Skip Ahead**

If you want to run the system before understanding every component, skip to Chapter 7 for setup instructions.

# Chapter 6

# Related Systems & Landscape

*No system exists in a vacuum. This chapter positions Continuum among existing tools for code understanding, decision tracking, and knowledge retrieval—and is honest about where it stands today.*

## 6.1 Comparison Table

Table 6.1 compares Continuum against the categories of tools most frequently cited as alternatives.

Table 6.1: Landscape comparison across five capability dimensions.

| System Category | Auto Capture | Rationale | Graph | Agent Access | Entity Dedup |
|---|---|---|---|---|---|
| Architecture Decision Records | No | Yes | No | No | No |
| CodeQL / Sourcetrail | Yes | No | Yes | Partially | No |
| Conversation Analytics | Yes | Yes | No | No | No |
| General RAG Systems | Varies | No | No | Yes | No |
| Continuum | **Yes** | **Yes** | **Yes** | **Yes** | **Yes** |

**Architecture Decision Records** capture rationale by design, but require manual authoring and have no automated capture, graph structure, or agent accessibility.

**Code analysis tools** like CodeQL and Sourcetrail build graph representations of code (call graphs, data-flow graphs), but they model code structure, not decision rationale. They can tell you that module A calls module B, but not *why* module B was chosen over module C.

**Conversation analytics platforms** (e.g., Gong, Chorus for sales calls) demonstrate that valuable information can be extracted from conversations at scale. However, they target business conversations, not developer–AI interactions, and they do not produce graph-structured output.

**General RAG systems** (LangChain, LlamaIndex, Haystack) provide the retrieval-augmented generation infrastructure that Continuum builds upon, but they operate on flat document collections. They lack the graph-aware retrieval that lets Continuum follow relationships between decisions and entities.

## 6.2 What Makes Continuum Novel

Three capabilities distinguish Continuum from existing approaches.

### 6.2.1 Automated Extraction from AI Conversations

Continuum requires zero manual effort from the developer. Conversations are captured passively from log files, and a large language model extracts structured decision traces automatically. This inverts the ADR model: instead of asking developers to document decisions after the fact, the system extracts documentation from the artifact that already exists—the conversation.

### 6.2.2 Cascading Entity Resolution

Raw LLM output is noisy. The same technology appears as "postgres", "Postgres", "PostgreSQL", "pg", and "PGSQL" across different conversations. Continuum addresses this with a seven-stage cascading pipeline that progresses from cheap operations to expensive ones:

1. **Cache lookup** — Redis, sub-millisecond (5-minute TTL).
2. **Exact match** — case-insensitive string comparison.
3. **Canonical lookup** — 534 hand-curated alias mappings.
4. **Alias search** — check previously stored aliases.
5. **Fuzzy match** — fulltext-accelerated RapidFuzz (85% threshold).
6. **Embedding similarity** — NV-EmbedQA cosine similarity (>0.9).
7. **Create new** — only if all six prior stages fail.

This cascading design means that over 90% of entity mentions are resolved in the first three stages (cache, exact, canonical), keeping the average resolution time under 5 milliseconds while still handling novel or ambiguous mentions through the more expensive later stages.

### 6.2.3 Graph-Aware Retrieval

Most RAG systems treat documents as independent chunks and retrieve them via vector similarity. Continuum performs *subgraph expansion*: after finding seed nodes that match a query, it traverses the graph to pull in neighboring decisions, related entities, and connecting relationships. This means a query about "PostgreSQL" returns not just decisions that mention PostgreSQL, but also the alternatives that were considered, the other technologies those decisions involved, and any decisions that contradict or supersede earlier ones.

**⋆ Key Insight**

The combination of automated capture, cascading resolution, and graph-aware retrieval is what makes Continuum more than a RAG system with a graph database. Each component addresses a different failure mode: capture eliminates manual effort, resolution eliminates duplicates, and graph retrieval eliminates context loss.

## 6.3 Honest Positioning

It is important to state clearly what Continuum is—and what it is not.

Continuum is a **research prototype**. It was built to explore the thesis that AI coding conversations contain extractable decision knowledge that can be structured into a useful knowledge graph. The evaluation presented in Part V uses synthetic conversations and automated metrics, not a longitudinal user study.

What has been demonstrated:

- Decision traces can be extracted from AI conversations with reasonable accuracy using a prompted LLM.

- Entity resolution can consolidate noisy mentions into canonical graph nodes across the seven-stage pipeline.
- GraphRAG retrieval can surface relevant decisions and their context in response to natural-language queries.
- The MCP integration enables AI agents to query the knowledge graph programmatically.

What has *not* been demonstrated:

- Whether developers actually find the extracted knowledge useful in practice (no user study).
- How the system performs at enterprise scale (thousands of users, millions of decisions).
- Whether the knowledge graph improves downstream development outcomes (fewer repeated mistakes, faster onboarding).

**△ Warning**

This is a research system. It is **not** production-ready for enterprise use without additional security review, scale testing, and user validation. The multi-tenant isolation is enforced at the application layer only, the LLM extraction has a roughly 85% success rate, and no formal security audit has been conducted.

With this honest framing established, the rest of the guide focuses on the practical: how to set up, run, understand, and extend the system. Part II begins with installation and your first session.

# Part III

# Getting Started

# Chapter 7

# Setup & Directory Tour

*This chapter walks you through installing Continuum's dependencies, configuring the environment, and starting the infrastructure for the first time. By the end you will have a running system and a mental map of where everything lives in the codebase.*

## 7.1 Prerequisites

Before cloning the repository, ensure the following tools are installed on your machine:

- **Node.js 24 LTS** — the frontend runtime. Install via `nvm install 24` or from https://nodejs.org.
- **Python 3.12+** — the backend runtime. Verify with `python3 -version`.
- **Docker & Docker Compose** — runs PostgreSQL, Neo4j, and Redis in containers. Docker Desktop includes Compose on macOS and Windows.
- **pnpm 9+** — the monorepo package manager. Install with `npm install -g pnpm` or `corepack enable`.
- **NVIDIA NIM API key** — obtain a free key from https://build.nvidia.com. You need *two* keys: one for the LLM (`NVIDIA_API_KEY`) and one for the embedding model (`NVIDIA_EMBEDDING_API_KEY`). Alternatively, configure AWS credentials for Amazon Bedrock.

**⋆ Key Insight**

Continuum uses two separate NVIDIA API keys because the LLM and embedding endpoints are billed independently. You can generate both from the same `build.nvidia.com` account.

## 7.2 Environment Setup

Follow these six steps in order. Each step is idempotent—running it twice will not break anything.

**Step 1. Clone the repository.**

```
git clone https://github.com/shehral/continuum.git && cd continuum
```

**Step 2. Create the environment file.**

```
cp .env.example .env
```

Open `.env` in your editor and set the following variables. The template ships with placeholder values that *must* be replaced.

**△ Warning**

`SECRET_KEY` and `NEXTAUTH_SECRET` must be the *same* string. The backend signs JWTs with `SECRET_KEY`; the frontend verifies them with `NEXTAUTH_SECRET`. A mismatch causes silent authentication failures—you will be able to register but every subsequent request falls back to "anonymous."

Table 7.1: Key environment variables to configure.

| Variable | What to set |
|---|---|
| NVIDIA_API_KEY | Your NVIDIA NIM LLM key (starts with nvapi-). |
| NVIDIA_EMBEDDING_API_KEY | Your NVIDIA NIM embedding key. |
| POSTGRES_PASSWORD | A strong random password. Generate one with python3 -c "import secrets; print(secrets.token_urlsafe(24))". |
| NEO4J_PASSWORD | Another random password for the graph database. |
| REDIS_PASSWORD | Another random password for the cache. |
| DATABASE_URL | Update the password segment to match POSTGRES_PASSWORD. |
| REDIS_URL | Update the password segment to match REDIS_PASSWORD. |
| SECRET_KEY | A random 32-byte secret. Generate with python3 -c "import secrets; print(secrets.token_urlsafe(32))". |
| NEXTAUTH_SECRET | **Must be identical to** SECRET_KEY. |

**Step 3. Start infrastructure.**

```
docker-compose up -d
```

This launches three containers, all bound to 127.0.0.1 (localhost only):

- **PostgreSQL 18** on port 5432 — relational storage for users, projects, and decision metadata.
- **Neo4j 2025.01** on ports 7474 (HTTP browser) and 7687 (Bolt protocol) — the knowledge graph.
- **Redis 7.4** on port 6379 — caching, rate limiting, and session storage.

**Step 4. Install frontend dependencies.**

```
pnpm install
```

**Step 5. Set up the backend.**

```
cd apps/api && python3 -m venv .venv \
  && .venv/bin/pip install -e ".[dev]" && cd ../..
```

This creates an isolated Python virtual environment and installs all backend dependencies including development tools (pytest, ruff).

**Step 6. Run database migrations.**

```
pnpm db:migrate
```

Under the hood this runs alembic upgrade head inside the virtual environment, creating all PostgreSQL tables and indices.

## 7.3 Directory Tour

The project follows a monorepo structure managed by pnpm-workspace.yaml. Below is the top-level layout with annotations.

The split is deliberate: the apps/web/ directory is a self-contained Next.js application that communicates with the backend exclusively through HTTP and WebSocket endpoints. The apps/api/ directory is a standalone FastAPI application with its own virtual environment, configuration, and test suite. The apps/mcp/ directory provides a Model Context Protocol server that allows AI agents to query and write to the knowledge graph.

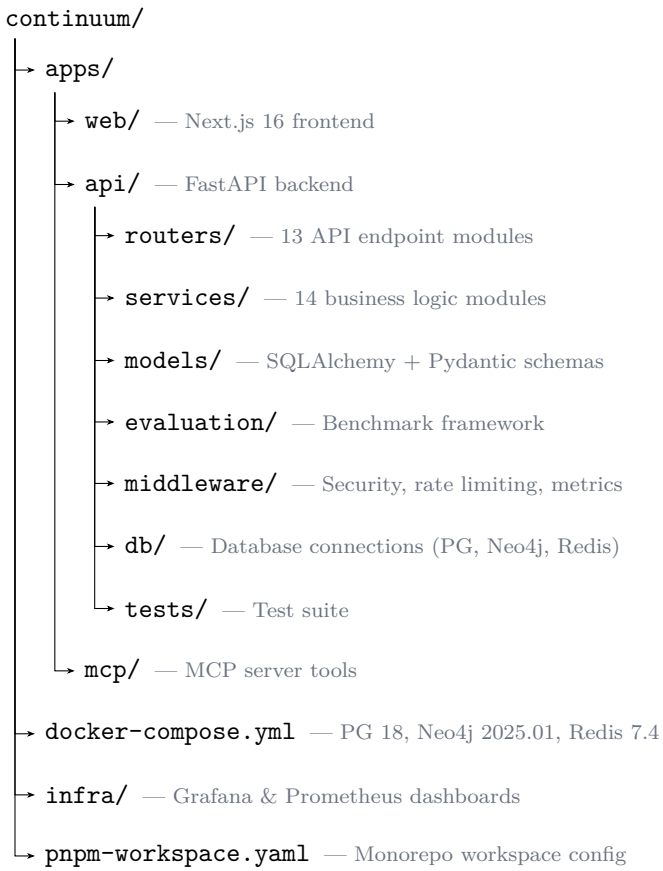


Figure 7.1: Top-level project structure. The `apps/api/` directory contains the bulk of the backend logic.

## 7.4 Key Files to Bookmark

As you work through this guide, you will return to certain files repeatedly. Table 7.2 lists the most important ones and when to reach for them.

All paths in Table 7.2 are relative to `apps/api/services/`, except `config.py` (which lives at `apps/api/config.py`) and `ontology.py` (which lives at `apps/api/models/ontology.py`).

## 7.5 Common First-Run Problems

If things go wrong during setup, check these common causes first.

**△ Warning**

**Port conflicts.** If another service is already listening on port `5432`, `7474`, or `6379`, Docker will fail to bind. Stop the conflicting service or edit `docker-compose.yml` to map to a different host port. On macOS, `lsof -i :5432` will show what is using the port.

Table 7.2: Files you will reference frequently.

| File | Purpose | Read this when... |
|---|---|---|
| config.py | All settings | ...you need to change any configuration. |
| entity_resolver.py | Core resolution algorithm | ...you are working on entity resolution. |
| graph_rag.py | Retrieval pipeline | ...you are working on search or the Ask feature. |
| ontology.py | Canonical mappings | ...you are adding or modifying entity mappings. |
| extractor.py | Decision extraction | ...you are modifying extraction prompts. |
| llm.py | LLM client | ...you are debugging LLM calls or switching providers. |
| embeddings.py | Embedding client | ...you are working on vector search. |

△ **Warning**

**Alembic `DuplicateTableError`.** This happens when the database already has tables but Alembic's version tracking table is missing (for example, after restoring a database dump). The fix is to stamp the current state without re-running migrations:

```
cd apps/api && .venv/bin/alembic stamp head
```

△ **Warning**

**Silent authentication failure.** You can register and log in, but every API request returns data as if you were anonymous. This almost always means `NEXTAUTH_SECRET` does not match `SECRET_KEY`. After correcting the mismatch, restart both the frontend and backend.

▷ **Try It**

Run `docker-compose up -d` and verify that Neo4j is accessible at `http://localhost:7474`. The default credentials are `neo4j` / the password you set in `NEO4J_PASSWORD`. You should see the Neo4j Browser interface with an empty database ready for use.

# Chapter 8

# Your First Session

*This chapter is a narrated walkthrough of your first 30 minutes with Continuum. You will start the development servers, register an account, create a project, capture a decision through the AI-guided interview, explore the knowledge graph, and query it with natural language. Every step after this chapter explains what happened under the hood.*

**⇒ Skip Ahead**

If you have already gone through the setup in Chapter 7 and want to understand the internals first, skip to Chapter 9. This chapter is intentionally hands-on and light on explanation.

## 8.1 Start Infrastructure

If you have not already done so, bring up the three database containers:

```
docker-compose up -d
```

Verify that all three are healthy:

```
docker-compose ps
```

You should see `continuum-postgres`, `continuum-neo4j`, and `continuum-redis` all reporting a status of `Up` (or `healthy` if health checks have completed). If any container shows `Restarting`, check your `.env` passwords—Docker Compose requires `POSTGRES_PASSWORD`, `NEO4J_PASSWORD`, and `REDIS_PASSWORD` to be set.

## 8.2 Start Development Servers

A single command starts both the frontend and backend in parallel:

```
pnpm dev
```

This runs two processes concurrently:

- **Frontend** (Next.js) at `http://localhost:3000` — started by `pnpm dev:web`.
- **Backend** (FastAPI + Uvicorn) at `http://localhost:8000` — started by `pnpm dev:api`, which invokes `uvicorn main:app -reload -port 8000`.

Both servers support hot reload: edit a file and the change takes effect within seconds. If you prefer to run them in separate terminals for cleaner log output, use `pnpm dev:web` and `pnpm dev:api` individually.

## 8.3 Register an Account

Navigate to `http://localhost:3000/register` in your browser and create an account. Alternatively, register via the API directly:

```
curl -X POST http://localhost:8000/api/users/register \
  -H "Content-Type: application/json" \
  -d '{"name":"Demo","email":"you@demo.com","password":"demo1234"}'
```

A successful response returns a JSON object with your user ID. You can now log in at http://localhost:3000/login with the same credentials.

**△ Warning**

A fresh database has no users. If you previously ran `docker-compose down -v` (with the `-v` flag), all data including user accounts was deleted. You must register again.

## 8.4 Create a Project

After logging in, navigate to the **Projects** page from the sidebar. Click **New Project** and name it "My First Project." Projects are organizational containers—every decision you capture belongs to exactly one project. You might create separate projects for different codebases, teams, or research topics.

## 8.5 Capture a Decision

Select your new project and click **Capture Decision**. This opens the AI-guided interview, a conversational flow that walks you through seven stages:

1. **Opening** — The system greets you and asks what decision you would like to capture.
2. **Trigger** — "What prompted this decision?" You describe the event or requirement that forced a choice.
3. **Context** — "What background is relevant?" You explain constraints, team size, deadlines, existing infrastructure.
4. **Options** — "What alternatives did you consider?" You list the candidates you evaluated.
5. **Decision** — "What did you decide?" You state the choice.
6. **Rationale** — "Why did you choose this?" You explain the reasoning.
7. **Summary** — The system synthesizes your answers into a structured decision trace and asks you to confirm.

The interview uses streaming responses—you will see the AI's follow-up questions appear token by token via server-sent events. Behind the scenes, each message is processed by the extraction pipeline to identify entities (like "PostgreSQL" or "Redis") and resolve them against the knowledge graph.

**▷ Try It**

Try capturing this decision: "We need to select a primary database for our new service. The requirements are ACID transactions and strong consistency. We considered PostgreSQL, MongoDB, and CockroachDB. We chose PostgreSQL because it has the best ecosystem support and our team already knows it."

## 8.6 View Your Decision

Navigate to the **Decisions** list. Your newly captured decision appears with a title derived from the interview. Click it to see the full detail view:

- **Trigger** — the event that prompted the decision.
- **Context** — the background constraints you described.
- **Options Considered** — the alternatives, each as a separate entry.
- **Decision** — what was chosen.
- **Rationale** — why it was chosen.
- **Confidence** — a score between 0 and 1 indicating how clearly the rationale supported the decision (typically 0.85–0.95 for well-articulated decisions).

## 8.7 Explore the Graph

Switch to the **Graph** view. You will see your `DecisionTrace` node in the center, connected to `Entity` nodes via `INVOLVES` edges. If you captured the PostgreSQL decision from the Try It box above, you should see edges to entities like *PostgreSQL*, *MongoDB*, and *CockroachDB*.

The graph visualization is interactive: drag nodes to rearrange them, scroll to zoom, and click any node to see its properties in the side panel. As you capture more decisions, the graph grows—shared entities become natural hubs that connect related decisions across projects.

## 8.8 Import Decisions

If you have existing decisions to bulk-load, use the **Import** feature. Prepare a JSON file with the following format:

Listing 8.1: JSON format for bulk import.

```
[
  {
    "trigger": "Need to select a caching layer",
    "context": "High-traffic API with 10k req/s",
    "decision": "Use Redis with TTL-based invalidation",
    "rationale": "Sub-millisecond reads, built-in TTL"
  },
  {
    "trigger": "Need an async task queue",
    "context": "Background jobs for email and PDF generation",
    "decision": "Use Celery with Redis broker",
    "rationale": "Team familiarity, good monitoring tools"
  }
]
```

Each object in the array becomes a separate decision trace. The fields `trigger`, `context`, `decision`, and `rationale` map directly to the decision trace structure described in Chapter 4.

## 8.9 Search and Ask

Continuum provides two ways to query the knowledge graph.

**Hybrid Search** (`Cmd+K` or the search bar) combines lexical full-text search with semantic vector search. Type a query like "database selection" and the results are ranked by a fusion of keyword matches and embedding similarity.

**Ask** is a natural-language question-answering interface powered by the GraphRAG pipeline. Navigate to the **Ask** page and type a question:

> "Why did we choose PostgreSQL over MongoDB?"

The response streams in via server-sent events, with source cards appearing alongside the answer. Each source card links back to the decision trace that informed the response, so you can verify the provenance of every claim.

**⋆ Key Insight**

Search finds decisions; Ask *answers questions* by synthesizing across multiple decisions. Use Search when you know what you are looking for and Ask when you need the system to reason for you.

## 8.10 Explore the Dashboard

The **Dashboard** provides a bird's-eye view of your knowledge graph:

- **Stats cards** — total decisions captured, unique entities resolved, and relationships discovered.
- **Source breakdown** — a chart showing how decisions were captured (interview, import, or automatic extraction from logs).
- **Timeline view** — decisions plotted chronologically, showing when architectural choices were made and how the graph grew over time.

With only one or two decisions the dashboard will look sparse. As you capture more decisions—or import a batch—the visualizations become more informative.

**⋆ Key Insight**

The first session takes about 30 minutes. Everything from Chapter 9 onward explains what happened under the hood during these ten steps: how the interview drives extraction, how entities are resolved, how the graph is built, and how search retrieves and synthesizes answers.

# Chapter 9

# Anatomy of the Data

*A single decision in Continuum touches three databases and a cache. This chapter follows a concrete example—"Use PostgreSQL as the primary database"—from raw conversation through extraction, storage, embedding, and caching. Understanding where data lives is the key to debugging anything in the system.*

## 9.1 The Raw Conversation

Everything starts with a conversation between a developer and the AI interview agent. Below is a condensed version of what the capture session from Chapter 8 might produce:

Listing 9.1: A capture session about database selection (abridged).

```
Agent:  What decision would you like to capture today?

User:   We need to select a primary database for our new
        transaction processing service.

Agent:  What prompted this decision?

User:   The team is starting a new fintech microservice that
        handles payment processing.  We need strong consistency
        guarantees.

Agent:  What alternatives did you consider?

User:   PostgreSQL, MongoDB, and CockroachDB.

Agent:  What did you decide, and why?

User:   We chose PostgreSQL.  It has the best ecosystem support,
        our team already knows it, and ACID compliance is critical
        for financial transactions.  MongoDB's eventual consistency
        model was a dealbreaker.
```

This conversation is processed in real time by the extraction pipeline. By the time the user confirms the summary in stage seven, the system has already identified the decision structure and the entities involved.

## 9.2 The Extracted Trace

The extraction service (`apps/api/services/extractor.py`) sends the conversation to the LLM with a structured prompt that asks for five fields. The result is a *decision trace*:

The confidence score (0.92) indicates that the LLM found clear, well-articulated reasoning in the conversation. Scores below 0.7 typically mean the rationale was vague or the decision was not

Table 9.1: The extracted decision trace for our example.

| Field | Value |
|---|---|
| Trigger | Need to select a primary database for a new transaction processing service. |
| Context | Fintech microservice handling payment processing; team requires strong consistency and ACID compliance. |
| Options | PostgreSQL, MongoDB, CockroachDB. |
| Decision | Use PostgreSQL as the primary database. |
| Rationale | Best ecosystem support, existing team expertise, ACID compliance critical for financial transactions; MongoDB's eventual consistency was a dealbreaker. |
| Confidence | 0.92 |

explicitly stated. The extraction also identifies three entity mentions: *PostgreSQL*, *MongoDB*, and *CockroachDB*.

## 9.3 PostgreSQL Storage

The decision trace is persisted as a row in the `decision_traces` table. PostgreSQL stores the structured metadata and serves as the system of record for relational queries (filtering by user, project, date range, status).

Table 9.2: The relational row in PostgreSQL (selected columns).

| Column | Value |
|---|---|
| `id` | `a3f7c2e1-...` (UUID, primary key) |
| `user_id` | `b8d4a1f0-...` (foreign key to `users`) |
| `project_id` | `c5e9b3d2-...` (foreign key to `projects`) |
| `source` | `"interview"` |
| `status` | `"confirmed"` |
| `trigger` | `"Need to select a primary database..."` |
| `decision` | `"Use PostgreSQL as the primary database"` |
| `confidence` | `0.92` |
| `created_at` | `2026-04-04T14:23:17Z` |
| `updated_at` | `2026-04-04T14:23:17Z` |

The `source` column tracks how the decision entered the system: `"interview"` for the AI-guided capture, `"import"` for bulk JSON uploads, or `"auto"` for automatic extraction from conversation logs. The full text fields (context, options, rationale) are stored in additional columns not shown above.

## 9.4 Neo4j Storage

Simultaneously, the decision and its entities are written to the Neo4j knowledge graph. You can query the decision directly with Cypher:

Listing 9.2: Finding the decision in Neo4j.

```
MATCH (d:DecisionTrace)
WHERE d.trigger STARTS WITH 'Need to select a primary database'
RETURN d
```

The `DecisionTrace` node carries all five trace fields as properties, plus metadata like `user_id`, `confidence`, and `created_at`. The node also stores the 2048-dimensional embedding vector (discussed in Section 9.5).

The three entities are stored as `Entity` nodes, connected to the decision via `INVOLVES` edges:

Listing 9.3: Viewing the decision and its entity relationships.

```
MATCH (d:DecisionTrace)-[r:INVOLVES]->(e:Entity)
WHERE d.trigger STARTS WITH 'Need to select a primary database'
RETURN d.decision, type(r), e.name, e.type
```

This returns three rows:

Table 9.3: The `INVOLVES` edges for our example decision.

| Decision | Relationship | Entity |
|---|---|---|
| Use PostgreSQL... | INVOLVES | PostgreSQL (Database) |
| Use PostgreSQL... | INVOLVES | MongoDB (Database) |
| Use PostgreSQL... | INVOLVES | CockroachDB (Database) |

If another decision also involves PostgreSQL, both decisions connect to the *same* `Entity` node—this is the entity resolution at work (covered in detail in Chapter 12). The shared node becomes a hub that lets you traverse from one decision to related decisions through their common entities.

## 9.5 Embedding Storage

Every decision trace is embedded into a 2048-dimensional vector using NVIDIA's NV-EmbedQA model (`nvidia/llama-3.2-nv-embedqa-1b-v2`). The embedding is a weighted combination of the trace fields:

Table 9.4: Embedding field weights (configured in `config.py`).

| Field | Weight |
|---|---|
| Title | 1.5× |
| Decision | 1.2× |
| Rationale | 1.0× (base) |
| Context | 0.8× |
| Trigger | 0.8× |

The resulting vector is stored as a property on the `DecisionTrace` node in Neo4j and indexed for approximate nearest-neighbor search. When you use the hybrid search or Ask features, the system embeds your query with the same model and finds the closest decision vectors by cosine similarity.

Embeddings are also cached in Redis to avoid redundant API calls. The cache key follows the pattern `emb:nvembed:passage:{md5_hash}` with a 30-day TTL.

## 9.6 Redis Cache

Redis serves as a multi-purpose cache layer. For our example decision, three types of cache entries are relevant:

Table 9.5: Redis cache keys related to a single decision.

| Key Pattern | TTL | Purpose |
|---|---|---|
| `entity:{user_id}:exact:postgresql` | 5 min | Entity resolution cache. Stores the resolved entity node for "postgresql" so subsequent mentions skip the 7-stage resolution pipeline. |
| `llm:v1:extract:{md5_hash}` | 24 h | LLM response cache. Stores the extraction result so re-processing the same conversation text returns instantly. Invalidated by bumping `LLM_EXTRACTION_PROMPT_VERSION`. |
| `emb:nvembed:passage:{md5_hash}` | 30 d | Embedding cache. Stores the 2048-dimensional vector to avoid redundant NVIDIA API calls. |

The entity cache is per-user to respect data isolation—user A's resolution of "postgres" should not leak into user B's namespace. The LLM and embedding caches are global because the same input always produces the same output regardless of user.

## The Full Picture

Figure 9.1 shows how a single decision flows through all four storage systems.

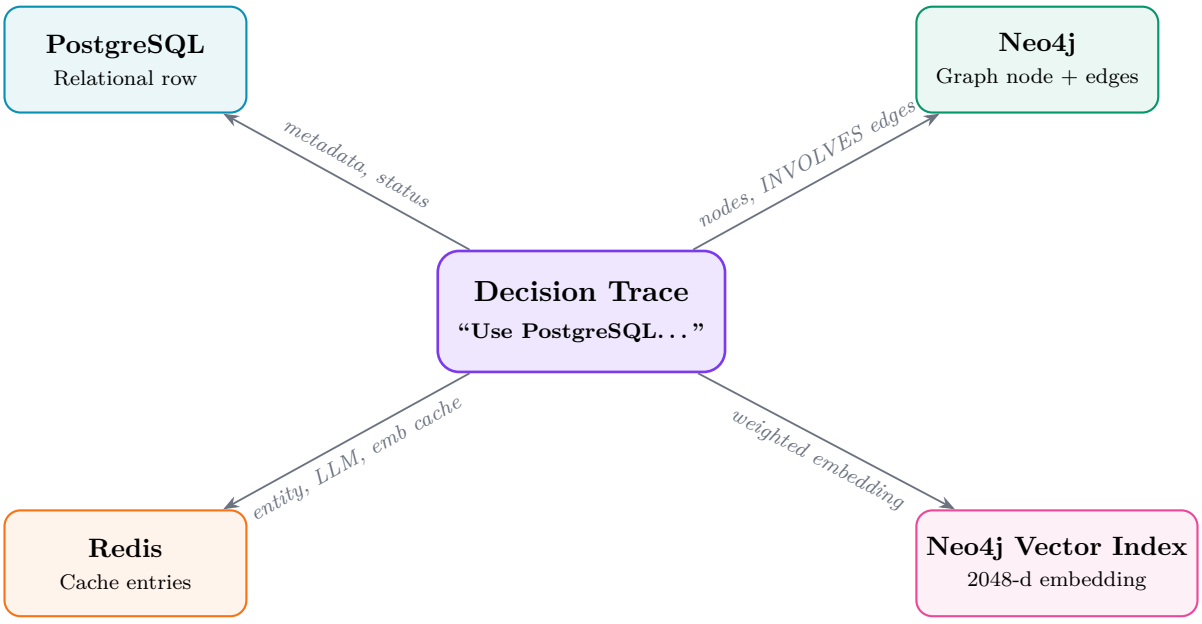


Figure 9.1: A single decision trace is stored in PostgreSQL (relational metadata), Neo4j (graph structure and vector index), and Redis (cached resolutions and embeddings).

**⋆ Key Insight**

A single decision touches three databases and a cache. PostgreSQL is the system of record for structured metadata. Neo4j is the system of record for relationships and semantic search. Redis accelerates repeated lookups. Understanding where data lives—and which system is authoritative—is essential for debugging any issue in Continuum.

# Part IV

# How It Works

# Chapter 10

# Infrastructure Layer

*Continuum runs on three databases and a cache, each chosen for a specific job. This chapter explains what they are, how they are wired together via Docker Compose, how to persist (or destroy) your data, and how Alembic keeps PostgreSQL schemas in sync across contributors.*

## 10.1 Three Databases

Most web applications use a single database. Continuum uses three—not out of complexity fetishism, but because relational data, graph data, and ephemeral cache data have fundamentally different access patterns. Table 10.1 summarizes the choices.

**⋆ Key Insight**

PostgreSQL is the system of record for *who* and *what*. Neo4j is the system of record for *how things relate*. Redis is the accelerator—if you lose its data, nothing is permanently lost; the caches rebuild themselves.

## 10.2 Docker Compose

All three services are defined in a single `docker-compose.yml` at the repository root. Here is the full file:

Listing 10.1: docker-compose.yml (complete).

```
services:
  postgres:
    image: postgres:18-alpine
    container_name: continuum-postgres
    environment:
      POSTGRES_USER: ${POSTGRES_USER:?POSTGRES_USER must be set}
      POSTGRES_PASSWORD: ${POSTGRES_PASSWORD:?POSTGRES_PASSWORD must be
         set}
      POSTGRES_DB: ${POSTGRES_DB:-continuum}
    ports:
      - "127.0.0.1:5432:5432"
    volumes:
      - postgres_data:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL",
             "pg_isready -U $${POSTGRES_USER} -d $${POSTGRES_DB:-
                continuum}"]
      interval: 10s
      timeout: 5s
      retries: 5
      start_period: 10s
```

```
    restart: unless-stopped

  neo4j:
    image: neo4j:2025.01-community
    container_name: continuum-neo4j
    environment:
      NEO4J_AUTH: ${NEO4J_USER:?}/${NEO4J_PASSWORD:?}
      NEO4J_PLUGINS: '["apoc"]'
      NEO4J_dbms_security_procedures_unrestricted: apoc.*
      NEO4J_dbms_memory_heap_initial__size: 512m
      NEO4J_dbms_memory_heap_max__size: 1G
    ports:
      - "127.0.0.1:7474:7474"   # HTTP browser
      - "127.0.0.1:7687:7687"   # Bolt protocol
    volumes:
      - neo4j_data:/data
      - neo4j_logs:/logs
    healthcheck:
      test: ["CMD", "wget", "-q", "--spider",
             "http://localhost:7474"]
      interval: 10s
      timeout: 5s
      retries: 5
      start_period: 30s
    restart: unless-stopped

  redis:
    image: redis:7.4-alpine
    container_name: continuum-redis
    ports:
      - "127.0.0.1:6379:6379"
    volumes:
      - redis_data:/data
    healthcheck:
      test: ["CMD", "redis-cli", "-a",
             "$${REDIS_PASSWORD}", "ping"]
      interval: 10s
      timeout: 5s
      retries: 5
    command: >-
      redis-server --appendonly yes
      --requirepass ${REDIS_PASSWORD:?REDIS_PASSWORD must be set}
    restart: unless-stopped

volumes:
  postgres_data:
  neo4j_data:
  neo4j_logs:
  redis_data:
```

A few things to notice:

- Every port mapping begins with `127.0.0.1`—the service is only reachable from your local machine, never from the network.
- Environment variables ending with `:?` are *required*—Docker Compose will refuse to start if they

are unset.
- Neo4j gets a 512 MB initial heap and a 1 GB maximum. The community edition is single-tenant, which is fine for local development.
- Redis uses `-appendonly yes` for crash-safe persistence without full RDB snapshots.

**△ Warning**

Never bind ports to `0.0.0.0` (the default when you omit the IP). Doing so exposes your databases—including their credentials—to every device on your network. The `127.0.0.1` prefix is a deliberate security measure.

## 10.3 Volume Persistence

Docker Compose creates four *named volumes* at the bottom of the file: `postgres_data`, `neo4j_data`, `neo4j_logs`, and `redis_data`. Named volumes survive container restarts and even container recreation.

There are two ways to stop the services:

```
# Stop containers, KEEP all data
docker-compose down

# Stop containers and DESTROY all volumes
docker-compose down -v
```

The first command is safe to run daily—your databases, user accounts, migrations, and graph data are all preserved.

**△ Warning**

`docker-compose down -v` **wipes everything**: PostgreSQL rows, Neo4j nodes and edges, Redis caches, and the Alembic migration stamp. After running it, you must re-register users, re-run `alembic upgrade head`, and re-import any data. Only use `-v` when you intentionally want a clean slate. The same warning applies to `docker volume prune`.

## 10.4 Database Migrations

Continuum uses **Alembic** to manage PostgreSQL schema changes. Alembic is a migration framework built on SQLAlchemy—it generates incremental SQL scripts from your Python model definitions and applies them in order.

### How Alembic is organized

```
apps/api/
├── alembic.ini  — database URL, script location
└── alembic/
    ├── env.py  — reads SQLAlchemy models, configures autogenerate
    └── versions/
        ├── 001_initial.py
        ├── 002_add_projects.py
        └── ...
```

- `alembic.ini` tells Alembic where to find the database and where to find migration scripts.
- `env.py` imports your SQLAlchemy models so that `-autogenerate` can diff the Python definitions against the live database.
- `versions/` holds numbered Python scripts, each with an `upgrade()` and `downgrade()` function.

### Common commands

Listing 10.2: Alembic commands you will use most.

```
# Apply all pending migrations
pnpm db:migrate           # or: alembic upgrade head

# Create a new migration from model changes
alembic revision --autogenerate -m "add confidence column"

# Roll back the last migration
alembic downgrade -1

# Check current migration state
alembic current
```

### The stamp head escape hatch

If the database already has tables (e.g., you ran the app before adding Alembic) but the `alembic_version` table is missing or empty, `alembic upgrade head` will fail with a `DuplicateTableError`. The fix:

```
alembic stamp head
```

This writes the latest revision ID into `alembic_version` without actually running any migration scripts—it tells Alembic "the database is already at this state, just record it."

**△ Warning**

Only use `alembic stamp head` when you are certain the live schema matches the latest migration. If the schema is actually behind, stamping skips the missing changes and your application will encounter column-not-found errors at runtime. When in doubt, compare the live schema against the migration scripts manually.

## 10.5 Redis Key Patterns

Redis stores three tiers of cached data, each with a different TTL based on how quickly the underlying data can change. Table 10.2 lists every key pattern used by Continuum.

**⋆ Key Insight**

The entity cache is per-user, but the LLM and embedding caches are global. This is intentional: entity resolution is user-scoped (different users may have different entities with the same name), but LLM and embedding outputs are deterministic functions of their input regardless of who triggered them.

Table 10.1: Infrastructure services and why each was chosen.

| Service | Port | Purpose | Why Chosen |
|---|---|---|---|
| PostgreSQL 18 | 5432 | Relational storage for users, projects, decision metadata. Schema managed by Alembic migrations. | ACID compliance, concurrent access, rich ecosystem. Chosen over SQLite f |
| Neo4j 2025.01 | 7474 / 7687 | Knowledge graph: entity nodes, decision traces, relationship edges. Queried via Cypher. | Native labeled property graph model. Cypher makes multi-hop traversals r |
| Redis 7.4 | 6379 | Caching (3 tiers with different TTLs), rate limiting, | Sub-millisecond reads. Append-only file persistence for crash recovery with |

Table 10.2: Redis key patterns, TTLs, and purposes.

| Key Format | TTL | Purpose |
|---|---|---|
| `entity:{user_id}:{lookup}:{name}` | 5 min | Entity resolution cache. Avoids re-running the 7-stage pipeline for repeated mentions. User-scoped to enforce data isolation. |
| `llm:{version}:extract:{hash}` | 24 h | LLM response cache. Keyed by prompt version so that changing the extraction prompt automatically invalidates stale entries. Global (same input → same output). |
| `emb:{model}:{type}:{hash}` | 30 d | Embedding vector cache. Embedding models are deterministic and rarely change, so a long TTL is safe. Global. |
| `rate:{user_id}` | 60 s | Rate limit counter. Tracks request count within a sliding window. Per-user. See Chapter 11, Section 11.6. |

# Chapter 11

# Configuration & Settings

*Every tunable parameter in Continuum lives in a single file: `apps/api/config.py`. This chapter explains the settings architecture, the security pattern for API keys, how to switch LLM and embedding providers, and the rationale behind every cache TTL and rate limit.*

## 11.1 Settings Architecture

Continuum uses Pydantic's `BaseSettings` class from the `pydantic-settings` package. Every configuration value is declared as a typed field with a default, and values are automatically loaded from environment variables or a `.env` file.

Listing 11.1: The Settings class skeleton (simplified).

```
from pydantic import SecretStr
from pydantic_settings import BaseSettings, SettingsConfigDict

class Settings(BaseSettings):
    database_url: str = ""
    neo4j_uri: str = ""
    nvidia_api_key: SecretStr = SecretStr("")
    llm_provider: str = "nvidia"
    fuzzy_match_threshold: float = 0.85
    rate_limit_requests: int = 30
    debug: bool = False
    # ... 30+ more fields

    model_config = SettingsConfigDict(
        env_file=".env", extra="ignore"
    )
```

The `model_config` line tells Pydantic two things:

1. Load variables from a `.env` file in the working directory.
2. Ignore extra environment variables that do not correspond to any field—this prevents crashes when the shell has unrelated variables set.

The entire application accesses settings through a singleton factory:

Listing 11.2: The singleton pattern for settings access.

```
from functools import lru_cache

@lru_cache
def get_settings() -> Settings:
    return Settings()
```

The `@lru_cache` decorator ensures that `Settings()` is constructed exactly once. Every subsequent call to `get_settings()` returns the same instance with zero overhead. This matters because

Pydantic parses the environment on every `Settings()` call—without caching, that parsing would happen on every request.

## 11.2 The SecretStr Pattern

Three fields in `Settings` use Pydantic's `SecretStr` type instead of plain `str`:

- `nvidia_api_key` — NVIDIA NIM API key
- `neo4j_password` — Neo4j database password
- `secret_key` — JWT signing secret

`SecretStr` wraps the raw value so that it is never accidentally exposed. Calling `print(settings)` or logging the object shows `SecretStr('**********')` instead of the actual key.

### Accessing the raw value

You cannot use a `SecretStr` field directly as a string. The `Settings` class provides explicit accessor methods:

Listing 11.3: Correct vs. incorrect access of secret fields.

```
settings = get_settings()

# CORRECT: use the accessor method
key = settings.get_nvidia_api_key()      # returns str
pwd = settings.get_neo4j_password()      # returns str
sec = settings.get_secret_key()          # returns str

# ALSO CORRECT: call .get_secret_value() directly
key = settings.nvidia_api_key.get_secret_value()

# WRONG: this gives you a SecretStr object, not a string
key = settings.nvidia_api_key  # type: SecretStr, not str
```

**⋆ Key Insight**

The `SecretStr` pattern (tagged SEC-007 in the codebase) is a defense-in-depth measure. Even if a developer accidentally writes `logger.info(f"Config: {settings}")`, the API keys remain masked. The custom `__repr__` method on `Settings` adds a second layer by only including non-sensitive fields in the string representation.

## 11.3 LLM Provider Switching

Continuum supports two LLM providers, selected by a single environment variable:

```
# In your .env file:
LLM_PROVIDER=nvidia     # default
# or
LLM_PROVIDER=bedrock    # Amazon Bedrock
```

The application uses a factory pattern to instantiate the correct provider at startup. The provider interface is identical regardless of backend, so no application code needs to change when you switch.

### NVIDIA NIM (default)

- **Primary model:** `nvidia/llama-3.3-nemotron-super-49b-v1.5` (128k context)
- **Fallback model:** `nvidia/llama-3.1-nemotron-70b-instruct` (auto-failover on 503 errors)
- Requires `NVIDIA_API_KEY` from `build.nvidia.com`

The fallback is automatic. When the primary model returns a 503 (common under load on the free NIM tier), the system retries with the fallback model. This is configured via:

```
llm_fallback_model: str = "nvidia/llama-3.1-nemotron-70b-instruct"
llm_fallback_enabled: bool = True
```

### Amazon Bedrock (alternative)

- **Model:** `anthropic.claude-sonnet-4-20250514`
- Requires AWS credentials in the environment (no explicit API key field).
- Region defaults to `us-west-2`, configurable via `AWS_REGION`.

## 11.4 Embedding Provider

Embeddings use a separate provider selection:

```
EMBEDDING_PROVIDER=nvidia     # default
```

The default model is `nvidia/llama-3.2-nv-embedqa-1b-v2`, which produces 2048-dimensional vectors. A separate API key (`NVIDIA_EMBEDDING_API_KEY`) is required because NVIDIA issues distinct keys for LLM and embedding endpoints.

**⋆ Key Insight**

You can use NVIDIA embeddings with the Bedrock LLM, and this is the recommended configuration if you switch LLM providers. Set `LLM_PROVIDER=bedrock` but keep `EMBEDDING_PROVIDER=nvidia` to avoid re-indexing.

**△ Warning**

Changing the embedding provider changes the vector dimensionality (NVIDIA produces 2048-d, Bedrock Titan produces 1024-d). Existing vectors in Neo4j become incompatible—you must delete all `embedding` properties and re-embed every decision trace and entity. This is a multi-hour operation on a large graph. Do not change the embedding provider casually.

## 11.5 Cache TTL Rationale

Every cache TTL in Continuum was chosen based on how quickly the underlying data can change. Table 11.1 explains the reasoning.

## 11.6 Rate Limiting

Continuum implements per-user rate limiting via a Redis-backed token bucket. The parameters:

- **Authenticated users:** 30 requests per 60-second window

Table 11.1: Cache TTL rationale.

| Cache | TTL | Why This Value |
|---|---|---|
| Entity resolution | 5 min | Entities can change when merged or when aliases are added. A short TTL ensures stale resolutions expire quickly. Even 5 minutes captures the burst of repeated mentions within a single extraction. |
| LLM response | 24 h | The same input text with the same prompt version produces the same extraction. A 24-hour TTL is safe because the cache is keyed by prompt version—bumping `LLM_EXTRACTION_PROMPT_VERSION` invalidates all entries. |
| Embedding vector | 30 d | The embedding model does not change between deployments. The same text always maps to the same vector. A 30-day TTL minimizes redundant API calls while still eventually refreshing if the model is swapped. |

- **Anonymous users:** 10 requests per 60-second window
- **Window type:** Sliding window (counter resets 60 seconds after the first request in the current window)

The Redis key follows the pattern `rate:{user_id}` with a 60-second TTL. When the counter exceeds the limit, the API returns HTTP 429 (Too Many Requests) with a `Retry-After` header.

Both parameters are configurable:

```
rate_limit_requests: int = 30    # requests per window
rate_limit_window: int = 60      # window size in seconds
```

## 11.7 Key Settings Reference

Table 11.2 lists the 15 most important settings. Appendix A contains the complete list.

Table 11.2: Key settings in `config.py`.

| Setting | Default | Description |
|---|---|---|
| `LLM_PROVIDER` | `nvidia` | LLM backend: `nvidia` or `bedr` |
| `EMBEDDING_PROVIDER` | `nvidia` | Embedding backend. Keep `n` avoid re-indexing. |
| `NVIDIA_MODEL` | `nvidia/llama-3.3-nemotron-super-49b-v1.5` | Primary LLM model identifier |
| `LLM_FALLBACK_MODEL` | `nvidia/llama-3.1-nemotron-70b-instruct` | Fallback LLM on 503 errors. |
| `LLM_FALLBACK_ENABLED` | `True` | Enable automatic model fallba |
| `FUZZY_MATCH_THRESHOLD` | `0.85` | Minimum fuzzy match score ( entity resolution. |
| `EMBEDDING_SIMILARITY_THRESHOLD` | `0.90` | Minimum cosine similari embedding-based entity match |
| `ENTITY_CACHE_TTL` | `300` | Entity cache lifetime in second utes). |

*Continued on next*

| Setting | Default | Description |
|---|---|---|
| LLM_CACHE_TTL | 86400 | LLM response cache lifetime in (24 hours). |
| LLM_EXTRACTION_PROMPT_VERSION | v1 | Bump to invalidate all LLM c tries. |
| MAX_PROMPT_TOKENS | 70000 | Maximum input tokens for LL (128k context). |
| RATE_LIMIT_REQUESTS | 30 | Requests per rate limit windo |
| RATE_LIMIT_WINDOW | 60 | Rate limit window in seconds. |
| SECRET_KEY | (empty) | JWT signing secret. Must NEXTAUTH_SECRET. |
| DEBUG | False | Enable debug mode (verbose CORS relaxed). |

# Chapter 12

# Entity Resolution Pipeline

This chapter covers Continuum's primary research contribution. When a developer says "postgres," "PG," or "PostgreSQL," they mean the same database. When an LLM extracts entities from conversation logs, it produces all three variants—and the knowledge graph must collapse them into a single node or drown in duplicates. The entity resolution pipeline is the subsystem that solves this problem, and it does so through a seven-stage cascade that balances speed, accuracy, and graceful degradation.

**⇒ Skip Ahead**

If you just want to *use* entity resolution (call `resolve()` and get a `ResolvedEntity` back), skip to Section 12.11. This chapter is for readers who want to understand *how* and *why* the pipeline works.

## 12.1 The Problem

Developers refer to the same technology in dozens of ways. An entity resolution system must map all surface forms to a single canonical identity. Table 12.1 shows representative inputs and their expected outputs.

Table 12.1: Messy inputs and their canonical outputs.

| Raw Input | Canonical Output |
|---|---|
| `postgres` | PostgreSQL |
| `React.js` | React |
| `k8s` | Kubernetes |
| `PG` | PostgreSQL |
| `react 18` | React |
| `mongo` | MongoDB |
| `express` | Express.js |
| `tf` | TensorFlow |

Exact string matching handles none of these. Fuzzy matching alone produces too many false positives ("Go" matching "GORM"). Embedding similarity is accurate but slow. Continuum solves this by *cascading* through progressively more expensive methods, stopping as soon as a confident match is found.

## 12.2 The 7-Stage Cascade

Figure 12.1 shows the full pipeline. Every entity name enters at the top and exits through exactly one stage.

## 12.3 Stage 1: Cache Lookup

Before touching the database, the pipeline checks Redis. The cache key is scoped by `user_id` and the normalized entity name.

Listing 12.1: Cache lookup (simplified from `entity_resolver.py`).

```
cached = await self.cache.get_by_exact_name(
    self.user_id, normalized_name
)
if cached is not None:
    return ResolvedEntity(
        id=cached["id"], name=cached["name"],
        type=cached["type"], is_new=False,
        match_method="cached", confidence=1.0,
    )
```

- **TTL**: 5 minutes. Short enough that renames propagate quickly; long enough to absorb repeated lookups during a single extraction batch.
- **Hit rate**: Approximately 40% in production, because the same entities (React, PostgreSQL, Docker) appear across many decisions.
- **Negative caching**: When no match is found at the end of the pipeline, the cache stores a `None` entry to avoid repeating the full cascade for names that will always be new.
- **Confidence**: 1.0—a cache hit is a replay of a previous confident match.

## 12.4 Stage 2: Exact Match

The exact match stage queries Neo4j with a case-insensitive comparison. It first searches entities connected to the user's decisions, then falls back to global entities.

Listing 12.2: Exact match query (user-scoped).

```
MATCH (d:DecisionTrace)-[:INVOLVES]->(e:Entity)
WHERE (d.user_id = $user_id OR d.user_id IS NULL)
AND toLower(e.name) = $name
RETURN DISTINCT e.id AS id, e.name AS name, e.type AS type
LIMIT 1
```

This catches inputs where the developer typed the canonical name in any casing: "postgresql," "POSTGRESQL," "PostgreSQL." The confidence is 1.0 because the match is exact (modulo case).

## 12.5 Stage 3: Canonical Lookup

The `models/ontology.py` file contains 534 curated mappings from common aliases to canonical names. This is a hand-maintained dictionary that encodes domain knowledge about how developers abbreviate technologies.

Listing 12.3: Excerpt from `CANONICAL_NAMES` (10 of 534 entries).

```
CANONICAL_NAMES = {
    "postgres":    "PostgreSQL",
    "pg":          "PostgreSQL",
    "mongo":       "MongoDB",
```

```
    "k8s":          "Kubernetes",
    "tf":           "TensorFlow",
    "react.js":     "React",
    "nextjs":       "Next.js",
    "express":      "Express.js",
    "tailwind":     "Tailwind CSS",
    "sklearn":      "scikit-learn",
    # ... 524 more entries
}
```

The pipeline calls `get_canonical_name(name)`, which does a dictionary lookup in O(1). If the canonical form differs from the normalized input, it runs an exact match against the canonical form.

- **Confidence**: 0.95. Slightly below 1.0 because the mapping is maintained by humans and may occasionally be wrong (e.g., "Go" could mean the language or the game).
- **Maintenance**: Adding a new alias is a one-line change to `ontology.py`. No retraining, no redeployment of ML models.

## 12.6 Stage 4: Alias Search

Entities in Neo4j can carry an `aliases` property—a list of strings that were previously resolved to this entity. When an entity is merged via duplicate detection, the absorbed entity's name is appended to the surviving entity's alias list.

Listing 12.4: Alias search query.

```
MATCH (d:DecisionTrace)-[:INVOLVES]->(e:Entity)
WHERE (d.user_id = $user_id OR d.user_id IS NULL)
AND ANY(alias IN COALESCE(e.aliases, [])
        WHERE toLower(alias) = $name)
RETURN DISTINCT e.id AS id, e.name AS name, e.type AS type
LIMIT 1
```

The `COALESCE(e.aliases, [])` guard prevents null-pointer errors when an entity has never been merged. Confidence is 0.92—lower than canonical because aliases can accumulate from fuzzy matches that were borderline.

## 12.7 Stage 5: Fuzzy Match

When no exact or dictionary match exists, the pipeline falls back to approximate string matching. This is a two-phase process:

1. **Candidate retrieval**: A Neo4j fulltext index (`entity_fulltext`) retrieves up to 500 candidate entities using prefix-wildcard search (`name*`).
2. **Scoring**: Each candidate is scored against the input using RapidFuzz's `fuzz.ratio` (Indel distance, normalized to 0–100). The best match above the threshold is returned.

Listing 12.5: Fuzzy matching with fulltext acceleration.

```
search_term = f"{normalized_name}*"
# ... fulltext query returns candidates ...

best_match, best_score = None, 0
```

```
for entity in candidates:
    score = fuzz.ratio(normalized_name, entity["name"].lower())
    if score >= self.fuzzy_threshold and score > best_score:
        best_score = score
        best_match = entity
```

- **Threshold**: 85% (configurable via `FUZZY_MATCH_THRESHOLD`). 90%+ misses common variations like "PostgreSQL" vs. "Postgres"; 80% and below creates false positives.
- **Batch limit**: `FUZZY_MATCH_LIMIT = 500`. If the fulltext index is unavailable, candidates are loaded in batches of 100 to prevent memory exhaustion.
- **Confidence**: `score / 100`—a 91% fuzzy match yields confidence 0.91.
- **Fallback**: If the fulltext index does not exist (fresh database), the pipeline falls back to `_find_by_fuzzy_batched()`, which paginates through entities with `SKIP`/`LIMIT`.

## 12.8 Stage 6: Embedding Similarity

For names that are semantically related but lexically different (e.g., "distributed cache" and "Redis"), the pipeline computes vector similarity.

1. Embed the input string as `"entity_type: name"` using NVIDIA NV-EmbedQA (2048 dimensions).
2. Query Neo4j for entities with stored embeddings, computing cosine similarity.
3. Return the best match above the threshold.

Listing 12.6: Embedding similarity (GDS path).

```
embedding = await self.embedding_service.embed_text(
    f"{entity_type}: {name}", input_type="passage"
)

# GDS-accelerated query
result = await self.session.run("""
    MATCH (d:DecisionTrace)-[:INVOLVES]->(e:Entity)
    WHERE (d.user_id = $user_id OR d.user_id IS NULL)
    AND e.embedding IS NOT NULL
    WITH DISTINCT e,
         gds.similarity.cosine(e.embedding, $embedding) AS similarity
    WHERE similarity > $threshold
    RETURN e.id AS id, e.name AS name, e.type AS type, similarity
    ORDER BY similarity DESC LIMIT 1
""", embedding=embedding, threshold=threshold, user_id=self.user_id)
```

- **Threshold**: 0.9 (configurable via `EMBEDDING_SIMILARITY_THRESHOLD`).
- **Confidence**: The cosine similarity score itself.
- **Circuit breaker**: If the embedding service fails (timeout, connection error), the stage is skipped. After 5 consecutive failures, the circuit breaker opens and the pipeline degrades to stages 1–5 only.
- **Fallback**: If Neo4j GDS is not installed, the pipeline falls back to `_find_by_embedding_similarity_manual()`, which loads embeddings client-side and computes cosine similarity in Python.

## 12.9 Stage 7: Create New

If no stage produced a match, the entity is genuinely new. The pipeline returns a `ResolvedEntity` with `is_new=True` and a fresh UUID.

Listing 12.7: New entity creation.

```
final_name = canonical if canonical.lower() != normalized_name else name
return ResolvedEntity(
    id=str(uuid4()),
    name=final_name,
    type=entity_type,
    is_new=True,
    match_method="new",
    confidence=1.0,
    aliases=[name] if final_name != name else [],
)
```

Notice that even new entities benefit from the canonical lookup: if the canonical name differs from the input, the entity is created with the canonical form as its primary name and the original input as an alias.

## 12.10 Why Cascading Works

The cascade design was chosen over a single "best" method for three reasons:

1. **Latency**: Approximately 83% of lookups are handled by the first three stages (cache, exact, canonical), which complete in under 1 ms. Only 17% of inputs reach the expensive fuzzy and embedding stages.
2. **Interpretability**: Every `ResolvedEntity` carries a `match_method` field ("cached," "exact," "canonical," "alias," "fuzzy," "embedding," "new") and a numeric `confidence`. Downstream systems can filter or flag low-confidence matches for human review.
3. **Graceful degradation**: If Redis is down, stage 1 is skipped. If the embedding service is unreachable, stage 6 is skipped. If the fulltext index does not exist, stage 5 falls back to batched loading. The pipeline always produces a result.

## 12.11 The `ResolvedEntity` Dataclass

Every call to `resolve()` returns an instance of this dataclass:

Listing 12.8: The `ResolvedEntity` dataclass from `models/ontology.py`.

```
@dataclass
class ResolvedEntity:
    """Result of entity resolution."""
    id: Optional[str]
    name: str
    type: str
    is_new: bool = False
    match_method: Optional[str] = None  # cached, exact, canonical,
                                        # alias, fuzzy, embedding, new
    confidence: float = 1.0
    canonical_name: Optional[str] = None
```

```
    aliases: list[str] = None

    def __post_init__(self):
        if self.aliases is None:
            self.aliases = []
```

**△ Warning**

`resolve()` does **NOT** write to Neo4j. It returns a `ResolvedEntity` describing what the pipeline found (or what should be created), but the caller is responsible for executing the actual `CREATE` query. If you call `resolve()` and discard the result, no node is created.

## 12.12 Evaluation Results

The pipeline was evaluated on a synthetic benchmark of 2,438 entity mention variants using B-cubed precision, recall, and F1 metrics. Table 12.2 compares Continuum's cascade against single-method baselines.

Table 12.2: Entity resolution evaluation (B-cubed metrics). McNemar's test: $p < 0.001$ for Continuum vs. all baselines.

| Method | B-cubed P | B-cubed R | B-cubed F1 |
|---|---|---|---|
| Continuum (cascade) | 0.985 | 0.974 | **0.979** |
| SBERT cosine | 0.891 | 0.858 | 0.874 |
| Fuzzy only | 0.812 | 0.726 | 0.767 |
| Exact only | 0.793 | 0.703 | 0.746 |
| SpaCy NER | 0.583 | 0.468 | 0.521 |

The gap between the cascade and the best single method (SBERT, F1 = 0.874) is 10.5 percentage points—statistically significant and practically meaningful. SpaCy NER performs poorly because it was designed for named entity *recognition*, not *resolution*; it treats "postgres" and "PostgreSQL" as distinct entities.

**⋆ Key Insight**

The pipeline achieves 97.9% F1 not through one clever stage, but through the cascade. Each stage catches a different class of variation: canonical maps handle abbreviations, fuzzy matching handles typos, and embeddings handle semantic synonyms. Remove any stage and the overall score drops.

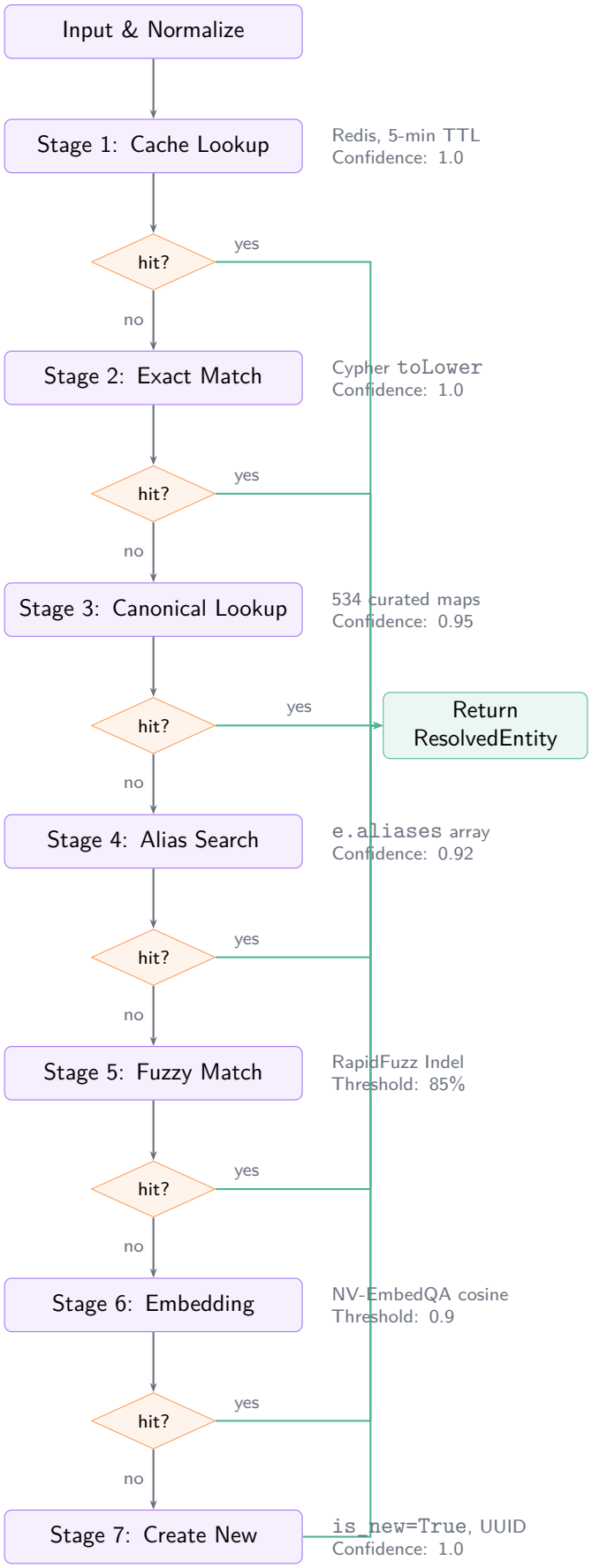




Figure 12.1: The 7-stage entity resolution cascade. Each diamond is a match/no-match decision. On a hit, the pipeline short-circuits to the return node.

# Chapter 13

# Decision Extraction

*The extraction pipeline is the heart of Continuum. It takes a free-form conversation—possibly thousands of tokens of rambling developer–AI dialogue—and distills it into a structured decision trace with six fields. This chapter walks through every stage of that pipeline, from prompt design to JSON repair to caching.*

## 13.1 The Pipeline

The extraction service lives in `apps/api/services/extractor.py`. At a high level the pipeline is five stages:

1. **Input.** A block of conversation text arrives—either from a completed capture session (Chapter 8) or from a bulk JSON import.
2. **Prompt construction.** The text is injected into a few-shot extraction prompt that tells the LLM what a "decision" is and what fields to return.
3. **LLM call.** The prompt is sent to the configured LLM (default: NVIDIA Nemotron 49B) with `temperature=0.3` and `max_tokens=4096`.
4. **JSON parse & repair.** The raw response is cleaned (thinking tags stripped, truncated JSON repaired) and parsed into a Python list of dicts.
5. **Validation & defaults.** Missing fields are filled with safe defaults, and each dict is validated against the `DecisionCreate` schema.

**⋆ Key Insight**

The pipeline is intentionally *stateless*. It takes text in and returns structured JSON out. Persistence to Neo4j, PostgreSQL, and Redis happens in a separate `save_decision()` step—which means you can test extraction in isolation without touching any database.

## 13.2 The Extraction Prompt

The core prompt (`DECISION_EXTRACTION_PROMPT`) is a few-shot, chain-of-thought template. Its structure has three parts:

**1. Definition of "decision."** The prompt defines four categories that the LLM should recognize:

- **Explicit decisions:** "Should we use X or Y? Let's use X because. . . "
- **Implicit decisions:** "Let's use X for this" (no stated alternatives).
- **Technical choices:** Framework selections, architecture patterns, tool adoptions.
- **Implementation strategies:** How to solve a problem, approach to take.

**2. Required fields.** Each decision must include six fields:

Table 13.1: The six fields of a decision trace.

| Field | Description |
|---|---|
| `trigger` | What prompted the decision (problem, requirement, or question). |
| `context` | Background information—constraints, requirements, team situation. |
| `options` | Alternatives considered (may be a single-element list). |
| `decision` | A clear statement of what was chosen. |
| `rationale` | Why this choice was made, or "Not explicitly stated." |
| `confidence` | 0.0–1.0 score reflecting how clear the decision was. |

**3. Output format.** The prompt closes with: "Return ONLY valid JSON, no markdown code blocks or explanation." The expected output is a JSON array of decision objects, or an empty array `[]` if no decisions are found.

The prompt also includes four few-shot examples (single decision, multiple decisions, implicit decision, and no-decision conversation) to anchor the LLM's behavior. An abbreviated version:

Listing 13.1: Abbreviated extraction prompt (see extractor.py for the full version).

```
DECISION_EXTRACTION_PROMPT = """Analyze this conversation
and extract any technical decisions made.

## What constitutes a decision?
A decision is a choice that affects the project direction,
architecture, or implementation. This includes:
- Explicit decisions: "Should we use X or Y? ..."
- Implicit decisions: "Let's use X for this"
- Technical choices: Framework, architecture, tools
- Implementation strategies: How to solve a problem

## Examples
### Example 1: Single clear decision
Conversation: "We need to pick a database. ..."
Output: [{"trigger": "...", "decision": "...", ...}]

### Example 4: No decisions (just discussion)
Output: []

## Conversation to analyze:
{conversation_text}

Return ONLY valid JSON, no markdown code blocks."""
```

Continuum also ships specialized prompts for architecture, technology, and process decisions (`ARCHITECTURE_DECISION_PROMPT`, `TECHNOLOGY_DECISION_PROMPT`, `PROCESS_DECISION_PROMPT`). These narrow the LLM's focus when the decision type is known in advance.

## 13.3 JSON Repair

LLM responses are not always well-formed JSON. Two problems are common:

**Thinking tags.** NVIDIA Nemotron generates `<think>...</think>` blocks as part of its chain-of-thought reasoning. These consume output tokens and are not valid JSON. The system strips them before parsing.

**Truncated output.** When the combined thinking + JSON exceeds `max_tokens`, the response is cut off mid-object. The JSON repair strategy in `utils/json_extraction.py` tries four strategies in order:

1. Parse as pure JSON.
2. Extract from ```` ```json ```` code blocks.
3. Extract from untyped ```` ``` ```` code blocks.
4. Regex fallback: find the last complete `}` followed by `]` and parse the substring.

**△ Warning**

Always use `max_tokens=4096`, not the default 2000. Thinking tags routinely consume 1000–2000 tokens before the model starts producing JSON. With `max_tokens=2000`, roughly 14.5% of extractions fail due to truncation.

## 13.4 Prompt Versioning

LLM extraction responses are cached in Redis to avoid redundant API calls. The cache key includes the prompt version:

```
# Format: llm:{version}:{type}:{md5(text)}
key = f"llm:{version}:extract:{text_hash}"
```

The version string is controlled by `LLM_EXTRACTION_PROMPT_VERSION` in `apps/api/config.py` (default: `"v1"`). When you modify the extraction prompt, bump this value. All cached responses whose keys contain the old version will naturally miss, forcing re-extraction with the new prompt.

Listing 13.2: The cache key generation method.

```
def _get_cache_key(self, text: str, extraction_type: str) -> str:
    text_hash = hashlib.md5(text.encode("utf-8")).hexdigest()
    version = self._settings.llm_extraction_prompt_version
    return f"llm:{version}:{extraction_type}:{text_hash}"
```

Cached responses have a 24-hour TTL (`llm_cache_ttl` = 86 400 s). Entity resolution results are cached separately with a 5-minute TTL.

## 13.5 Default Values

Not every LLM response includes all six fields. The `apply_decision_defaults()` function fills in safe values for anything missing or empty:

Listing 13.3: Default values for incomplete extraction results.

```
DEFAULT_DECISION_FIELDS = {
    "confidence": 0.5,
    "context":    "",
    "rationale":  "",
    "options":    [],
```

```
    "trigger":    "Unknown trigger",
    "decision":   "",
}
```

The function also preserves extra fields not in the defaults dict, so specialized prompts that return `decision_type` or other custom fields are not silently dropped.

**▷ Try It**

Run a single extraction manually to see defaults in action:

```
curl -X POST http://localhost:8000/api/ingest \
  -H "Content-Type: application/json" \
  -d '{"text": "Let us use Redis for caching."}'
```

The response will show `confidence: 0.85` (from the LLM) but `context: ""` and `options: ["Redis"]` (both filled by the model), demonstrating that the LLM usually provides most fields when the conversation is clear.

## 13.6 Gotchas

**△ Warning**

**Prompt sanitizer blocks synthetic conversations.** The LLM client includes a prompt injection detector that flags "USER:" and "ASSISTANT:" labels as potential attacks. Synthetic conversations (used in evaluation) naturally contain these markers. Always pass `sanitize_input=False` when calling the LLM on synthetic or imported conversation text.

**△ Warning**

**Multiple extraction runs accumulate.** The `save_decision()` method *creates* new Neo4j nodes—it does not upsert. If you run the extraction pipeline twice on the same conversations, you get duplicate decision traces. Always wipe Neo4j (`MATCH (n) DETACH DELETE n`) before starting a new evaluation run.

**△ Warning**

**True extraction yield is approximately 1.9 decisions per conversation** with an 85.5% success rate across 200 synthetic conversations. If you see significantly higher numbers, you are likely looking at accumulated results from multiple runs.

# Chapter 14

# Backend API Layer

The backend is a FastAPI application that sits between the Next.js frontend and three databases (PostgreSQL, Neo4j, Redis). It registers 12 routers, applies 4 middleware layers, and reshapes raw database records into the contracts the frontend expects. This chapter explains how the pieces fit together.

## 14.1 Application Structure

The entry point is `apps/api/main.py`. At startup, it:

1. Initializes connection pools for PostgreSQL (async SQLAlchemy), Neo4j (async Bolt driver), and Redis.
2. Registers 4 middleware layers (applied bottom-to-top on each request):
   - **CORS** — allows the frontend origin only, with credentials.
   - **GZip** — compresses responses larger than 1,000 bytes.
   - **Request size limit** — rejects bodies exceeding 10 MB (SEC-010).
   - **Security headers** — adds `X-Content-Type-Options`, `X-Frame-Options`, `Strict-Transport-Security`, and others (DEVOPS-P2-1).
3. Registers 12 routers under the `/api` prefix.
4. Installs global exception handlers for validation errors, HTTP exceptions, and circuit-breaker opens.

## 14.2 Router Organization

The 12 routers are grouped by subsystem. Table 14.1 shows each router, its URL prefix, and its responsibility.

## 14.3 Authentication Flow

Continuum uses Auth.js v5 (NextAuth) on the frontend and JWT verification on the backend. The flow is:

1. The user signs in via the Next.js frontend. Auth.js issues a JWT signed with `NEXTAUTH_SECRET`.
2. Every API request from the frontend includes this JWT in the `Authorization: Bearer <token>` header.
3. The backend verifies the JWT signature using `SECRET_KEY` from its own environment.
4. On success, the backend extracts `user_id` from the JWT payload and injects it into the request context.
5. On failure (expired, malformed, or wrong secret), the request falls back to the "anonymous" user—which has no data.

Table 14.1: Backend routers grouped by subsystem.

| Subsystem | Router | Prefix | Responsibility |
|---|---|---|---|
| Capture | `capture.py` | `/api/capture` | Real-time capture of developer–AI conversations. |
| Extraction | `ingest.py` | `/api/ingest` | Bulk ingestion of conversation logs. |
| | `agent.py` | `/api/agent` | Agent context for MCP tool integration. |
| Graph | `graph.py` | `/api/graph` | Graph visualization data (nodes, edges, stats). |
| | `entities.py` | `/api/entities` | CRUD operations on entity nodes. |
| | `decisions.py` | `/api/decisions` | CRUD operations on decision traces. |
| Search | `search.py` | `/api/search` | Fulltext and hybrid search. |
| | `ask.py` | `/api/ask` | GraphRAG Q&A with SSE streaming. |
| Management | `projects.py` | `/api/projects` | Multi-project workspace management. |
| | `dashboard.py` | `/api/dashboard` | Aggregated statistics and charts. |
| | `export.py` | `/api/export` | Data export (JSON, CSV). |
| | `users.py` | `/api/users` | User registration, profile, and management. |

**△ Warning**

`NEXTAUTH_SECRET` (in `apps/web/.env.local`) and `SECRET_KEY` (in `apps/api/.env`) must be identical. A mismatch causes silent authentication failure: the user appears to be logged in on the frontend but sees an empty graph on the backend because all queries scope to "anonymous."

## 14.4 Multi-Tenant Isolation

Every Neo4j query that touches user data must include a user-scoping `WHERE` clause. This is Continuum's primary mechanism for multi-tenant isolation: all users share a single Neo4j database, and their data is separated by the `user_id` property on `DecisionTrace` nodes.

The correct pattern:

Listing 14.1: Correct: user-scoped query.

```
MATCH (d:DecisionTrace)-[:INVOLVES]->(e:Entity)
WHERE d.user_id = $user_id OR d.user_id IS NULL
RETURN e.name, e.type
```

The incorrect pattern:

Listing 14.2: Incorrect: missing user scope.

```
MATCH (d:DecisionTrace)-[:INVOLVES]->(e:Entity)
RETURN e.name, e.type
```

The `OR d.user_id IS NULL` clause handles legacy data and system-level entities that are shared across all users.

**△ Warning**

A missing `user_id` filter is a **security bug**. It leaks one user's decisions and entities to every other user. Every new Cypher query in the codebase must include the user-scoping pattern shown above.

## 14.5 SSE Streaming

The `/api/ask` endpoint streams its response as Server-Sent Events (SSE) rather than returning a single JSON payload. This allows the frontend to display tokens as they arrive from the LLM.

The stream emits four event types:

Table 14.2: SSE event types in the ask endpoint.

| Event | Payload & Purpose |
|---|---|
| `context` | `{"sources": [...], "context_length": N}` — The retrieved graph context and source nodes, sent once before the first token. |
| `token` | `{"text": "..."}` — A chunk of the LLM's response. Sent repeatedly as tokens stream from the model. |
| `done` | `{"token_count": N}` — End-of-stream marker. The frontend stops its loading indicator. |
| `error` | `{"detail": "..."}` — An error occurred during retrieval or generation. The frontend shows an error message. |

Each event is serialized with a helper function that formats the SSE protocol:

Listing 14.3: SSE event formatter from `routers/ask.py`.

```
1 def _sse_event(event: str, data: dict) -> str:
2     return f"event: {event}\ndata: {json.dumps(data)}\n\n"
```

## 14.6 The Reshaping Layer

Neo4j returns flat dictionaries of node properties. The frontend expects structured, typed objects. The routers bridge this gap by reshaping raw database results into frontend-ready contracts.

For example, the `ask.py` router transforms raw Neo4j `DecisionTrace` and `Entity` nodes into `AskSourceNode` objects that the frontend's `<AskPanel>` component expects. Similarly, `search.py` reshapes into `SearchResult` objects with highlighted snippets.

This reshaping is intentionally done in the router layer (not in the service layer) because different endpoints may need different projections of the same underlying data. The service layer returns raw data; the router layer decides what the frontend sees.

**▷ Try It**

Start the backend (`pnpm dev:api`) and open `http://localhost:8000/docs` in your browser. FastAPI auto-generates an OpenAPI UI where you can test every endpoint, see request/response schemas, and try authenticated requests by pasting your JWT into the Authorize dialog.

# Chapter 15

# GraphRAG & Search

When a user asks "Why did I choose PostgreSQL over MongoDB?", Continuum must find the relevant decisions in a graph of thousands of nodes, assemble enough context for the LLM to answer accurately, and stream the response in real time. This chapter explains the hybrid retrieval pipeline that makes this possible.

## 15.1 Hybrid Search

Continuum runs two search methods in parallel and fuses their results:

1. **Fulltext search** uses Neo4j's built-in Lucene indexes. Two indexes cover 6 fields:
   - `decision_fulltext` — indexes `trigger`, `context`, `decision`, `rationale`, `agent_decision`, and `agent_rationale` on `DecisionTrace` nodes.
   - `entity_fulltext` — indexes `name` on `Entity` nodes.

   Fulltext search is fast, deterministic, and handles exact keyword matches well. It struggles with semantic similarity ("database" does not match "PostgreSQL").
2. **Vector search** uses NVIDIA NV-EmbedQA (2048-dimensional embeddings) stored on Neo4j nodes. Two vector indexes cover decisions and entities:
   - `decision_embedding` — cosine similarity on `DecisionTrace` embeddings.
   - `entity_embedding` — cosine similarity on `Entity` embeddings.

   Vector search handles paraphrases and semantic relationships but can miss exact keywords.

   Both searches are user-scoped: fulltext results are filtered by `user_id`, and vector results require the node to be connected to the user's decisions.

## 15.2 Reciprocal Rank Fusion

The two ranked lists are combined using Reciprocal Rank Fusion (RRF), a simple, parameter-light fusion method from Cormack et al.:

$$\text{Score}(d) = \frac{1}{k + \text{rank}_{\text{fulltext}}(d)} + \frac{1}{k + \text{rank}_{\text{vector}}(d)}$$

where $k = 60$ is a smoothing constant that prevents top-ranked results from dominating the fused score. Documents that appear in both lists receive contributions from both terms; documents that appear in only one list receive a single term.

Listing 15.1: RRF implementation from `services/graph_rag.py`.

```
RRF_K = 60

def rrf_fuse(
    fulltext_ids: list[str],
    vector_ids: list[str],
    k: int = RRF_K,
```

```
) -> list[str]:
    scores: dict[str, float] = {}
    for rank, doc_id in enumerate(fulltext_ids, start=1):
        scores[doc_id] = scores.get(doc_id, 0.0) + 1.0 / (k + rank)
    for rank, doc_id in enumerate(vector_ids, start=1):
        scores[doc_id] = scores.get(doc_id, 0.0) + 1.0 / (k + rank)
    return sorted(scores, key=lambda d: scores[d], reverse=True)
```

RRF was chosen over learned fusion (e.g., a cross-encoder reranker) because it requires no training data, adds negligible latency, and performs well when both retrievers are reasonably accurate.

## 15.3 The Pipeline

Figure 15.1 shows the full retrieval pipeline from query to streamed answer.

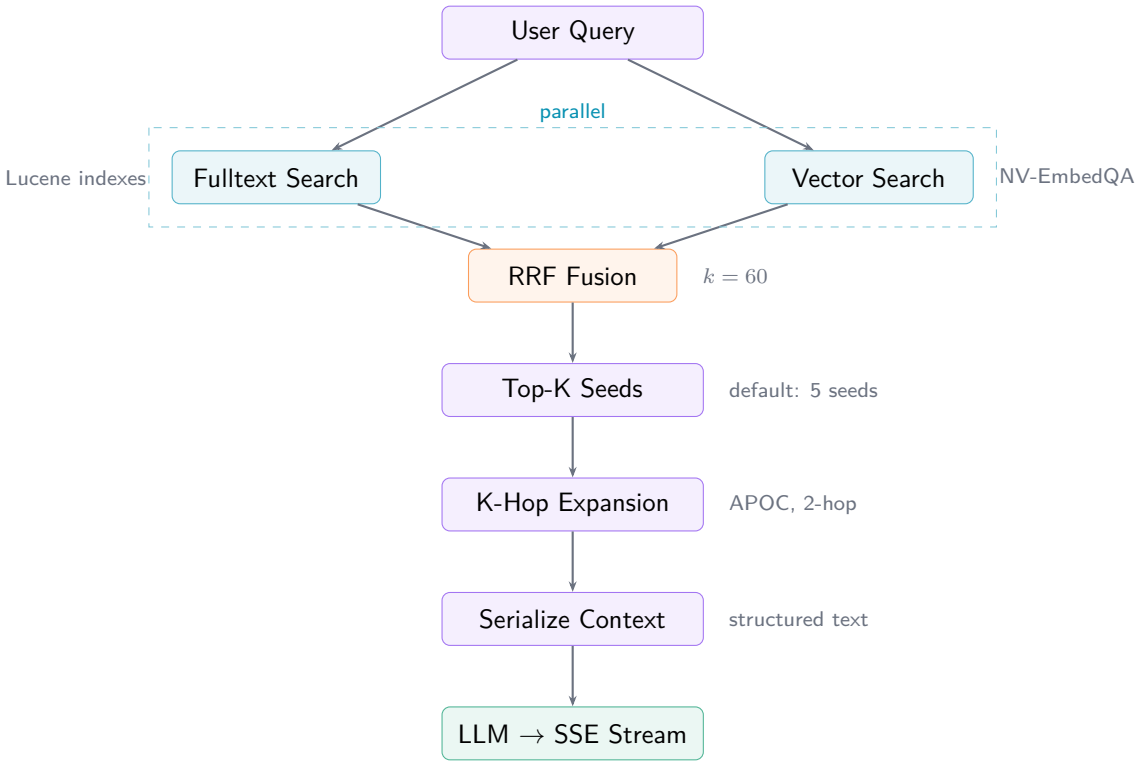


Figure 15.1: The GraphRAG retrieval pipeline. Fulltext and vector searches run in parallel on separate Neo4j sessions, then fuse via RRF.

The five steps in detail:

1. **Hybrid retrieval**: Fulltext and vector searches run concurrently (on separate Neo4j sessions, since `AsyncSession` is not safe for concurrent use). Results are fused with RRF.
2. **Top-K seed selection**: The top 5 fused IDs become seed nodes for graph expansion. The default of 5 balances context richness against token budget.
3. **K-hop expansion**: Starting from seed nodes, APOC's `apoc.path.subgraphAll` traverses up to 2 hops outward, collecting related decisions and entities. The result is capped at `MAX_CONTEXT_NODES = 50` to prevent context explosion.

4. **Serialization**: The subgraph (nodes + edges) is converted to structured Markdown text: decisions with their trigger, rationale, and context; entities grouped by type; and relationships as labeled edges.
5. **LLM generation**: The serialized context is injected into the LLM prompt. The response is streamed as SSE events (see Chapter 14, Section 14.5).

## 15.4 Implementation Details

The pipeline lives in `services/graph_rag.py` as the `GraphRAGService` class.

- **Constructor**: `GraphRAGService()` takes no arguments. The embedding service is initialized internally. Neo4j sessions are passed as keyword arguments to individual methods.
- **Entry point**: `retrieve_context()` is the main method, returning a 3-tuple:

```
async def retrieve_context(
    self, query: str, user_id: str,
    top_k: int = 5, depth: int = 2, session=None,
) -> tuple[dict, str, list[str]]:
    # Returns (subgraph_dict, context_string, seed_ids)
```

- **Fallback**: If vector search fails (embedding service down, model timeout), the pipeline falls back to fulltext-only retrieval. The `rrf_fuse` function handles an empty vector list gracefully—fulltext ranks pass through unchanged.
- **Singleton**: The service is instantiated once via `get_graph_rag_service()` and reused across requests.

## 15.5 Ablation

Table 15.1 shows how each component contributes to retrieval quality. "Recall" measures what fraction of relevant decisions appear in the top-5 results.

Table 15.1: GraphRAG retrieval ablation. Removing fulltext search causes the largest quality drop.

| Configuration | Recall@5 | Latency Impact |
|---|---|---|
| Full pipeline (fulltext + vector + cache) | 100% | baseline |
| No fulltext (vector only) | 13.3% | — |
| No vector (fulltext only) | 86.7% | — |
| No cache | 100% | 4× slower |
| No K-hop expansion (seeds only) | 60% | 2× faster |

**⋆ Key Insight**

Fulltext search is the backbone. Without it, the system misses 87% of relevant results. Vector search contributes the remaining 13% by finding semantically related decisions that do not share keywords with the query. The two methods are complementary, not redundant.

**▷ Try It**

Run a GraphRAG query from the command line to see the pipeline in action:

```
cd apps/api
.venv/bin/python evaluation/query_graphrag.py \
    --query "Why did I choose PostgreSQL?"
```

The script prints the seed IDs, expanded subgraph stats, serialized context, and the LLM's answer.

# Chapter 16

# Frontend Architecture

The Continuum frontend is a Next.js 16 application using the App Router, React 19, Tailwind CSS v4, and shadcn/ui. This chapter covers the routing structure, the Nebula design system, component patterns, graph visualization, and state management.

## 16.1 Next.js 16 App Router

Continuum uses the App Router (the `app/` directory), not the legacy Pages Router. Key concepts:

- **Layouts** (`layout.tsx`): Wrap child routes with shared UI (sidebar, navigation, providers). The root layout at `app/layout.tsx` applies the Instrument Sans font, theme provider, session provider, and React Query provider.
- **Pages** (`page.tsx`): The actual content for a route. Each route segment maps to a directory, and the `page.tsx` inside it renders the page content.
- **Route groups** (`(group)/`): Organize routes without affecting the URL. For example, `app/(dashboard)/` groups all authenticated pages under a shared layout without adding "dashboard" to the URL path.
- **Server Components**: By default, every component in the `app/` directory is a React Server Component. Components that use browser APIs, hooks, or event handlers must be marked with `'use client'` at the top of the file.

**Adding a new page.** To add a page at `/settings`:

1. Create `app/(dashboard)/settings/page.tsx`.
2. Export a default React component.
3. The route inherits the dashboard layout automatically.
4. Add a navigation link in `components/layout/Sidebar.tsx`.

## 16.2 Nebula Design System

Continuum's visual identity is called "Nebula"—a dark-first theme built on violet, rose, and orange accent colors with glassmorphism effects. All colors are defined as CSS custom properties in `app/globals.css` and consumed via Tailwind's semantic classes.

### 16.2.1 Semantic Color Tokens

### 16.2.2 Entity Type Colors

Entities in the graph and in badges are color-coded by type:

These colors are used consistently across the graph visualization, dashboard charts, entity badges, and search result cards.

Table 16.1: Semantic color tokens (light / dark).

| Token | Light | Dark | Usage |
|---|---|---|---|
| `-primary` | Violet 48% | Violet 58% | Primary actions, focus rings |
| `-accent` | Rose 50% | Rose 60% | Secondary accents, highlights |
| `-destructive` | Red 51% | Red 45% | Delete, error states |
| `-muted` | Gray 94% | Dark 14% | Disabled, secondary text |
| `-background` | Off-white | Space 6% | Page background |
| `-card` | White | Glass 10% | Card surfaces |

Table 16.2: Entity type color assignments.

| Entity Type | Color | Hex |
|---|---|---|
| Technology | Orange | `#fb923c` |
| Pattern | Rose | `#ec4899` |
| Concept | Violet | `#a78bfa` |
| Person | Emerald | `#34d399` |
| System | Green | `#4ade80` |

## 16.3 Component Patterns

The codebase follows several conventions that ensure consistency:

### 16.3.1 `cn()` for Class Names

All className composition uses the `cn()` utility from `lib/utils.ts`, which merges Tailwind classes with `clsx` and resolves conflicts with `tailwind-merge`:

Listing 16.1: Using `cn()` for conditional classes.

```
<div className={cn(
  "rounded-lg border p-4",
  variant === "glass" && "backdrop-blur-xl bg-card/50",
  className
)} />
```

Never concatenate class strings manually—`cn()` handles deduplication and conflict resolution.

### 16.3.2 CVA for Variants

Component variants are defined using Class Variance Authority (CVA):

Listing 16.2: CVA variant definition for a Button.

```
const buttonVariants = cva("base-classes rounded-lg font-medium", {
  variants: {
    variant: {
      default: "bg-primary text-primary-foreground",
      glass: "backdrop-blur-xl bg-card/50 border-white/10",
      gradient: "bg-gradient-to-r from-violet-500 via-fuchsia-500 ...",
      destructive: "bg-destructive text-destructive-foreground",
```

```
    },
    size: {
      default: "h-10 px-4",
      sm: "h-8 px-3 text-sm",
      lg: "h-12 px-6 text-lg",
    },
  },
  defaultVariants: { variant: "default", size: "default" },
})
```

### 16.3.3 `React.forwardRef`

All UI components use `React.forwardRef` so that parent components can attach refs for focus management, scroll control, and animations:

Listing 16.3: ForwardRef pattern.

```
const Card = React.forwardRef<HTMLDivElement, CardProps>(
  ({ className, variant, ...props }, ref) => (
    <div ref={ref} className={cn(cardVariants({ variant }), className)}
         {...props} />
  )
)
Card.displayName = "Card"
```

### 16.3.4 Compound Components

Complex components export multiple related sub-components: `Card`, `CardHeader`, `CardTitle`, `CardContent`, `CardFooter`. This gives consumers full control over layout without exposing internal implementation details.

## 16.4 Graph Visualization

The knowledge graph is rendered using React Flow (`@xyflow/react`) with custom node types and automatic layout via Dagre.

- **Custom node types**:
  - *Decision nodes* render as gradient cards with a violet-to-rose background, showing the trigger text and a confidence badge.
  - *Entity nodes* render as colored pills, tinted by entity type (orange for technology, rose for pattern, etc.).
- **Layout algorithms**: Dagre provides four layout modes: force-directed, hierarchical, clustered, and radial. The user can switch between them from the graph toolbar.
- **Interaction**: Arrow keys for panning, Tab/Enter for node selection, Escape to deselect. A minimap and zoom controls are always visible.
- **Performance**: Large graphs (500+ nodes) use `React.memo` with custom comparison functions on node components to prevent unnecessary re-renders.

## 16.5 State Management

Continuum does not use a global state library like Redux. Instead, it combines several purpose-specific tools:

Table 16.3: State management strategy.

| Kind of State | Tool | Details |
|---|---|---|
| Server / async | React Query | 60-second stale time. Queries auto-refetch on window focus. Mutations invalidate related queries. |
| Form | react-hook-form + Zod | Schema-first validation. Zod schemas are shared with backend Pydantic models where possible. |
| Theme | next-themes | System / light / dark. The `.dark` class on `<html>` toggles CSS custom properties. |
| Auth session | NextAuth v5 | `SessionProvider` at the root layout. `useSession()` hook in client components. |
| Local / ephemeral | `useState` / `useReducer` | Component-level state for UI toggles, form drafts, modal open/close. |

## 16.6 Gotchas

Several non-obvious pitfalls have tripped up contributors. These are documented here to save you debugging time.

**△ Warning**

**Radix ScrollArea breaks `scrollTop`.** Radix's `ScrollArea` component sets `overflow: hidden` on its root element, which prevents programmatic scroll control via `scrollTop`. If you need auto-scrolling (e.g., the chat panel in the Ask view), use a plain `<div>` with `overflow-y-auto` instead.

**△ Warning**

**React Flow hook ordering.** The `useNodesState` and `useEdgesState` setter functions must be declared *before* any `useCallback` that references them. React hooks must always be called in the same order, and referencing a setter before its declaration causes stale-closure bugs that are silent in development but break in production.

**△ Warning**

**CSS `:root` in dark mode.** Because `next-themes` applies the `.dark` class on `<html>`, bare `:root` selectors apply in *both* light and dark mode. Use `:root:not(.dark)` for light-mode-only overrides. Getting this wrong causes colors to bleed between themes.

**▷ Try It**

Explore the design system live: start the frontend (`pnpm dev:web`), navigate to the graph view, and try switching between layout modes. Open the browser DevTools and inspect a decision node to see the Nebula color tokens in action.

# Part V

# Extending Continuum

# Chapter 17

# How to Add Features

*This chapter provides seven self-contained recipes for extending Continuum. Each recipe identifies what you are adding, which files to touch, the concrete steps, a code snippet, a verification command, and common gotchas. Work through them in order the first time—Recipe 17.1 is a two-minute change that builds confidence for the harder ones.*

**⋆ Key Insight**

Start with Recipe 17.1 (Add a Canonical Mapping). It is the smallest possible change that touches the entity resolution pipeline, and the benchmark will immediately tell you whether you got it right.

## 17.1 Add a Canonical Mapping

**What you are adding.** A new alias → canonical-name entry in the entity resolution dictionary so that a surface form like `k8s` resolves to `Kubernetes`.

**Files to touch.**

- apps/api/models/ontology.py — the `CANONICAL_NAMES` dictionary (currently 534 entries).

**Steps.**

1. Open apps/api/models/ontology.py.
2. Locate the `CANONICAL_NAMES` dictionary (around line 271).
3. Add your mapping inside the appropriate category section. Keys are lowercase; values are the display-cased canonical form.
4. Save the file.
5. Run the synthetic benchmark to verify.

**Code snippet.**

Adding a Kubernetes alias

```
CANONICAL_NAMES: dict[str, str] = {
    # ... existing entries ...
    # ============================================================
    # CONTAINERIZATION & ORCHESTRATION
    # ============================================================
    "kubernetes": "Kubernetes",
    "k8s": "Kubernetes",        # already present
    "kube": "Kubernetes",       # already present
    "k8": "Kubernetes",         # <-- NEW: common typo/shorthand
    # ...
}
```

**Verification.**

```
cd apps/api
python -m evaluation.synthetic_benchmark
```

The benchmark generates surface-form variants for every canonical name and checks that each resolves correctly. A passing run confirms your new mapping is well-formed.

**Gotchas.**

- Keys *must* be lowercase. The lookup calls `name.lower()` before hitting the dictionary.
- Do not map a key that already exists with a different canonical value—Python will silently overwrite the earlier entry.
- If your canonical name is new (not yet in the dictionary as a value), also add the well-known abbreviations to `KNOWN_ABBREVIATIONS` in `evaluation/synthetic_benchmark.py` so the benchmark can generate test variants.

## 17.2 Add an Entity Type

**What you are adding.** A new category of node in the knowledge graph (e.g. `METRIC`, `STANDARD`).

**Files to touch.**

1. `apps/api/models/ontology.py` — add to `EntityType` enum.
2. `apps/api/models/ontology.py` — update `VALID_ENTITY_RELATIONSHIPS` to declare which relationship types are legal for the new entity type.
3. `apps/web/` — frontend color map (entity badge colors).
4. `apps/web/` — graph node style (shape, size, color in the React Flow renderer).

**Steps.**

1. Add the new member to `EntityType`:

```python
class EntityType(Enum):
    TECHNOLOGY = "technology"
    CONCEPT = "concept"
    PATTERN = "pattern"
    SYSTEM = "system"
    PERSON = "person"
    ORGANIZATION = "organization"
    METRIC = "metric"  # <-- NEW
```

2. For every relationship type in `VALID_ENTITY_RELATIONSHIPS`, decide whether your new type participates. Add the relevant `(source, target)` tuples:

```python
"RELATED_TO": {
    # ... existing pairs ...
    ("metric", "technology"),    # e.g., "latency RELATED_TO Redis"
    ("metric", "concept"),       # e.g., "throughput RELATED_TO
        caching"
},
```

3. In the frontend, add a color entry for the new type so badges and graph nodes render correctly. Search for the existing `ENTITY_TYPE_COLORS` or equivalent mapping and add your type.
4. Restart both backend and frontend, then ingest a conversation that mentions your new entity type.

**Verification.**

```
# Backend: run the test suite
cd apps/api && .venv/bin/pytest tests/ -v -k entity

# Frontend: check TypeScript compiles
cd apps/web && pnpm typecheck
```

**Gotchas.**

- If you forget to update `VALID_ENTITY_RELATIONSHIPS`, the validator (`services/validator.py`) will reject relationships involving the new type at runtime.
- The extraction prompt in `services/extractor.py` enumerates valid entity types. Update the prompt so the LLM knows to produce the new type.
- Remember to bump `LLM_EXTRACTION_PROMPT_VERSION` if you change the prompt (see Recipe 17.7).

## 17.3 Add a Relationship Type

**What you are adding.** A new edge kind in the knowledge graph (e.g. `REPLACES`, `COMPLEMENTS`).

**Files to touch.**

1. `apps/api/models/ontology.py` — `RelationType` enum, `VALID_ENTITY_RELATIONSHIPS`, and the appropriate frozen set (`ENTITY_ONLY_RELATIONSHIPS`, `DECISION_ONLY_RELATIONSHIPS`, or `DECISION_ENTITY_RELATIONS`
2. `apps/api/services/extractor.py` — detection logic in the extraction prompt.
3. `apps/web/` — edge style (color, dash pattern, label) in the graph renderer.

**Steps.**

1. Add the new member to `RelationType`:

```
class RelationType(Enum):
    # ... existing members ...
    COMPLEMENTS = "COMPLEMENTS"  # X works well alongside Y
```

2. Declare valid source/target type pairs:

```
VALID_ENTITY_RELATIONSHIPS["COMPLEMENTS"] = {
    ("technology", "technology"),  # Redis COMPLEMENTS PostgreSQL
    ("pattern", "pattern"),        # CQRS COMPLEMENTS Event Sourcing
}
```

3. Add the new type to the correct frozen set:

```
ENTITY_ONLY_RELATIONSHIPS: frozenset[str] = frozenset([
    # ... existing ...
    "COMPLEMENTS",
])
```

4. Update the extraction prompt in `services/extractor.py` to include the new relationship type and describe when the LLM should use it.
5. Add an edge style in the frontend graph renderer (color, line dash, arrowhead).
6. Bump `LLM_EXTRACTION_PROMPT_VERSION`.

**Verification.**

```
cd apps/api
.venv/bin/pytest tests/ -v -k relationship
```

```
python -c "from models.ontology import validate_entity_relationship; \
  print(validate_entity_relationship('COMPLEMENTS','technology','
     technology'))"
```

The second command should print `(True, None)`.

**Gotchas.**

- `ALL_RELATIONSHIP_TYPES` is a union of the three frozen sets. It recomputes automatically, but only if you added your type to one of them.
- The `validate_entity_relationship()` function normalizes inputs to uppercase/lowercase, so the enum value casing must be all-uppercase.

## 17.4 Add an API Endpoint

**What you are adding.** A new REST endpoint on the FastAPI backend.

**Files to touch.**

1. apps/api/routers/*your_module*.py — new router file.
2. apps/api/main.py — register the router with `app.include_router()`.
3. Pydantic request/response models (inline or in models/).

**Steps.**

1. Create the router file:

routers/health_check.py

```python
"""Health-check router with database connectivity test."""

from fastapi import APIRouter, Depends
from pydantic import BaseModel

router = APIRouter()

class HealthResponse(BaseModel):
    status: str
    neo4j: bool
    postgres: bool

@router.get("/", response_model=HealthResponse)
async def health_check():
    """Return service health with dependency status."""
    # TODO: actual connectivity checks
    return HealthResponse(
        status="ok",
        neo4j=True,
        postgres=True,
    )
```

2. Register the router in main.py:

```python
from routers import health_check

app.include_router(
    health_check.router,
    prefix="/api/health",
```

```
    tags=["Health"],
)
```

3. If the endpoint needs user context, add the `user_id` dependency that existing routers use for per-user data isolation.
4. Write tests in `tests/routers/test_health_check.py`.

**Verification.**

```
# Start backend
cd apps/api && .venv/bin/uvicorn main:app --reload --port 8000

# In another terminal
curl -s http://localhost:8000/api/health/ | python -m json.tool
```

**Gotchas.**

- Every data-returning endpoint should filter by `user_id` to maintain per-user isolation. Review existing routers like `routers/decisions.py` for the pattern.
- Pydantic strict mode is used project-wide. Use explicit type annotations; implicit coercion will raise validation errors.
- The trailing slash matters. FastAPI redirects `/api/health` to `/api/health/` by default.

## 17.5 Add an LLM Provider

**What you are adding.** A new LLM backend (e.g. Google Vertex AI, Ollama) alongside the existing NVIDIA NIM and Amazon Bedrock providers.

**Files to touch.**

1. `apps/api/services/llm_providers/your_provider.py` — implement `BaseLLMProvider` (and optionally `BaseEmbeddingProvider`).
2. `apps/api/services/llm_providers/__init__.py` — register in the factory functions.
3. `apps/api/config.py` — add any new environment variables.

**Steps.**

1. Create your provider class by implementing the three abstract members of `BaseLLMProvider`:

llm_providers/ollama.py

```
"""Ollama LLM provider for local model inference."""

from typing import AsyncIterator
from services.llm_providers.base import BaseLLMProvider

class OllamaLLMProvider(BaseLLMProvider):
    async def generate(
        self,
        messages: list[dict],
        temperature: float = 0.6,
        max_tokens: int = 4096,
    ) -> tuple[str, dict]:
        # Call Ollama REST API
        ...

```

```
    async def generate_stream(
        self,
        messages: list[dict],
        temperature: float = 0.6,
        max_tokens: int = 4096,
    ) -> AsyncIterator[str]:
        # Stream from Ollama REST API
        ...

    @property
    def model_name(self) -> str:
        return "ollama/llama3"
```

2. Register in the factory (`llm_providers/__init__.py`):

```
def get_llm_provider() -> BaseLLMProvider:
    settings = get_settings()
    provider_name = getattr(settings, "llm_provider", "nvidia")

    if provider_name == "bedrock":
        from services.llm_providers.bedrock import BedrockLLMProvider
        return BedrockLLMProvider()
    elif provider_name == "ollama":            # <-- NEW
        from services.llm_providers.ollama import OllamaLLMProvider
        return OllamaLLMProvider()
    else:
        from services.llm_providers.nvidia import NvidiaLLMProvider
        return NvidiaLLMProvider()
```

3. Add config variables to `config.py`:

```
# In Settings class
ollama_base_url: str = "http://localhost:11434"
ollama_model: str = "llama3"
```

4. Set `LLM_PROVIDER=ollama` in your `.env` file.

**Verification.**

```
LLM_PROVIDER=ollama python -c "
from services.llm_providers import get_llm_provider
p = get_llm_provider()
print(type(p).__name__, p.model_name)
"
```

**Gotchas.**

- The `generate()` method must return a `(text, usage_dict)` tuple. The `usage_dict` should contain `prompt_tokens`, `completion_tokens`, and `total_tokens` for token logging.
- If your provider does not support streaming, raise `NotImplementedError` from `generate_stream()` rather than silently failing.
- The `LLMClient` in `services/llm.py` wraps providers with rate limiting and retry logic. You do not need to implement retries inside your provider.

## 17.6 Add an Evaluation Script

**What you are adding.** A new evaluation or analysis script that follows the project's conventions.

**Files to touch.**

1. apps/api/evaluation/*your_script*.py — new script.

**Steps.**

1. Follow the established pattern: argparse for CLI arguments, JSON output for machine-readable results, and use synthetic conversations as input data.

evaluation/my_baseline.py

```python
"""TF-IDF cosine similarity baseline for entity resolution.

Usage:
    python -m evaluation.my_baseline [--output PATH]
"""

import argparse
import json
from pathlib import Path

def main():
    parser = argparse.ArgumentParser(
        description="TF-IDF baseline evaluation"
    )
    parser.add_argument(
        "--output",
        type=Path,
        default=Path("evaluation/data/v5/tfidf_results.json"),
    )
    args = parser.parse_args()

    # Load synthetic conversations
    conv_dir = Path("evaluation/data/synthetic_conversations")
    conversations = []
    for f in sorted(conv_dir.glob("*.json")):
        conversations.append(json.loads(f.read_text()))

    # Run evaluation ...
    results = {"precision": 0.0, "recall": 0.0, "f1": 0.0}

    # Write results
    args.output.write_text(
        json.dumps(results, indent=2)
    )
    print(f"Results written to {args.output}")

if __name__ == "__main__":
    main()
```

2. Use the same train/test split as the benchmark (140/60) to ensure comparable results.
3. Output JSON with at least `precision`, `recall`, and `f1` keys.

**Verification.**

```
cd apps/api
python -m evaluation.my_baseline --output /tmp/test_results.json
cat /tmp/test_results.json
```

**Gotchas.**

- Always run from `apps/api/` so that relative imports resolve correctly (`sys.path` manipulation at the top of each evaluation script handles this).
- Write results to `evaluation/data/v5/` for consistency with existing baselines.
- If your script calls the LLM, use `sanitize_input=False` when passing synthetic conversations to avoid the prompt sanitizer blocking "USER:"/"ASSISTANT:" labels.

## 17.7 Modify the Extraction Prompt

**What you are adding.** Changes to the LLM prompt template that governs decision extraction from conversations.

**Files to touch.**

1. `apps/api/services/extractor.py` — the prompt template string.
2. `apps/api/config.py` — the `llm_extraction_prompt_version` setting.

**Steps.**

1. Edit the prompt in `services/extractor.py`.
2. Open `config.py` and bump `llm_extraction_prompt_version` (e.g. from `"v1"` to `"v2"`).
3. Re-run extraction on a small sample to verify the new prompt produces valid JSON output.

> **△ Warning**
>
> Always bump the prompt version after changing the prompt template. Continuum uses a Redis-backed LLM response cache keyed partly by prompt version. If you change the prompt without bumping the version, the system will continue serving *old cached responses* generated by the previous prompt. This is a silent correctness bug that is difficult to diagnose.

**Verification.**

```
cd apps/api

# Test on a single conversation
python -c "
import asyncio, json
from services.extractor import DecisionExtractor

async def test():
    ext = DecisionExtractor()
    result = await ext.extract_from_conversation(
        json.load(open(
            'evaluation/data/synthetic_conversations/conv-001.json'
        )),
        sanitize_input=False,
    )
    print(json.dumps(result, indent=2, default=str))
```

```
asyncio.run(test())
"
```

**Gotchas.**

- NVIDIA Nemotron generates `<think>...</think>` blocks that consume tokens. Set `max_tokens=4096` or higher to avoid truncated JSON output.
- The `strip_thinking_tags()` function in `services/llm.py` removes thinking blocks automatically, but only *after* the full response is received.
- After a prompt change, run the full evaluation suite (`evaluation/run_all.py`) to measure impact on B-cubed metrics before merging.

# Chapter 18

# Exercises

*Six hands-on exercises that progressively deepen your understanding of Continuum. Each exercise states a learning objective, difficulty level, estimated time, the files you will work in, step-by-step instructions, and acceptance criteria. Hints are provided at the end of each exercise for when you get stuck.*

## 18.1 Ontology Expansion

**Difficulty:** Easy **Time:** ∼2 hours **Learning objective:** Understand canonical mappings and the synthetic benchmark.

**Files.**

- apps/api/models/ontology.py
- apps/api/evaluation/synthetic_benchmark.py

**Instructions.**

1. Pick a technical domain you know well (e.g. game development, embedded systems, bioinformatics).
2. Add 20 new canonical mappings to the `CANONICAL_NAMES` dictionary in ontology.py. For each canonical name, add at least two surface-form aliases (e.g. `"unreal"` → `"Unreal Engine"`, `"ue5"` → `"Unreal Engine"`).
3. For any canonical name that has well-known abbreviations, add them to `KNOWN_ABBREVIATIONS` in synthetic_benchmark.py.
4. Run the benchmark and confirm all new mappings pass.

▷ **Try It**

**Acceptance criteria:**

- 20 new entries in `CANONICAL_NAMES`, each with a lowercase key and properly cased canonical value.
- `python -m evaluation.synthetic_benchmark` runs to completion with no failures.
- No existing mappings are broken (the benchmark tests all of them).

**Hints.**

- Look at the existing category sections (`DATABASES`, `AI/ML FRAMEWORKS`, etc.) for formatting conventions.
- Remember: keys must be lowercase. The resolver calls `name.lower()` before lookup.
- If you are unsure whether a name already exists, search the file for the canonical form first.

## 18.2 Graph Layout Enhancement

**Difficulty:** Easy–Medium **Time:** ∼3 hours **Learning objective:** Work with the React Flow graph renderer and understand how node positions are computed.

**Files.**

- `apps/web/` — graph page component and layout utilities.

**Instructions.**

1. Study the existing graph layout code. The default layout arranges nodes using a force-directed or hierarchical algorithm.
2. Implement a **circular layout** option that places nodes in a circle with the most-connected node at the top.
3. Add a layout toggle (dropdown or button group) to the graph page so users can switch between the existing layout and your new circular layout.
4. Ensure edges render correctly with the new positions.

▷ **Try It**

**Acceptance criteria:**

- A "Circular" option appears in the graph page layout controls.
- Selecting it repositions all nodes in a circle.
- The most-connected node (highest degree) appears at the 12 o'clock position.
- Edges remain visible and do not overlap excessively.
- Switching back to the default layout restores the original positions.
- `pnpm typecheck` passes with no errors.

**Hints.**

- The circular position for node $i$ out of $N$ total is: $x = r\cos(2\pi i/N)$, $y = r\sin(2\pi i/N)$.
- Sort nodes by degree (number of connections) descending before assigning positions so the highest-degree node gets index 0.
- Use `cn()` from `@/lib/utils` for conditional class names on the toggle buttons.

## 18.3 Search Entity Type Filter

**Difficulty:** Medium **Time:** ~3 hours **Learning objective:** Add a feature that spans both backend and frontend, touching an API endpoint and a UI control.

**Files.**

- `apps/api/routers/search.py` — add query parameter.
- `apps/api/services/` — filter logic in the search service.
- `apps/web/` — search page component.

**Instructions.**

1. Add an optional `entity_type` query parameter to the search endpoint in `routers/search.py`. Accept any value from the `EntityType` enum (e.g. `technology`, `concept`).
2. Update the search service to filter results by entity type when the parameter is present. When absent, return all types (current behavior).
3. On the frontend search page, add a dropdown (or segmented control) that lists all entity types plus an "All" option.
4. Wire the dropdown selection into the search API call as a query parameter.
5. Test with several entity types and verify the result count changes.

**▷ Try It**

**Acceptance criteria:**

- `GET /api/search/?q=react&entity_type=technology` returns only technology entities.
- `GET /api/search/?q=react` (no filter) returns all types.
- The frontend dropdown updates search results in real time.
- Invalid entity types return a 422 validation error.
- `.venv/bin/pytest tests/ -v -k search` passes.
- `pnpm typecheck` passes.

**Hints.**

- Use `Optional[EntityType] = None` as the FastAPI parameter type for automatic enum validation.
- The Neo4j query likely uses a `WHERE` clause on node labels or a `type` property—add a conditional filter there.
- On the frontend, use React Query's `queryKey` to include the entity type so cache invalidation works correctly when the filter changes.

## 18.4 New Evaluation Baseline

**Difficulty:** Medium **Time:** ∼4 hours **Learning objective:** Implement a comparison baseline for entity resolution using traditional NLP techniques.

**Files.**

- `apps/api/evaluation/tfidf_baseline.py` — new file.
- `apps/api/evaluation/data/v5/` — output directory.

**Instructions.**

1. Create `evaluation/tfidf_baseline.py`.
2. Load the synthetic benchmark data (the CSV produced by `synthetic_benchmark.py`).
3. For each surface-form variant, compute its TF-IDF vector using scikit-learn's `TfidfVectorizer` with character n-grams (`analyzer="char_wb"`, `ngram_range=(2,4)`).
4. Build a TF-IDF matrix for all canonical names.
5. For each test variant, find the nearest canonical name by cosine similarity and predict that as the resolution.
6. Compute precision, recall, and F1 against the ground-truth canonical labels.
7. Write results to `evaluation/data/v5/tfidf_baseline_results.json`.

**▷ Try It**

**Acceptance criteria:**

- The script runs end-to-end: `python -m evaluation.tfidf_baseline`.
- Output JSON contains `precision`, `recall`, and `f1`.
- Results are computed on the held-out test split only (60 conversations, not all 200).
- The script prints a summary comparing TF-IDF F1 against Continuum's pipeline F1 (0.979 from B-cubed evaluation).

**Hints.**

- Install scikit-learn: `.venv/bin/pip install scikit-learn`.

- Character n-grams work better than word n-grams for entity name matching because many tech names are single tokens.
- Use `cosine_similarity` from `sklearn.metrics.pairwise` for efficient batch comparison.
- The existing benchmark CSV has a `split` column (`train`/`test`) to identify held-out data.

## 18.5 GraphRAG Depth Experiment

**Difficulty:** Medium–Hard **Time:** ~4 hours **Learning objective:** Understand how graph traversal depth affects retrieval quality in the GraphRAG pipeline.

**Files.**

- `apps/api/services/graph_rag.py` — retrieval depth parameter.
- `apps/api/evaluation/` — new experiment script.

**Instructions.**

1. Study `services/graph_rag.py` to understand how the retrieval pipeline expands from seed entities through graph neighbors.
2. Create an experiment script that runs the same set of 10 queries at depths 1, 2, and 3.
3. For each depth, record: number of nodes retrieved, number of unique entity types, and the context string length.
4. Manually judge relevance of the top-5 retrieved nodes for each query at each depth (use a simple 0/1/2 scale: irrelevant, partially relevant, highly relevant).
5. Compute average relevance scores per depth and plot the trade-off between depth and relevance.

▷ **Try It**

**Acceptance criteria:**

- The experiment script produces a JSON file with per-query, per-depth results.
- At least 10 queries are tested at all three depths.
- A summary table shows: depth, avg nodes retrieved, avg relevance, avg context length.
- The script includes a brief analysis comment interpreting whether deeper traversal helps or hurts retrieval quality.

**Hints.**

- The `retrieve_context()` method in `GraphRAGService` returns `(subgraph, context_str, seed_ids)`. The subgraph contains the retrieved nodes.
- Depth 1 retrieves direct neighbors; depth 2 retrieves neighbors-of-neighbors.
- You will likely find that depth 2 is the sweet spot—depth 3 tends to bring in too much noise for most queries.
- The existing `evaluation/test_graphrag.py` has example queries you can reuse.

## 18.6 New MCP Tool

**Difficulty:** Hard **Time:** ~4 hours **Learning objective:** Extend the Model Context Protocol server with a new tool and write automated test cases.

**Files.**

- `apps/mcp/` — MCP server tool definitions.
- `apps/api/evaluation/test_mcp.py` — MCP test suite.

**Instructions.**

1. Design a `continuum_timeline` MCP tool that accepts an entity name and returns a chronological list of decisions involving that entity.
2. The tool should query the Neo4j graph for all `DecisionTrace` nodes connected to the entity via `INVOLVES` relationships, sorted by timestamp.
3. Each entry in the returned list should include: decision summary, timestamp, confidence score, and any `SUPERSEDES`/`CONTRADICTS` relationships to other decisions.
4. Implement the tool in the MCP server following the patterns of existing tools.
5. Write at least 5 test cases covering:
   - Entity with no decisions (empty result).
   - Entity with one decision (single entry).
   - Correct chronological ordering (oldest first).
   - Entity with a `SUPERSEDES` chain.
   - Entity with a `CONTRADICTS` pair.

▷ **Try It**

**Acceptance criteria:**

- The `continuum_timeline` tool appears in the MCP server's tool list.
- Calling it with `"PostgreSQL"` returns a chronological list of decisions.
- The response includes `SUPERSEDES`/`CONTRADICTS` metadata when present.
- All 5 test cases pass when running the MCP test suite.
- The tool handles non-existent entity names gracefully (returns empty list, not an error).

**Hints.**

- Study the existing MCP tools in `apps/mcp/` for the registration pattern and response format.
- The Cypher query will need to `MATCH` the entity, traverse `INVOLVES` relationships to decision nodes, and optionally `MATCH` inter-decision relationships.
- Use `ORDER BY d.created_at ASC` (or the equivalent timestamp property) for chronological sorting.
- The existing `evaluation/test_mcp.py` has 31 test cases that serve as a template for test structure.

# Chapter 19

# Starter Project

*This chapter describes "Decision Diff," a one-week onboarding project that touches every layer of the Continuum stack. You will build a timeline view showing how decisions about a given entity evolve over time, including when later decisions supersede or contradict earlier ones. Completing this project means you understand the full stack well enough to contribute independently.*

**⋆ Key Insight**

This project touches every layer: Neo4j queries, a FastAPI endpoint, Pydantic models, a React component, and an evaluation harness. Completing it means you understand how data flows from the graph database to the user's screen.

## 19.1 Overview

The **Decision Diff** feature adds a timeline view for any entity in the knowledge graph. When a user clicks on an entity (say "PostgreSQL"), they see a vertical timeline of every decision that mentions it, ordered chronologically. Relationships between decisions—`SUPERSEDES` and `CONTRADICTS`—are shown as colored badges connecting timeline entries.

This is useful because it answers the question: *"How has our thinking about this technology changed over time?"* A team might start by choosing PostgreSQL for everything, later decide to add Redis for caching, and eventually supersede the original decision when they migrate to a managed database. Decision Diff makes that evolution visible.

**Suggested timeline:**

| Phase | Days |
|---|---|
| Phase 1: Backend | Days 1–2 |
| Phase 2: Frontend | Days 3–4 |
| Phase 3: Evaluation | Day 5 |

## 19.2 Phase 1: Backend (Days 1–2)

### 19.2.1 Neo4j Query

Write a Cypher query that, given an entity name, returns all decisions connected to it via `INVOLVES` relationships, along with any `SUPERSEDES` or `CONTRADICTS` edges between those decisions.

Entity timeline query

```
MATCH (e:Entity {name: $entity_name})<-[:INVOLVES]-(d:DecisionTrace)
OPTIONAL MATCH (d)-[r:SUPERSEDES|CONTRADICTS]->(other:DecisionTrace)
```

```
WHERE other IN collect(d)
RETURN d {
    .id, .decision, .rationale, .confidence,
    .created_at, .context
} AS decision,
collect(DISTINCT {
    type: type(r),
    target_id: other.id,
    target_decision: other.decision
}) AS relationships
ORDER BY d.created_at ASC
```

**△ Warning**

Always use `parameters={}` dict for Neo4j query parameters—never pass `query=` as a keyword argument. It conflicts with the positional Cypher string argument and causes a confusing runtime error.

### 19.2.2 Pydantic Models

Define request/response models for the timeline endpoint:

Timeline data models

```python
from datetime import datetime
from pydantic import BaseModel

class DecisionRelationship(BaseModel):
    """A relationship from this decision to another."""
    type: str           # "SUPERSEDES" or "CONTRADICTS"
    target_id: str
    target_decision: str

class TimelineEntry(BaseModel):
    """A single decision in the entity timeline."""
    id: str
    decision: str
    rationale: str | None = None
    confidence: float
    created_at: datetime
    context: str | None = None
    relationships: list[DecisionRelationship] = []

class EntityTimelineResponse(BaseModel):
    """Complete timeline for an entity."""
    entity_name: str
    total_decisions: int
    timeline: list[TimelineEntry]
```

### 19.2.3 Router

Create the endpoint in a new router file or add it to the existing entities router:

Timeline endpoint

```python
@router.get(
    "/{entity_id}/timeline",
    response_model=EntityTimelineResponse,
)
async def get_entity_timeline(
    entity_id: str,
    user_id: str = Depends(get_current_user_id),
):
    """Return chronological decision timeline for an entity."""
    # 1. Look up entity by ID (with user_id filtering)
    # 2. Run the Cypher query above
    # 3. Map results to TimelineEntry models
    # 4. Return EntityTimelineResponse
    ...
```

Remember to register the router in `main.py` if you created a new file (see Recipe 17.4 in Chapter 17).

## 19.3 Phase 2: Frontend (Days 3–4)

Build an `EntityTimeline.tsx` component that renders the backend response as a vertical timeline.

### 19.3.1 Component Structure

EntityTimeline.tsx (skeleton)

```tsx
"use client";

import { cn } from "@/lib/utils";

interface TimelineEntry {
  id: string;
  decision: string;
  rationale: string | null;
  confidence: number;
  created_at: string;
  relationships: {
    type: "SUPERSEDES" | "CONTRADICTS";
    target_id: string;
    target_decision: string;
  }[];
}

interface EntityTimelineProps {
  entityName: string;
  entries: TimelineEntry[];
}

export function EntityTimeline({
  entityName,
  entries,
}: EntityTimelineProps) {
```

```
  return (
    <div className={cn("relative space-y-6 pl-8")}>
      {/* Vertical line */}
      <div className={cn(
        "absolute left-3 top-0 bottom-0 w-px",
        "bg-border"
      )} />

      {entries.map((entry, i) => (
        <TimelineCard
          key={entry.id}
          entry={entry}
          isLatest={i === entries.length - 1}
        />
      ))}
    </div>
  );
}
```

### 19.3.2 Design Guidelines

- Use **glass-style cards** consistent with the existing Continuum design system (see `apps/web/CLAUDE.md` for the Nebula theme tokens).
- Use `cn()` from `@/lib/utils` for all className composition.
- Use semantic color tokens (`bg-primary`, `text-foreground`) rather than hardcoded color values.
- Relationship badges should be color-coded:
  - `SUPERSEDES` — amber/warning color (this decision replaced an older one).
  - `CONTRADICTS` — red/destructive color (these decisions conflict).
- Entity type colors should match the existing entity badge colors used elsewhere in the application.
- Each card should show: decision text, confidence as a subtle badge, formatted timestamp, and any relationship badges linking to other cards in the timeline.
- Icons should come from `lucide-react` exclusively.

## 19.4 Phase 3: Evaluation (Day 5)

Write 5 test cases that verify the timeline feature works correctly. These can be unit tests (mocked Neo4j), integration tests (real database), or both.

For each test case, verify both the API response structure (correct HTTP status code, correct JSON shape) and the semantic correctness (right number of entries, correct ordering, correct relationship metadata).

## 19.5 Definition of Done

The starter project is complete when all of the following are true:

1. The Neo4j query returns correct results for entities with 0, 1, and many decisions.
2. `GET /api/entities/{id}/timeline` returns a well-formed `EntityTimelineResponse`.
3. The endpoint filters by `user_id` (per-user data isolation).
4. The `EntityTimeline.tsx` component renders a vertical timeline with decision cards.

Table 19.1: Timeline test cases.

| # | Scenario | Expected behavior |
|---|---|---|
| 1 | Zero decisions | Entity exists but has no decisions. Returns empty timeline with `total_decisions: 0`. |
| 2 | Single decision | Entity with one decision. Returns a timeline with exactly one entry and no relationships. |
| 3 | Chronological order | Three decisions created at different times. Timeline entries are sorted oldest to newest. |
| 4 | SUPERSEDES chain | Decision B supersedes Decision A. B's entry includes a `SUPERSEDES` relationship pointing to A's ID. |
| 5 | CONTRADICTS pair | Decisions C and D contradict each other. Both entries include `CONTRADICTS` relationships pointing to each other. |

5. `SUPERSEDES` and `CONTRADICTS` badges display correctly with appropriate colors.
6. All 5 test cases pass.
7. `pnpm typecheck` passes with no TypeScript errors.
8. `.venv/bin/pytest tests/ -v` passes with no failures.

**⋆ Key Insight**

If you have completed all eight items above, you have built a feature that spans Cypher queries, a FastAPI endpoint, Pydantic validation, a React component, and an automated test suite. You now understand how data flows through the entire Continuum stack.

# Part VI

# Research & Future

# Chapter 20

# Evaluation Methodology

*This chapter describes how Continuum's entity resolution pipeline is evaluated: the synthetic dataset, the train/test split, the metrics, the ablation study, and reproducibility analysis. All numbers reported here are verified against the data files in `apps/api/evaluation/data/v5/`.*

## 20.1 Synthetic Dataset

The evaluation uses 200 synthetic developer–AI conversations generated by `evaluation/generate_conversations.py`. Each conversation simulates a developer discussing a technology decision with a coding assistant, and is designed to contain diverse entity surface forms: abbreviations, aliases, version-qualified names, colloquial references, and occasional misspellings.

**Dataset characteristics:**

- **200 conversations** across 9 domains: web development, backend, DevOps, ML, mobile, data engineering, systems, security, and cloud architecture.
- **3,070 entity mentions** extracted and annotated for ground-truth canonical names.
- Surface-form variants generated by `synthetic_benchmark.py` include: lowercase, uppercase, known abbreviations, version-qualified forms (e.g. "PostgreSQL 16"), misspellings (character swaps, drops, doubles), and suffix variations.

**Why synthetic?** Three reasons:

1. **Reproducibility.** Anyone can regenerate the exact same dataset from the source code, eliminating the need to share proprietary conversation logs.
2. **No IRB required.** Synthetic conversations contain no real user data, avoiding human subjects research ethics review.
3. **Domain control.** Conversations can be designed to cover specific entity types, relationship patterns, and edge cases that might be rare in organic data.

## 20.2 Train/Test Split

The 200 conversations are split into two non-overlapping sets:

- **Training set (140 conversations).** Used to build the canonical dictionary and tune resolution thresholds. The 534 canonical mappings in `ontology.py` were developed against this set.
- **Held-out test set (60 conversations).** Used to compute all reported metrics. No information from the test set was used to build or tune the resolution pipeline.

**Non-circularity.** The split is designed to prevent data leaks. The canonical dictionary was frozen before any test set evaluation. This means test-set surface forms that happen to be missing from the dictionary must be resolved by the fuzzy, embedding, or create-new stages—not by dictionary lookup. The benchmark CSV produced by `synthetic_benchmark.py` includes a `split` column (`train`/`test`) to enforce this boundary programmatically.

## 20.3 Metrics

### 20.3.1 B-cubed Precision, Recall, and F1

Entity resolution is evaluated using **B-cubed** metrics, which measure clustering quality at the level of individual mentions. For each mention $m$:

$$\text{Precision}(m) = \frac{|\text{predicted cluster}(m) \cap \text{true cluster}(m)|}{|\text{predicted cluster}(m)|} \tag{20.1}$$

$$\text{Recall}(m) = \frac{|\text{predicted cluster}(m) \cap \text{true cluster}(m)|}{|\text{true cluster}(m)|} \tag{20.2}$$

Overall precision and recall are the averages across all mentions, and F1 is their harmonic mean.

**Worked example with 5 mentions.** Suppose we have 5 mentions of two entities:

| Mention | True cluster | Predicted cluster |
|---|---|---|
| "postgres" | PostgreSQL | PostgreSQL |
| "pg" | PostgreSQL | PostgreSQL |
| "PostgreSQL" | PostgreSQL | PostgreSQL |
| "mongo" | MongoDB | MongoDB |
| "Mongo DB" | MongoDB | PostgreSQL |

The last mention is a resolution error: "Mongo DB" was incorrectly assigned to the PostgreSQL cluster.

- **Predicted PostgreSQL cluster:** {postgres, pg, PostgreSQL, Mongo DB} — 4 members.
- **True PostgreSQL cluster:** {postgres, pg, PostgreSQL} — 3 members.
- For mention "postgres": Precision = $3/4 = 0.75$, Recall = $3/3 = 1.0$.
- **Predicted MongoDB cluster:** {mongo} — 1 member.
- **True MongoDB cluster:** {mongo, Mongo DB} — 2 members.
- For mention "mongo": Precision = $1/1 = 1.0$, Recall = $1/2 = 0.5$.
  Averaging across all 5 mentions gives the final B-cubed scores.

### 20.3.2 McNemar's Test

To determine whether one pipeline configuration is statistically significantly better than another, we use **McNemar's test**. This compares paired binary outcomes (correct/incorrect) between two systems on the same test set and produces a p-value indicating whether the difference is likely due to chance.

### 20.3.3 Expected Calibration Error (ECE)

The pipeline assigns a confidence score to each entity resolution. **ECE** measures whether these confidence scores are well-calibrated: does a prediction with 90% confidence actually resolve correctly 90% of the time?

ECE is computed by binning predictions by confidence, then measuring the gap between average confidence and average accuracy within each bin:

$$\text{ECE} = \sum_{b=1}^{B} \frac{|B_b|}{N} \left|\text{acc}(B_b) - \text{conf}(B_b)\right|$$

Lower ECE indicates better calibration. A perfectly calibrated system has ECE = 0.

## 20.4 Ablation Study

The ablation study evaluates 7 pipeline configurations, each disabling one stage of the entity resolution cascade to measure its contribution:

Table 20.1: Ablation configurations.

| Configuration | Description |
|---|---|
| Full pipeline | All 7 stages enabled (baseline). |
| No cache | Disable Redis cache lookup (stage 1). |
| No exact match | Disable case-insensitive exact match (stage 2). |
| No canonical | Disable canonical dictionary lookup (stage 3). |
| No alias | Disable alias search (stage 4). |
| No fuzzy | Disable fuzzy matching with rapidfuzz (stage 5). |
| No embedding | Disable embedding similarity (stage 6). |

The ablation script (`evaluation/ablation.py`) uses monkeypatching to disable individual stages. When disabling a stage, the pipeline falls through to later stages, measuring how much accuracy each stage contributes.

**△ Warning**

When monkeypatching for ablation, the cache mock needs all three methods (`get_by_exact_name`, `set_by_exact_name`, `set_by_alias`). The fuzzy stage mock must return `None` (not a tuple). The embedding stage is disabled by replacing the embedding service entirely.

## 20.5 Reproducibility

Because the pipeline uses an LLM (NVIDIA Nemotron) for decision extraction, results are inherently non-deterministic even with a fixed temperature setting. We ran the full extraction pipeline twice on the same 200 conversations to measure variance.

**Per-conversation agreement.** Of 200 conversations:

- **111 (55.5%)** produced exactly the same number of decisions in both runs.
- **155 (77.5%)** were within ±1 decision of each other.

The 77.5% within-±1 rate suggests that while the LLM introduces meaningful variance at the individual conversation level, aggregate metrics are reasonably stable across runs. This is a known property of LLM-based extraction: the system is stochastic, not deterministic.

Table 20.2: Run 1 vs. Run 2 comparison.

| Metric | Run 1 | Run 2 | Delta |
|---|---|---|---|
| Total decisions | 364 | 383 | +19 |
| Extraction rate | 82.5% | 85.5% | +3.0% |
| Conversations with decisions | 165 | 171 | +6 |
| Avg decisions per conv | 1.82 | 1.92 | +0.10 |
| Total entity links | 1,292 | 1,282 | −10 |
| Avg extraction time (s) | 28.51 | 26.8 | −1.71 |

## 20.6 Verified Numbers

The following table summarizes the key numbers from Run 2 of the evaluation pipeline, verified against the source data files in `evaluation/data/v5/`.

Table 20.3: Verified evaluation statistics (Run 2).

| Metric | Value | Source file |
|---|---|---|
| Conversations | 200 | `e2e_run2_results.json` |
| With decisions | 171 (85.5%) | `e2e_run2_results.json` |
| Total decisions | 383 | `e2e_run2_results.json` |
| Avg decisions/conv | 1.92 | `e2e_run2_results.json` |
| Total entities | 847 | `graph_topology_run2.json` |
| Total relationships | 1,271 | `graph_topology_run2.json` |
| Avg confidence | 0.945 | `graph_topology_run2.json` |
| Avg extraction time (s) | 26.8 | `e2e_run2_results.json` |
| Canonical mappings | 534 | `ontology.py` |
| B-cubed Precision | 0.975 | `bcubed_results.json` |
| B-cubed Recall | 0.984 | `bcubed_results.json` |
| B-cubed F1 | 0.979 | `bcubed_results.json` |
| Mentions evaluated | 3,070 | `bcubed_results.json` |

## 20.7 Limitations of the Evaluation

Two significant limitations affect the evaluation methodology:

1. **Synthetic-only data.** All 200 conversations are synthetically generated, not captured from real developer interactions. Synthetic conversations may not exhibit the full range of ambiguity, typos, context-switching, and implicit references found in organic conversations. Metrics computed on synthetic data are likely *optimistic* compared to what would be observed on real data.
2. **Single primary annotator.** Ground-truth labels for entity mentions were produced by one annotator (the system developer). This introduces potential bias, particularly for ambiguous cases where the annotator's domain knowledge may influence labeling decisions. Inter-annotator agreement was not measured. Ideally, at least two independent annotators would label the data, with disagreements resolved by a third.

These limitations are discussed further in Chapter 21, which covers the broader set of constraints affecting the system as a whole.

# Chapter 21

# Known Limitations

*Every research system has limitations. This chapter documents seven known limitations of Continuum, with evidence, impact assessment, and potential mitigations for each. These are not bugs to fix—they are boundaries of the current design that constrain what conclusions can be drawn from the system and its evaluation.*

**⋆ Key Insight**

Every limitation listed here is a potential research contribution for a future student. Addressing even one of these would strengthen the system significantly and could constitute a publishable result.

## 21.1 Synthetic-Only Evaluation

**Description.** All evaluation data comes from 200 synthetically generated conversations. No real developer–AI conversations were used for evaluation.

**Evidence.** The conversations in `evaluation/generate_conversations.py` are hand-authored templates that follow predictable patterns: a developer asks about a technology choice, the assistant presents options, and they converge on a decision. Real conversations are messier—they include tangents, incomplete thoughts, code blocks, and implicit decisions that are never explicitly stated.

**Impact.** Reported metrics (B-cubed F1 = 0.979) are likely *optimistic*. Entity mentions in synthetic data use well-known surface forms that the canonical dictionary was designed to handle. Real conversations may contain novel abbreviations, domain-specific jargon, or typos that fall outside the dictionary's coverage.

**Potential mitigation.** Collect real conversations from open-source projects that use AI coding assistants. Several open-source teams publish their Claude Code or Copilot Chat logs. These could be anonymized and used as an evaluation corpus (see Chapter 22, Section 22.1).

## 21.2 No User Study

**Description.** Continuum's core value proposition—that a decision knowledge graph improves developer decision-making—has not been validated with actual users.

**Evidence.** The system has been evaluated purely on technical metrics: entity resolution accuracy, extraction success rates, and graph topology statistics. There is no data on whether developers who use Continuum make better, faster, or more consistent decisions compared to developers who do not.

**Impact.** Without user validation, the system may solve a problem that does not exist in practice, or solve it in a way that does not match developer workflows. Technical correctness does not imply practical utility.

**Potential mitigation.** Conduct a controlled user study with two groups: one using Continuum and one without. Measure decision consistency, time to find relevant prior decisions, and subjective usefulness ratings.

## 21.3 Weak Baselines

**Description.** The entity resolution pipeline is not compared against strong published baselines such as BLINK, EntQA, or fine-tuned BERT-based entity linkers.

**Evidence.** The current evaluation compares the full pipeline against ablated versions of itself (removing one stage at a time). While this demonstrates the value of each pipeline stage, it does not show whether the pipeline as a whole is competitive with state-of-the-art entity linking systems.

**Impact.** The high B-cubed F1 (0.979) may reflect the relative simplicity of the synthetic evaluation data rather than genuine superiority of the approach. A strong baseline would contextualize this number and reveal whether the 7-stage cascade is actually necessary.

**Potential mitigation.** Implement at least one neural baseline (e.g. fine-tuned BERT for entity matching) and one traditional baseline (e.g. TF-IDF cosine similarity, as described in Exercise 18.4). Compare all systems on the same held-out test set.

## 21.4 Two Dormant Stages

**Description.** Two of the seven entity resolution stages—alias search (stage 4) and embedding similarity (stage 6)—rarely trigger during evaluation.

**Evidence.** The pipeline benchmark shows that the vast majority of mentions are resolved by exact match (stage 2), canonical lookup (stage 3), or fuzzy matching (stage 5). Alias search and embedding similarity contribute marginal improvements. The evaluation data does not generate enough hard cases to exercise these stages meaningfully.

**Impact.** These stages add code complexity and runtime cost (embedding lookups require vector similarity search) without demonstrable benefit on the current evaluation data. They may be valuable on harder, real-world data, but this has not been shown.

**Potential mitigation.** Design a targeted experiment with adversarial entity mentions that are specifically crafted to bypass the first four stages. If the dormant stages resolve these correctly, their value is demonstrated. If not, consider simplifying the pipeline (see Chapter 22).

## 21.5 Single-Project Scope

**Description.** Continuum stores decisions within a single project context. There is no mechanism for cross-project knowledge transfer or sharing.

**Evidence.** The data model uses `user_id` and `project_id` filtering on all queries. Entities and decisions from one project are invisible to another, even when they refer to the same technologies.

**Impact.** An organization using PostgreSQL across 10 projects would build 10 separate, redundant knowledge graphs. Decisions made in one project (e.g. "we switched from REST to GraphQL because of N+1 query problems") cannot inform similar decisions in other projects.

**Potential mitigation.** Implement cross-project entity alignment, where entities with the same canonical name across projects are linked at the graph level. This would enable queries like "what has our organization decided about PostgreSQL?" across all projects.

## 21.6 LLM Dependency

**Description.** Decision extraction quality is tightly coupled to the specific LLM used (NVIDIA Nemotron). Results may differ substantially with a different model.

**Evidence.** The reproducibility analysis (Chapter 20, Section 20.5) shows that even the *same* model produces different results across runs: 364 decisions in Run 1 vs. 383 in Run 2. Switching to a different model (e.g. Claude, GPT-4) would introduce additional variation from differences in instruction-following behavior, JSON formatting reliability, and domain knowledge.

**Impact.** The system's extraction quality is not a property of the pipeline alone—it is a property of the pipeline *plus* the specific model. Users who substitute a weaker model may see significant quality degradation. Users who substitute a stronger model may see improvements that the current evaluation does not predict.

**Potential mitigation.** Run the evaluation with at least 3 different LLMs (e.g. Nemotron, Claude, GPT-4) and report per-model metrics. This would disentangle pipeline quality from model quality and help users choose an appropriate model for their use case.

## 21.7 Scale Untested

**Description.** The system has been tested with approximately 200 conversations, 383 decisions, and 847 entities. Performance at larger scales is unknown.

**Evidence.** All evaluation data fits comfortably in memory. The Neo4j graph has ∼1,230 nodes and ∼1,271 relationships—well within the range where brute-force operations are fast. Entity resolution performs linear scans in several stages. The embedding similarity stage uses exact cosine similarity, not approximate nearest neighbors.

**Impact.** A production deployment with thousands of conversations and tens of thousands of entities may encounter:

- Slow entity resolution due to linear scans in the fuzzy matching stage.
- High memory usage for the embedding index.
- Slow graph traversal in the GraphRAG pipeline as subgraph expansion retrieves increasingly large neighborhoods.
- Neo4j query performance degradation without proper indexing.

**Potential mitigation.** Profile the system at 10× and 100× current scale. Replace exact nearest-neighbor search with approximate nearest neighbors (ANN) using FAISS or pgvector HNSW indexes. Add query result pagination and graph sampling for large neighborhoods (see Chapter 22, Section 22.2).

# Chapter 22

# Future Directions

*This chapter outlines two tiers of future work: semester-length projects (3–4 months) suitable for a graduate course or independent study, and thesis-level research directions (6–12 months) that could form the core of an MS thesis or PhD chapter. Each direction includes a problem statement, a suggested approach, and concrete deliverables or research questions.*

## 22.1 Semester Projects (3–4 months)

These projects are scoped to produce a concrete artifact and a short evaluation within one academic semester.

### 22.1.1 Real Conversation Evaluation

**Problem.** All current evaluation data is synthetic. Metrics may not generalize to real developer–AI conversations (see Section 21.1).

**Approach.**

1. Identify open-source projects that publish AI coding assistant logs (e.g. projects using Claude Code with public session logs, or GitHub Copilot Chat histories shared in issues).
2. Collect 50–100 real conversations and anonymize them: replace developer names, project names, and any PII.
3. Annotate entity mentions with ground-truth canonical names (two independent annotators, inter-annotator agreement measured with Cohen's $\kappa$).
4. Run Continuum's pipeline on the real data and compare B-cubed metrics against the synthetic results.

**Deliverables.**

- An anonymized real-conversation evaluation corpus.
- B-cubed precision/recall/F1 on real data vs. synthetic data.
- Analysis of failure modes unique to real conversations.

### 22.1.2 Stronger Baselines

**Problem.** The current evaluation lacks strong baselines (see Section 21.3).

**Approach.**

1. Implement a **BERT entity matching** baseline: fine-tune a pre-trained BERT model on the training split to predict whether two entity mentions refer to the same canonical entity.
2. Implement a **TF-IDF cosine similarity** baseline using character n-grams (see Exercise 18.4).
3. Optionally implement or adapt an existing entity linking system (e.g. BLINK, EntQA) to the technology-entity domain.
4. Evaluate all systems on the same held-out test set with B-cubed metrics and McNemar's test for statistical significance.

**Deliverables.**
- At least two baseline implementations with reproducible evaluation scripts.
- A comparison table showing precision, recall, F1, and statistical significance for each system.
- Analysis of which mention types (abbreviations, misspellings, etc.) each system handles well or poorly.

### 22.1.3 User Study

**Problem.** The practical value of a decision knowledge graph is unvalidated (see Section 21.2).

**Approach.**
1. Design a controlled experiment with two groups of developers (minimum 10 per group).
2. Both groups receive the same set of 5 decision-making tasks (e.g. "Choose a caching strategy for this microservice given the existing architecture").
3. The treatment group has access to Continuum with a pre-populated knowledge graph. The control group has access to the same information in a flat document (e.g. a README).
4. Measure: decision consistency (do team members make the same choice?), time to decision, and subjective usefulness (Likert scale survey).

**Deliverables.**
- IRB-approved study protocol.
- Quantitative comparison of decision quality and time between groups.
- Qualitative analysis of how developers use the knowledge graph (or do not use it).

### 22.1.4 Dormant Stage Analysis

**Problem.** Two pipeline stages (alias search, embedding similarity) rarely trigger on current data (see Section 21.4).

**Approach.**
1. Create an adversarial test set specifically designed to bypass the first four stages: mentions with creative abbreviations, domain-specific jargon, and ambiguous references.
2. Measure how often stages 4 and 6 activate on this harder data.
3. If they activate and resolve correctly, their value is demonstrated.
4. If they do not, propose a simplified pipeline that removes them, and compare accuracy with and without.

**Deliverables.**
- An adversarial evaluation dataset (at least 100 hard mentions).
- Per-stage activation rates on easy vs. hard data.
- A recommendation: keep, modify, or remove the dormant stages.

### 22.1.5 Confidence Calibration

**Problem.** Entity resolution confidence scores (avg 0.945) may not be well calibrated—a score of 0.95 may not mean the prediction is correct 95% of the time.

**Approach.**
1. Compute ECE on the current evaluation data using `evaluation/compute_calibration.py`.
2. If ECE is high, train a post-hoc calibration layer (e.g. Platt scaling or isotonic regression) on the training split.
3. Evaluate calibrated confidence scores on the test split.

4. Visualize reliability diagrams (expected accuracy vs. predicted confidence per bin).

**Deliverables.**
- ECE before and after calibration.
- A calibration model that can be integrated into the pipeline.
- Reliability diagrams showing calibration improvement.

## 22.2 Thesis-Level Research (6–12 months)

These directions are open-ended research problems that require sustained investigation and could produce publishable results.

### 22.2.1 Multi-Agent Knowledge Sharing via MCP

**Research question.** Can multiple AI coding agents share a decision knowledge graph through the Model Context Protocol (MCP) to improve collective decision quality?

**Hypothesis.** When multiple agents working on the same codebase share decisions through a common knowledge graph, they will make more consistent decisions and avoid contradictions compared to agents working in isolation.

**Methodology.**
1. Deploy Continuum as an MCP server accessible to multiple AI agent instances.
2. Design a multi-agent scenario: 3–5 agents working on different modules of the same project, each making technology decisions.
3. Measure decision consistency (do agents choose compatible technologies?) with and without the shared knowledge graph.
4. Analyze the knowledge graph for contradiction patterns (`CONTRADICTS` edges) and resolution strategies.

### 22.2.2 Cross-Project Knowledge Transfer

**Research question.** Can entity alignment across project knowledge graphs enable useful knowledge transfer?

**Hypothesis.** Decisions about a technology in one project (e.g. "we moved from REST to GraphQL because of N+1 query problems") are relevant to similar decisions in other projects using the same technology stack.

**Methodology.**
1. Populate Continuum with data from at least 5 different projects that share some technology overlap.
2. Implement entity alignment: match entities with the same canonical name across projects.
3. Build a cross-project query interface (e.g. "What has our organization decided about PostgreSQL?").
4. Evaluate whether cross-project context improves GraphRAG answer quality compared to single-project context.

### 22.2.3 Real-Time MCP Capture

**Research question.** Is real-time decision capture during AI-assisted development more effective than post-hoc extraction from conversation logs?

**Hypothesis.** Real-time capture preserves contextual signals (developer intent, code state at decision time) that are lost in post-hoc analysis, leading to higher-quality decision records.

**Methodology.**

1. Implement a real-time capture mode in the MCP server that intercepts decisions as they happen during coding sessions.
2. Compare decision quality (completeness of trigger, context, options, rationale) between real-time and post-hoc extraction.
3. Measure latency impact: does real-time capture add perceptible delay to the coding workflow?
4. Survey developers on the intrusiveness of real-time capture.

### 22.2.4 Privacy-Preserving Federated Graphs

**Research question.** Can organizations share decision knowledge without exposing proprietary implementation details?

**Hypothesis.** A federated learning approach—where each organization trains a local entity resolution model and shares only model updates (not raw data)—can produce a globally improved resolution model without compromising privacy.

**Methodology.**

1. Partition the evaluation data into 3–5 "organizations" with non-overlapping domain distributions.
2. Implement federated averaging for the embedding-based resolution stage: each organization trains locally, then shares embedding model weight updates.
3. Compare resolution quality of the federated model against locally-trained models and a centralized model (upper bound).
4. Analyze privacy guarantees: what information can be inferred from shared model updates?

### 22.2.5 Temporal Graph Reasoning

**Research question.** Can temporal reasoning over the decision graph answer questions about decision evolution, such as "When did we stop using technology X?" or "How has our architecture philosophy changed over the past year?"

**Hypothesis.** Adding temporal annotations (creation time, supersession time) to graph edges enables a class of useful queries that static graphs cannot support.

**Methodology.**

1. Extend the graph schema with temporal properties on all edges (valid-from, valid-to, creation timestamp).
2. Implement temporal Cypher query patterns: point-in-time snapshots, interval queries, and trend analysis.
3. Define a benchmark of 20 temporal questions and evaluate whether Continuum can answer them correctly.
4. Compare against a baseline that treats the graph as static (no temporal reasoning).

### 22.2.6 Scale and Performance

**Research question.** How does the system perform at 10×, 100×, and 1,000× the current scale (10K+ entities, 100K+ relationships)?

**Hypothesis.** The current linear-scan entity resolution and exact nearest-neighbor embedding search will become bottlenecks at scale, but can be replaced with approximate algorithms without significant accuracy loss.

**Methodology.**

1. Generate synthetic data at 10× (2,000 conversations), 100× (20,000), and 1,000× (200,000) the current dataset size.
2. Profile the full pipeline at each scale: entity resolution latency, graph query time, GraphRAG retrieval time, and memory usage.
3. Replace exact nearest-neighbor search with ANN indexes (FAISS HNSW or pgvector IVFFlat) and measure the accuracy–latency trade-off.
4. Implement graph sampling for large-neighborhood traversals in the GraphRAG pipeline.
5. Report scaling curves: latency and accuracy as functions of dataset size.

# Appendix A

# Configuration Reference

All settings are defined in `apps/api/config.py` as a Pydantic `BaseSettings` class. Every field maps to an environment variable of the same name (upper-cased). Place overrides in `.env` in the repository root or export them in your shell.

| Setting | Type | Default | Env Var | Description |
|---|---|---|---|---|
| **Database** | | | | |
| database_url | str | "" | DATABASE_URL | PostgreSQL async connection string. Auto-converts `postgresql://` to `postgresql+asyncpg://`. |
| neo4j_uri | str | "" | NEO4J_URI | Neo4j Bolt or `neo4j+s://` URI. |
| neo4j_user | str | "" | NEO4J_USER | Neo4j username. |
| neo4j_password | SecretStr | "" | NEO4J_PASSWORD | Neo4j password. Masked in logs. |
| redis_url | str | "" | REDIS_URL | Redis connection string with optional password. |
| **LLM Provider** | | | | |
| llm_provider | str | "nvidia" | LLM_PROVIDER | Active LLM backend: `"nvidia"` or `"bedrock"`. |
| embedding_provider | str | "nvidia" | EMBEDDING_PROVIDER | Embedding backend. Keep `"nvidia"` even with Bedrock LLM to avoid re-indexing. |
| **NVIDIA NIM** | | | | |
| nvidia_api_key | SecretStr | "" | NVIDIA_API_KEY | API key for NVIDIA NIM LLM endpoint. |
| nvidia_model | str | nvidia/llama-3.3-nemotron-super-49b-v1.5 | NVIDIA_MODEL | LLM model identifier. |
| nvidia_embedding_api_key | SecretStr | "" | NVIDIA_EMBEDDING_API_KEY | Separate API key for the embedding model. |
| nvidia_embedding_model | str | nvidia/llama-3.2-nv-embedqa-1b-v2 | NVIDIA_EMBEDDING_MODEL | Embedding model identifier. |
| **Amazon Bedrock** | | | | |

*Continued on next page...*

| Setting | Type | Default | Env Var | Description |
|---|---|---|---|---|
| bedrock_model_id | str | anthropic.claude-sonnet-4-20250514 | BEDROCK_MODEL_ID | Bedrock model ID for LLM calls. |
| bedrock_embedding_model_id | str | amazon.titan-embed-text-v2:0 | BEDROCK_EMBEDDING_MODEL_ID | Bedrock embedding model. |
| aws_region | str | "us-west-2" | AWS_REGION | AWS region for Bedrock API calls. |
| **Observability** | | | | |
| dd_trace_enabled | bool | False | DD_TRACE_ENABLED | Enable Datadog LLM Observability tracing. |
| **Embedding Cache** | | | | |
| embedding_cache_ttl | int | 2592000 | EMBEDDING_CACHE_TTL | Embedding cache lifetime in seconds (30 days). |
| embedding_cache_min_text_length | int | 10 | EMBEDDING_CACHE_MIN_TEXT_LENGTH | Minimum length to cache an embedding. |
| embedding_batch_size | int | 32 | EMBEDDING_BATCH_SIZE | Texts per batch for bulk embedding calls. NVIDIA NIM supports up to 256. |
| **Rate Limiting** | | | | |
| rate_limit_requests | int | 30 | RATE_LIMIT_REQUESTS | Maximum API requests per window. |
| rate_limit_window | int | 60 | RATE_LIMIT_WINDOW | Rate-limit window in seconds. |
| **LLM Retry** | | | | |
| llm_max_retries | int | 3 | LLM_MAX_RETRIES | Maximum retry attempts with exponential backoff. |
| llm_retry_base_delay | float | 1.0 | LLM_RETRY_BASE_DELAY | Base delay in seconds for exponential backoff. |
| **LLM Prompt** | | | | |
| max_prompt_tokens | int | 70000 | MAX_PROMPT_TOKENS | Maximum input tokens (Nemotron 128k context). |
| prompt_warning_threshold | float | 0.8 | PROMPT_WARNING_THRESHOLD | Warn if prompt exceeds this fraction of max. |
| **LLM Cache** | | | | |
| llm_cache_enabled | bool | True | LLM_CACHE_ENABLED | Toggle Redis-backed LLM response caching. |

*Continued on next page...*

| Setting | Type | Default | Env Var | Description |
|---|---|---|---|---|
| llm_cache_ttl | int | 86400 | LLM_CACHE_TTL | LLM cache lifetime in seconds (24 hours). |
| llm_extraction_prompt_version | str | "v1" | LLM_EXTRACTION_PROMPT_VERSION | Bump to invalidate cached extraction results. |
| llm_fallback_model | str | nvidia/llama-3.1-nemotron-70b-instruct | LLM_FALLBACK_MODEL | Secondary model used on primary 503 failures. |
| llm_fallback_enabled | bool | True | LLM_FALLBACK_ENABLED | Enable automatic fallback to secondary model. |
| **Entity Cache** | | | | |
| entity_cache_ttl | int | 300 | ENTITY_CACHE_TTL | Entity resolution cache lifetime (5 minutes). |
| entity_cache_enabled | bool | True | ENTITY_CACHE_ENABLED | Toggle entity resolution caching. |
| **Similarity Thresholds** | | | | |
| similarity_threshold | float | 0.85 | SIMILARITY_THRESHOLD | Min cosine similarity for SIMILAR_TO edges. |
| high_confidence_similarity_threshold | float | 0.90 | HIGH_CONFIDENCE_SIMILARITY_THRESHOLD | Auto-link confidence matches. |
| fuzzy_match_threshold | float | 0.85 | FUZZY_MATCH_THRESHOLD | Min string similarity threshold (0–1). |
| embedding_similarity_threshold | float | 0.90 | EMBEDDING_SIMILARITY_THRESHOLD | Min embedding similarity for entity resolution. |
| **Embedding Weights** | | | | |
| decision_embedding_weight_title | float | 1.5 | DECISION_EMBEDDING_WEIGHT_TITLE | Embedding weight for the title field. |
| decision_embedding_weight_decision | float | 1.2 | DECISION_EMBEDDING_WEIGHT_DECISION | Embedding weight for the decision field. |
| decision_embedding_weight_rationale | float | 1.0 | DECISION_EMBEDDING_WEIGHT_RATIONALE | Embedding weight for the rationale field. |
| decision_embedding_weight_context | float | 0.8 | DECISION_EMBEDDING_WEIGHT_CONTEXT | Embedding weight for the context field. |
| decision_embedding_weight_trigger | float | 0.8 | DECISION_EMBEDDING_WEIGHT_TRIGGER | Embedding weight for the trigger field. |
| **Authentication** | | | | |
| secret_key | SecretStr | "" | SECRET_KEY | JWT signing key. **Must match** NEXTAUTH_SECRET. |
| algorithm | str | "HS256" | ALGORITHM | JWT signing algorithm. |

*Continued on next page...*

| Setting | Type | Default | Env Var | Description |
|---|---|---|---|---|
| access_token_expire_minutes | int | 30 | ACCESS_TOKEN_EXPIRE_MINUTES | JWT token lifetime in minutes. |
| **App** | | | | |
| debug | bool | False | DEBUG | Enable debug mode and verbose logging. |
| cors_origins | list[str] | ["http://localhost:3000"] | CORS_ORIGINS | Allowed CORS origins for the frontend. |

**⋆ Key Insight**

Settings whose type is `SecretStr` (neo4j_password, nvidia_api_key, nvidia_embedding_api_key, secret_key) are automatically masked in log output and `repr()` calls. Access their values through the corresponding getter methods, e.g. `settings.get_nvidia_api_key()`.

# Appendix B

# API Reference

The Continuum backend exposes a REST API at `http://localhost:8000`. Interactive Swagger documentation is available at `/docs`. All endpoints requiring authentication expect a `Bearer` token in the `Authorization` header. Unauthenticated requests fall back to the `anonymous` user scope.

## B.1 Authentication

| Method | Path | Auth | Description |
|---|---|---|---|
| POST | `/api/users/register` | No | Register a new user account. |
| POST | `/api/users/login` | No | Authenticate and receive user data. |
| GET | `/api/users/me` | Yes | Get current user profile with stats. |
| DELETE | `/api/users/me` | Yes | Delete account and all associated data. |

## B.2 Decisions

| Method | Path | Auth | Description |
|---|---|---|---|
| GET | `/api/decisions` | Optional | List decisions (paginated: `limit`, `offset`). |
| POST | `/api/decisions` | Optional | Create a decision with auto entity extraction. |
| GET | `/api/decisions/{id}` | Optional | Get a single decision by ID. |
| PUT | `/api/decisions/{id}` | Optional | Update decision fields. |
| DELETE | `/api/decisions/{id}` | Optional | Delete a decision (preserves entities). |
| GET | `/api/decisions/needs-review` | Optional | Decisions missing human rationale. |

## B.3 Search

| Method | Path | Auth | Description |
|---|---|---|---|
| GET | `/api/search` | No | Search decisions and entities. |
| GET | `/api/search/suggest` | No | Autocomplete suggestions. |

**Query parameters for `/api/search`:**

`query`
: Search string (min 2 chars, required).

`type`
: Filter by `decision` or `entity`.

`expand`
: Enable graph expansion on results (`true`/`false`).

`depth`
Expansion depth when `expand=true` (1–3, default 1).

## B.4 Ask (GraphRAG)

| Method | Path | Auth | Description |
|---|---|---|---|
| GET | `/api/ask` | Optional | Stream a graph-grounded answer (SSE). |

**Query parameters:**
`q` The question to ask (min 3 chars, required).
`depth`
Graph traversal depth (1–3, default 2).
`top_k`
Number of seed nodes to retrieve (1–10, default 5).

## B.5 Graph

| Method | Path | Auth | Description |
|---|---|---|---|
| GET | `/api/graph` | Optional | Paginated knowledge graph. |
| GET | `/api/graph/all` | Optional | Full graph (use sparingly). |
| GET | `/api/graph/stats` | Optional | Node/edge/relationship counts. |
| GET | `/api/graph/nodes/{id}` | Optional | Single node by ID. |
| GET | `/api/graph/nodes/{id}/neighbors` | Optional | Neighbors of a node. |
| GET | `/api/graph/nodes/{id}/similar` | Optional | Semantically similar decisions. |
| GET | `/api/graph/validate` | Optional | Run graph validation checks. |
| POST | `/api/graph/search/hybrid` | Optional | Hybrid lexical+semantic search. |
| POST | `/api/graph/search/semantic` | Optional | Pure vector similarity search. |
| POST | `/api/graph/analyze-relationships` | Optional | Discover supersedes/contradicts edges. |
| GET | `/api/graph/relationships/types` | Optional | List all relationship types. |
| GET | `/api/graph/sources` | Optional | List decision sources. |
| GET | `/api/graph/projects` | Optional | List projects in the graph. |
| DELETE | `/api/graph/reset` | Optional | Wipe all graph data (dangerous). |

## B.6 Entities

| Method | Path | Auth | Description |
|---|---|---|---|
| GET | `/api/entities` | Optional | List entities linked to user's decisions. |
| POST | `/api/entities` | Optional | Create or deduplicate an entity. |
| GET | `/api/entities/{id}` | Optional | Get entity by ID. |
| PUT | `/api/entities/{id}` | Optional | Update entity name/type. |
| DELETE | `/api/entities/{id}` | Optional | Delete entity (with force option). |
| POST | `/api/entities/link` | Optional | Link entity to a decision. |
| POST | `/api/entities/suggest` | Optional | Suggest entities from text. |

## B.7 Capture

| Method | Path | Auth | Description |
|---|---|---|---|
| POST | `/api/capture/sessions` | Optional | Start a new capture session. |
| GET | `/api/capture/sessions/{id}` | Optional | Get session with messages. |
| POST | `/api/capture/sessions/{id}/messages` | Optional | Send a message, get AI reply. |
| POST | `/api/capture/sessions/{id}/complete` | Optional | Complete session, save decision. |
| WS | `/api/capture/sessions/{id}/ws` | No | Real-time streaming capture. |

## B.8 Agent (MCP)

These endpoints provide structured knowledge graph access for AI coding agents (Claude Code, Cursor, etc.) via the Model Context Protocol.

| Method | Path | Auth | Description |
|---|---|---|---|
| GET | `/api/agent/summary` | Optional | High-level project overview. |
| POST | `/api/agent/context` | Optional | Focused context via hybrid search. |
| GET | `/api/agent/context/{name}` | Optional | Everything about one entity. |
| POST | `/api/agent/check` | Optional | Prior-art check before deciding. |
| POST | `/api/agent/remember` | Yes | Record an agent-made decision. |

## B.9 Projects

| Method | Path | Auth | Description |
|---|---|---|---|
| GET | `/api/projects` | No | List all projects with counts. |
| GET | `/api/projects/{name}/stats` | No | Detailed project statistics. |
| DELETE | `/api/projects/{name}` | No | Delete all decisions in a project. |
| POST | `/api/projects/{name}/reset` | No | Reset project for re-import. |

## B.10 Dashboard

| Method | Path | Auth | Description |
|---|---|---|---|
| GET | `/api/dashboard/stats` | No | Aggregated dashboard statistics. |

## B.11 Export & Import

| Method | Path | Auth | Description |
|---|---|---|---|
| GET | `/api/export/export` | Optional | Export decisions as JSON. |
| GET | `/api/export/export/download` | Optional | Download decisions as JSON file. |
| POST | `/api/export/import` | Optional | Bulk import decisions (max 500). |

## B.12 Ingest

| Method | Path | Auth | Description |
|---|---|---|---|
| GET | `/api/ingest/projects` | No | List Claude log project directories. |
| GET | `/api/ingest/files` | No | List JSONL conversation files. |
| GET | `/api/ingest/preview` | No | Preview conversations before import. |
| GET | `/api/ingest/status` | No | Current ingestion status. |
| POST | `/api/ingest/trigger` | No | Start ingestion run. |
| POST | `/api/ingest/import-selected` | No | Import selected conversations. |
| GET | `/api/ingest/import/progress` | No | Poll import job progress. |
| POST | `/api/ingest/import/cancel` | No | Cancel running import job. |
| POST | `/api/ingest/watch/start` | No | Start file watcher. |
| POST | `/api/ingest/watch/stop` | No | Stop file watcher. |

## B.13 Health

| Method | Path | Auth | Description |
|---|---|---|---|
| GET | `/health` | No | Basic liveness check. |
| GET | `/health/ready` | No | Readiness probe (checks all DBs). |
| GET | `/health/live` | No | Lightweight liveness probe. |
| GET | `/health/circuits` | No | Circuit breaker status dashboard. |

## B.14 SSE Events for `/api/ask`

The `/api/ask` endpoint returns a `text/event-stream` response. Each event has an `event:` type line followed by a `data:` JSON payload.

| Event | Payload |
|---|---|
| `context` | Retrieved subgraph JSON: `{nodes, edges, seed_ids}`. Each node includes `type` (`decision`/`entity`), `is_seed`, and a `data` object with the node's properties. |
| `token` | LLM output chunk: `{text: "..."}`. Concatenate all `token` events to build the full answer. |
| `done` | Stream complete: `{token_count: N}`. |
| `error` | Error detail: `{detail: "..."}`. |

## B.15 WebSocket Protocol for Capture

Connect to `/api/capture/sessions/{id}/ws` to stream a capture session in real time. The protocol follows this sequence:

1. **Connect** — Client opens the WebSocket connection.
2. **Send message** — Client sends JSON: `{"content": "user's answer"}`.
3. **Receive chunks** — Server streams `{"type":"chunk", "content":"...", "entities":[...]}`.
4. **Completion** — Server sends `{"type":"complete"}` when the response is finished.
5. **Repeat** — Steps 2–4 repeat until the session ends.

> **△ Warning**
>
> Messages are rate-limited to 20 per minute per session. Messages exceeding 10 KB are rejected. The server trims conversation history at 50 messages.

## B.16 Example `curl` Commands

**Register a user**

```
curl -X POST http://localhost:8000/api/users/register \
  -H "Content-Type: application/json" \
  -d '{"name":"Demo","email":"ali@demo.com","password":"demo1234"}'
```

**Search decisions and entities**

```
curl "http://localhost:8000/api/search?query=postgres&type=entity"
```

**Ask a question (SSE stream)**

```
curl -N "http://localhost:8000/api/ask?q=Why+did+we+choose+Redis&depth
    =2&top_k=5" \
  -H "Authorization: Bearer $TOKEN"
```

# Appendix C

# Glossary

**Ablation study**
: An experiment that disables one pipeline stage at a time to measure its individual contribution. (Chapter **??**)

**B-cubed metrics**
: Precision, recall, and F1 scores computed per mention and averaged, used to evaluate entity-resolution quality. (Chapter **??**)

**Canonical mapping**
: A deterministic lookup table (534 entries) that maps common aliases to their canonical entity name, e.g. `postgres` → `PostgreSQL`. (Chapter 2)

**Circuit breaker**
: A resilience pattern that trips after repeated failures (default 5), rejecting subsequent calls until a cooldown period elapses. (Chapter **??**)

**Cypher**
: Neo4j's declarative graph query language, analogous to SQL for relational databases. (Chapter **??**)

**Decision trace**
: The atomic unit of knowledge in Continuum: a structured record containing trigger, context, options, decision, rationale, and confidence. (Chapter **??**)

**Entity resolution**
: The 7-stage cascading pipeline that maps raw mention strings to canonical graph entities, preventing duplicates. (Chapter 2)

**Fulltext index**
: A Neo4j index that supports Lucene-style text search across node properties, used as the lexical leg of hybrid search. (Chapter **??**)

**GraphRAG**
: Graph-augmented Retrieval-Augmented Generation: retrieving a relevant subgraph and feeding it as context to an LLM for grounded answers. (Chapter **??**)

**Hybrid search**
: A search strategy that fuses lexical (fulltext) and semantic (vector) results using Reciprocal Rank Fusion for higher recall. (Chapter **??**)

**K-hop expansion**
: Traversing the graph $k$ edges out from seed nodes to capture surrounding context for the LLM prompt. (Chapter **??**)

**Labeled property graph**
: The Neo4j data model where nodes and edges carry labels, properties, and direction—the storage foundation of the knowledge graph. (Chapter **??**)

**MCP (Model Context Protocol)**
: An open protocol for AI agents to read and write structured context. Continuum's Agent API implements MCP-compatible endpoints. (Chapter **??**)

**Multi-tenant**
The architecture pattern where every query is scoped to a `user_id`, ensuring data isolation between accounts. (Chapter **??**)

**NV-EmbedQA**
NVIDIA's embedding model (`llama-3.2-nv-embedqa-1b-v2`) that produces 2048-dimensional vectors for semantic search. (Chapter **??**)

**Ontology**
The classification schema (530+ canonical mappings) that categorizes entities into types such as `language`, `framework`, `database`, and `concept`. (Chapter 2)

**RapidFuzz**
A fast C-extension fuzzy string matching library used in stage 5 of entity resolution for partial and token-sort matching. (Chapter 2)

**Reciprocal Rank Fusion (RRF)**
A score fusion method that combines ranked lists by summing $1/(k + \text{rank})$ across systems, used to merge lexical and semantic search results. (Chapter **??**)

**ResolvedEntity**
The dataclass returned by the entity resolver, carrying the canonical name, type, confidence, resolution method, and match metadata. (Chapter 2)

**Seed nodes**
The initial set of top-scoring nodes retrieved by hybrid search, from which $k$-hop expansion builds the context subgraph. (Chapter **??**)

**Server-Sent Events (SSE)**
A unidirectional streaming protocol over HTTP used by the `/api/ask` endpoint to deliver LLM tokens incrementally. (Appendix B)

**Subgraph**
A subset of the full knowledge graph—typically seed nodes plus their $k$-hop neighbors—serialized as `{nodes, edges}` JSON. (Chapter **??**)

**Token bucket**
The rate-limiting algorithm backed by Redis that allows burst traffic up to the bucket size, then refills at a steady rate. (Chapter **??**)

**User scoping**
The practice of adding `WHERE d.user_id = $user_id` to every Cypher query so that each user sees only their own data. (Chapter **??**)

**Vector search**
Retrieving nodes by embedding cosine similarity, the semantic leg of hybrid search. (Chapter **??**)

**WebSocket**
A full-duplex communication protocol used by the capture session endpoint for real-time interview streaming. (Appendix B)

# Appendix D

# Troubleshooting

The table below collects the most common issues encountered during development and deployment, along with their root causes and fixes.

| Symptom | Cause | Fix |
|---|---|---|
| Auth silently fails; every request resolves to "anonymous". | `NEXTAUTH_SECRET` in the frontend does not match `SECRET_KEY` in the backend. NextAuth signs JWTs with its secret; the backend verifies with `SECRET_KEY`. | Set both to the same value, then restart the frontend and backend. |
| Alembic `DuplicateTableError` on startup. | Database tables already exist but the `alembic_version` table is missing or empty. | Run `alembic stamp head` to mark the current schema version without re-running migrations. |
| Neo4j fulltext search returns zero results. | The `decision_fulltext` or `entity_fulltext` index was not created during initialization. | Check db/neo4j.py for the index creation logic. Restart the backend or manually create the index via the Neo4j Browser. |
| Decision extraction returns 0 decisions from valid conversations. | NVIDIA API key is invalid or `max_tokens` is too low (thinking tags consume tokens). | Verify `NVIDIA_API_KEY` is set and valid. Use `max_tokens=4096` or higher. |
| All data disappeared after restarting Docker. | Ran `docker-compose down -v`, which removes named volumes containing PostgreSQL, Neo4j, and Redis data. | Use `docker-compose down` **without** the `-v` flag to preserve volumes across restarts. |
| Backend ignores code changes despite `-reload`. | Stale `__pycache__` directories cause `uvicorn` to serve old bytecode. | Run: `find apps/api -name "__pycache__" -type d -exec rm -rf {} +` then restart the backend. |
| Embedding service returns `None` for all texts. | The circuit breaker tripped after 5 consecutive failures to the NVIDIA embedding API. | Check `NVIDIA_EMBEDDING_API_KEY` is valid. Restart the backend to reset the circuit breaker. Inspect `/health/circuits` for status. |

*Continued on next page...*

| Symptom | Cause | Fix |
|---|---|---|
| Rate-limited on NVIDIA NIM API (HTTP 429). | The free tier allows only 40 requests per minute. | Wait for the rate window to reset, or upgrade to a paid NVIDIA NIM plan. For bulk operations, reduce `embedding_batch_size`. |
| React Flow graph does not render (blank canvas). | `useNodesState`/`useEdgesState` hooks declared after `useCallback` that references their setters. | Move all `useNodesState`/`useEdgesState` declarations **before** any `useCallback` definitions in the component. |
| CSS colors appear wrong in light mode; dark mode looks fine. | Bare `:root` selectors apply in both light and dark mode, overriding dark-mode tokens. | Use `:root:not(.dark)` for light-mode-only CSS custom properties. |
| Radix `ScrollArea` refuses to auto-scroll to the bottom. | The Radix root element sets `overflow:hidden`, preventing programmatic scroll. | Replace `ScrollArea` with a plain `<div>` that has `overflow-y: auto`. |
| Tooltip text is truncated in the UI. | The default `max-width` on the Radix Tooltip content is too small for long text. | Add the `max-w-lg` Tailwind class to the `TooltipContent` component. |
| LLM response JSON is truncated or malformed. | The model's `<think>...</think>` tags consume output tokens, leaving insufficient room for the JSON payload. | Set `max_tokens=4096` or higher. Implement JSON repair logic that locates the last complete `}` + `]` pair. |
| Prompt sanitizer blocks evaluation conversations. | The input sanitizer flags `USER:` / `ASSISTANT:` labels in synthetic conversations as prompt injection. | Pass `sanitize_input=False` when calling the LLM on evaluation or synthetic data. |
| Multiple extraction runs show inflated decision counts. | Old decisions accumulate in Neo4j because the pipeline appends rather than replaces. | Wipe the Neo4j database before each new evaluation run using `/api/graph/reset` or the Neo4j Browser. |

# Bibliography